

HMI Framework 2.0

User Guide

Version 0.9.1

Marc Steilmann, Helmut Frericks, Stefanie Behme

Robert Bosch Car Multimedia GmbH

November 13, 2014

Pre-Release

Contents

I. Overview 6

1. Introduction 7
1.1. Objective . . . 7

II. Core Concepts 8

2. Architecture 9
2.1. Overview . . . 9
2.2. Sourcetree . . . 9
2.3. HMI-Framework . . . 10
2.3.1. Framework Components . . . 11
2.3.2. The Base Contract . . . 12
2.3.3. Extended CGI functionality . . . 13
2.4. HMI-Application basics . . . 13
2.4.1. General application setup . . . 13
2.4.2. Threads . . . 14
2.4.3. Common . . . 15
2.4.4. AppHmi_Master . . . 17
2.5. Widgets . . . 18
2.6. Screen Broker . . . 20
2.6.1. Core . . . 20
2.6.2. Plug-ins . . . 22
2.6.3. Threads . . . 25
2.6.4. Start-up . . . 25
2.6.5. Application activation . . . 26
2.6.6. Popup presentation . . . 27
2.6.7. Popup features . . . 30
2.7. Input Handling . . . 34
2.7.1. Touch . . . 35
2.7.2. Encoder . . . 36
2.7.3. Key . . . 36

3. Toolchain 38
3.1. Tools . . . 39
3.1.1. CGI-Studio . . . 39
3.1.2. VisualSTATE . . . 42
3.1.3. HMIContract . . . 44
3.1.4. Code Generators . . . 45
3.1.5. CGI Studio Tools . . . 45
3.2. Build System and Setup . . . 46
3.2.1. System Requirements . . . 46

Pre-Release

3.2.2. Installation . . . 46
3.2.3. Setting up Environment . . . 47
3.3. Simple Application Sample . . . 47
3.4. General Automation . . . 47

4. Framework Features **48**
4.1. Introduction . . . 48
4.2. Application Framework . . . 48
4.2.1. HmiAppControl Service . . . 48
4.2.2. Language Change . . . 50
4.2.3. Connection to LifeCycleManagement (LCM) . . . 51
4.2.4. Screenshot . . . 53
4.3. Audio Client . . . 53
4.3.1. Overview . . . 53
4.3.2. Audio Source Control Service . . . 53
4.3.3. Sound Settings Service . . . 54
4.4. Startup Animations . . . 55
4.5. Tracing/Debugging . . . 55
4.5.1. Tracing the framework . . . 56

III. HowTo: Hmi Application **59**

5. HowTo **60**
5.1. Main Goals . . . 60
5.1.1. Workflow . . . 60
5.2. Guidelines and Naming Conventions . . . 60
5.2.1. SourceCode . . . 60
5.2.2. HMI-Contract . . . 60
5.2.3. VisualState . . . 64
5.2.4. SceneComposer . . . 64
5.3. HMI-Contract as central development tool . . . 64
5.3.1. Motivation to have a proper contract . . . 64
5.3.2. HmiContract Editor . . . 64
5.3.3. Key features . . . 64
5.4. Setup applications . . . 68
5.4.1. Bosch Build XML . . . 69
5.4.2. Common . . . 73
5.4.3. Application . . . 76
5.4.4. AppHmi_Master . . . 79
5.4.5. Screen Broker plug-ins . . . 81
5.4.6. Guidelines . . . 81
5.5. Status Bar . . . 82
5.5.1. Status Bar in one HMI-App (layer bottom most) . . . 82
5.5.2. Status Bar in one HMI-App (layer top most) . . . 82
5.5.3. Status Bar in every HMI-App . . . 83
5.6. Popups . . . 83
5.6.1. Surface and Layer IDs . . . 83
5.6.2. HMI-Contract . . . 84
5.6.3. Popup Table . . . 84
5.6.4. State Machine . . . 84
5.7. Animations . . . 85
5.7.1. Widget Animations . . . 85

Pre-Release

5.7.2. Screen Transition Animations . . . 85
5.7.3. Application Change / Screen Broker Animations . . . 87
5.8. SCHost and Widget Reporitory . . . 87
5.9. Configuration . . . 87
5.9.1. KeyMapping . . . 88
5.9.2. Project specific base controller class . . . 88
5.10. Setup Automation . . . 89
5.10.1. The 'tools' directory . . . 89

6. Reference Applications **95**
6.1. Overview . . . 95
6.2. Applications . . . 95
6.2.1. AppHmi_Master . . . 95
6.2.2. AppHmi_Tuner . . . 95
6.2.3. AppHmi_Playground . . . 96

IV. WidgetRepository **97**

7. HMI-Base default Widgetset **98**
7.1. Introduction . . . 98
7.2. Animation Widget 2D . . . 98
7.2.1. Introduction . . . 98
7.3. BusyIndicator Widget 2D . . . 102
7.3.1. Introduction . . . 102
7.4. Button Widget 2D . . . 107
7.4.1. Introduction . . . 107
7.5. Calendar Widget 2D . . . 107
7.5.1. Introduction . . . 107
7.6. Clock Widget 2D . . . 111
7.6.1. Introduction . . . 111
7.7. Container Widget 2D . . . 112
7.7.1. Introduction . . . 112
7.8. ControlBar Widget 2D . . . 114
7.8.1. Introduction . . . 114
7.9. EditField Widget 2D . . . 115
7.9.1. Introduction . . . 115
7.10. FlexList Widget 2D . . . 116
7.11. FlipBook Widget 2D . . . 117
7.11.1. Introduction . . . 117
7.12. GlowEffect Widget 2D . . . 118
7.12.1. Introduction . . . 118
7.13. HandWritingRecognition Widget 2D . . . 119
7.13.1. Introduction . . . 119
7.14. Image Widget 2D . . . 120
7.14.1. Introduction . . . 120
7.15. IrisBargraph Widget 2D . . . 121
7.15.1. Introduction . . . 121
7.16. LaneGuidance Widget 2D . . . 123
7.16.1. Introduction . . . 123
7.17. List Widget 2D . . . 124
7.17.1. Introduction . . . 124

Pre-Release

7.18. Menu Widget 2D . . . 124
7.18.1. Introduction . . . 124
7.19. MulticolorBargraph Widget 2D . . . 125
7.19.1. Introduction . . . 125
7.20. Notebook Widget 2D . . . 126
7.20.1. Introduction . . . 126
7.21. ODR Widget 2D . . . 126
7.21.1. Introduction . . . 126
7.22. OPS Widget 2D . . . 127
7.22.1. Introduction . . . 127
7.23. ProgressBar Widget 2D . . . 127
7.23.1. Introduction . . . 127
7.24. RichTextBase Widget 2D . . . 128
7.24.1. Introduction . . . 128
7.25. Scrollable Widget 2D . . . 128
7.25.1. Introduction . . . 128
7.26. ScrollableText Widget 2D . . . 129
7.26.1. Introduction . . . 129
7.27. ScrollBar Widget 2D . . . 131
7.27.1. Introduction . . . 131
7.28. SequenceImage Widget 2D . . . 132
7.28.1. Introduction . . . 132
7.29. Slider Widget 2D . . . 132
7.29.1. Introduction . . . 132
7.30. Speller Widget 2D . . . 133
7.30.1. Introduction . . . 133
7.31. Splitter Widget 2D . . . 133
7.31.1. Introduction . . . 133
7.32. Switcher Widget 2D . . . 134
7.32.1. Introduction . . . 134
7.33. Text Widget 2D . . . 134
7.33.1. Introduction . . . 134
7.34. TextArea Widget 2D . . . 135
7.34.1. Introduction . . . 135
7.35. Timer Widget 2D . . . 136
7.35.1. Introduction . . . 136
7.36. TransitionMarker Widget 2D . . . 136
7.36.1. Introduction . . . 136
7.37. TwoDimensionSlider Widget 2D . . . 137
7.37.1. Introduction . . . 137
7.38. VideoContainer Widget 2D . . . 137
7.38.1. Introduction . . . 137

V. Appendix **140**

8. List of Terms and Abbreviations **141**

9. List of Internal Links **143**

10. List of External Links **145**

Pre-Release

Part I.

Overview

Pre-Release

1. Introduction

1.1. Objective

HMI-Base is a component dealing with HMI tool chain and framework. The aim is to provide a kind of HMI application framework and toolchain to be used on the Bosch CM Gen3 platform for project development.

Pre-Release

198

Part II.

199

Core Concepts

Pre-Release

2. Architecture

In order to have an HMI running on a device there will be a need to have a proper kind of framework to support basic functionality that has to be available in each HMI application. The framework should make life easier for the developer of an application and should support him in his daily work.

This document describes the concept and key features of the HMI framework approach used in Gen3 projects of CM-AI.

The main components as well as framework supported features are described in detail in the following sections

2.1. Overview

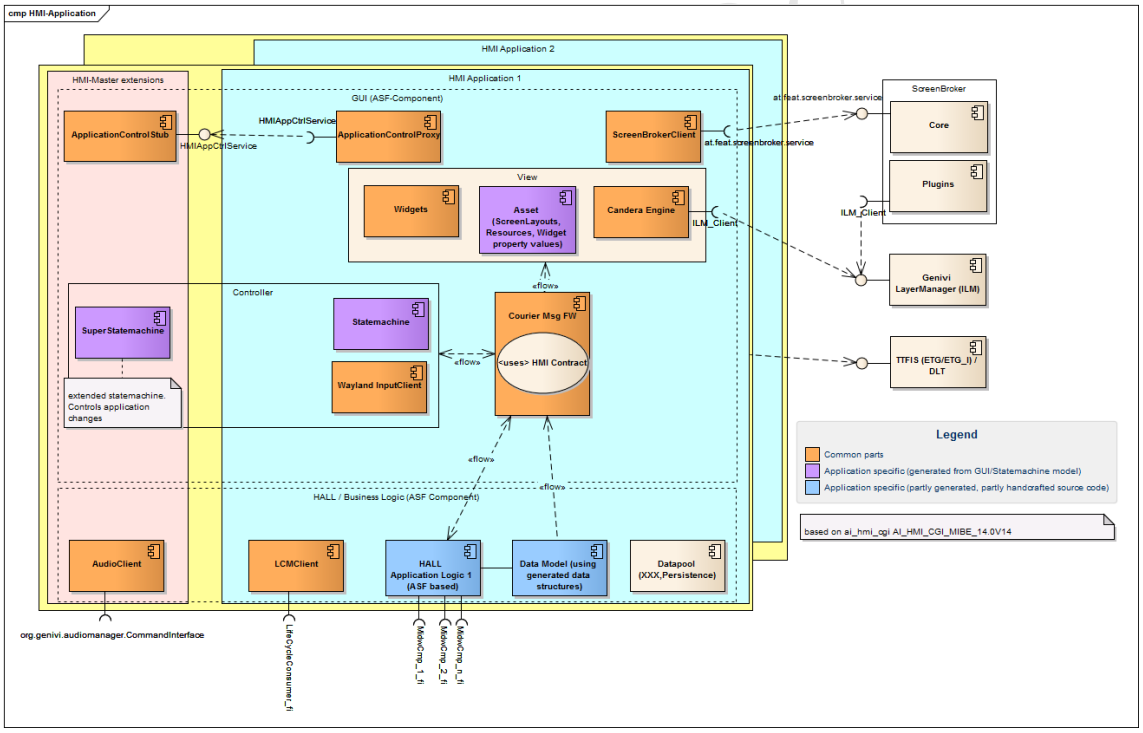


Figure 2.1.: Application architecture

2.2. Sourcetree

All framework related files are placed in the VOB(ClearCase)/Repository(git) 'ai_hmi_cgi'.

```
M:\AI_HMI_CGI
+---config
|   +---packages
|   \---xml
+---products
|   \---HMIBASE
|   |   \---Apps
+---tools
\---components
    +---CGI
    +---WidgetRepository
    +---ScreenBroker
    \---Framework
```

- config -> the configuration for buildproduct and packaging
- components/CGI -> chapter 2.3.1 Courier Messaging , chapter 2.3.1 Candera Render Engine
- components/Framework -> chapter 2.3 HMI-Framework
- components/ScreenBroker -> chapter 2.6 Screen Broker
- components/WidgetRepository -> chapter 2.5 Widgets
- products/HMIBASE/Apps -> find the reference apps here, chapter 6 Reference Applications
- tools -> chapter 3 Toolchain , chapter 3.4 General Automation

2.3. HMI-Framework

An HMI application is based on an HMI framework, which provides basic mechanisms and priciples. Main aims of the HMI framework are

- Communication mechanism for inner application communication
- Abstraction of the render mechanisms
- Connecting basic system functionality like power management, persistent data storage,etc.
- provide some kind of skeleton for an application, running within the framework
- tbc

The HMI-Framework provided by HMI-Base Component can basically be split up into two parts which will be described in the following chapter (see chapter 2.3.1 Framework Components): CGI-Framework and HMI-Application Framework.

In order to create an application, the company strategy is ASF (Automotive Service Framework). To get a deeper knowledge on this, please read ASF User Guide.

An HMI application has two components in terms of ASF: GUI and HALL. HALL(HMI Application Logic Layer) will deal with different Middleware Interfaces. GUI will take care about the rest what is basically all the graphic related stuff including views, widgets and state machine.

Pre-Release

Both parts interact together, communicating via the so called Courier Messaging Mechanism, chapter 2.3.1 Courier Messaging .

The fundamental pattern that is followed by this approach is the model view controller pattern. reduced to the main components, the setup of an application based on this framework look like Figure 2.2

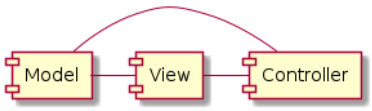


Figure 2.2.: Model-View-Controller

2.3.1. Framework Components

The HMI-framework can be split up in two major blocks. One is the so called CGI part, received from FEAT company and the second is the HMI-Application framework part developed by Bosch.

The CGI part itself can be roughly split up into the subcomponents

- Courier Message Framework, chapter 2.3.1 Courier Messaging
- Candera Render Engine, chapter 2.3.1 Candera Render Engine
- Feat Standard library, containing different utilities and helper classes

Also the HMI-Application framework part can be divided into several smaller parts

- AppBase -> base classes for a HMI application, chapter 6.2 Applications
- AppHmi_MasterBase -> base classes for a master application, chapter 2.4.4 AppHmi_Master
- AppUtils -> helper library, to be extended
- BaseContract -> chapter 2.3.2 The Base Contract
- CgiExtensions -> extensions of the CGI framework part, chapter 2.3.3 Extended CGI functionality
- HMIAppCtrl -> implementation for basic application control, chapter 4.2.1 HmiAppControl Service
- Services, chapter 4.2.1 HmiAppControl Service , chapter 4.3.2 Audio Source Control Service , chapter 4.3.3 Sound Settings Service
- TransitionAnimation, chapter 5.7 Animations

All part listed above for CGI and HMI-Application framework can be found in the folder structure, see chapter 2.2 Sourcetree (in the folders ”CGI” and ”Framework”).

Courier Messaging

For communication in between an HMI-Application it can make use of the Courier Message Framework delivered with the CGI-Toolchain by FEAT. It provides a tiny and simple to use messaging mechanism.

A message declaration is done in a xml dialect and basically looks like this

Pre-Release

```
<message name="DummyChangedUpdMsg" uid="{CAB96F02-129B-4227-ACBC-DF8C00C4C11I}">
   <subscribers>
     <subscriber id="Model" />
   </subscribers>
   <member name="DummyString" type="Candera::String" uid="{932ADB57-97CD-484A-8DE1-9
       BE59F3BACI7}"></member>
   <member name="DummyValue" type="Courier::UInt32" uid="{932ADB57-97CD-484A-8DE1-9
       BE59F3BACJ7}"></member>
 </message>
```

For more and detailed information about the xml description, please see chapter 3.1.3 HMIContract

Also posting the message is quite easy

```
DummyUpd* msg = COURIER_MESSAGE_NEW(DummyChangedUpdMsg)("DummyText", 0);
POST_MSG(msg)
```

Receiving a message can be done in every component that is in the subscriber list of the message. Only a method to handle the message has to be defined and the message type has to be known by the dispatcher. This can be done like shown in the following code snippet

```
class MyMessageReceiver : public Courier::ModelComponent
{
   private:

       // Incoming events ...
       void OnNewMessage(const UpdateDummyMsg& oMsg);


       COURIER_MSG_MAP_BEGIN()
       COURIER_OnNewMessage(UpdateDummyMsg)
       COURIER_MSG_MAP_END()
};
```

Candera Render Engine

Candera is the render engine coming with the CGI tool chain. It has the capability for 2D and 3D rendering and is applicable to different platforms and architectures as there are different so called device packages available (this includes the adaption to different operating system and hardware platforms). In case of HMI-Base the package for Genivi device and Linux OS is used.

Detailed information about the Candera render engine can be found in the official CGI documentation delivered by FEAT GmbH.

2.3.2. The Base Contract

The base contract mainly provides a set of basic messages and constants to be used all over the applications as well as all over all projects based on this framework.

This would be some things like:

- Hardkey/Encoder informations (like key XY pressed/released)

- Information about the current state of a particular surface (e.g. visible/invisible)
- Trace query to realize trace queries on a general base
- generic Popup and Statusbar related messages, chapter 5.6 Popups , chapter 5.5 Status Bar
- tbc

2.3.3. Extended CGI functionality

Extended CGI functionality is a collector for things that are available in CGI in principle but have to be extended/adopted to the needs of Bosch CM-AI.

The main implementations in here are listed below

- CGIAppControllerBase -> a base class implementation handling events that are not consumed/not defined as input by the statemachine. This is mainly used to route events from an application to the next one that is one level up in the state machine hierarchy (from AppHmi_XYZ to AppHmi_Master)
- CloseOnTouchSession -> base functionality to close popups on user input that is not related to button interaction (e.g. touching somewhere in the empty area of the screen can close a popup)
- MemoryMappedAssetRepository -> possibility to load assets memory mapped to save some RAM, use feature variant switch VARIANT_S_FTR_ENABLE_MEMORY_MAPPED_ASSET_REPOSITORY to enable the same
- VisualStateGlueLayer -> adapter class between Courier Message Framework and VisualState generated code

Additionally there are some other files containing different helper classes and macros which should not be described here in detail. chapter 5 HowTo gives some examples where such things have to be used in the application.

2.4. HMI-Application basics

In order to follow the company strategy for new components and applications, an HMI application has to be setup following the ASF User Guide. Detailed information about ASF and how to setup an application can be found in the link list at the end of this document.

Anyhow, this section should give a basic understanding about the general setup of an HMI application making use of ASF and HMI framework

2.4.1. General application setup

As already said an application has to be setup as an ASF application. This has the major advantage that ASF encapsulates the used communication technologies (like DBus, CCA and later on also Courier(ASF2.x, to be confirmed by ASF team)), so the developer only has to implement against a pure C++ interface. The second major advantage is the execution model, where all incoming messages independent from the service technology are synchronized and finally dispatched in a single thread environment. That means there will be no concurrent data access possible. Only one message will be processed at the time.

Pre-Release

An HMI application has at least to be divided into two components in terms of ASF, what means that their components are running in different threads and so there will be only a linker dependency between each other, nothing more.

In HMI these components are named 'GUI' and 'HALL'. 'GUI' contains the CGI framework parts (Courier Message Dispatcher, Candera Render Engine, Statemodel execution). 'HALL' is the connection to the middleware, where messages from/to middleware are processed. Courier messages are used to communicate from 'HALL' to 'GUI' and vice versa. As long as Courier is not officially supported from ASF, the so called CourierTunnelService is used to inject the message content into the ASF queue and so to execute them in the same thread as all other messages.

The tunnel service is a simple CMS service only, which sends a ping to HALL component, see component diagram Figure 2.3

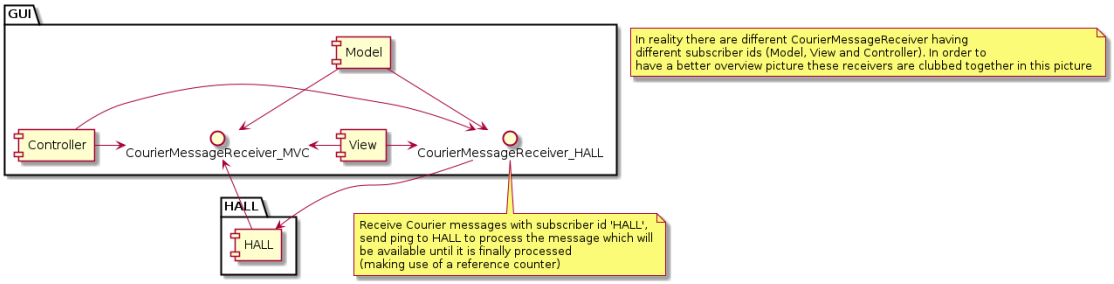


Figure 2.3.: ASF components of an HMI application

2.4.2. Threads

An HMI application is build up by different threads due to the fact that it is connected to different standard APIs like EGL and Genivi components like DLT and ILM. Also data from different service technologies (DBus, CCA, CMS, GDbus) have to be processed. To receive such data every service technology requires at least one receiver thread. These threads are setup by the ASF framework. Additionally to these technologies there is also a socket connection to Wayland, that requires an additional thread to receive information about input (touch, keys, encoder).

An example thread setup can be found in Figure 2.4

The number of threads may vary due to different application setup, e.g. when no CCA service is connected, there will be no CCA_mbx thread.

The most interesting threads for HMI are the 'Model'-Thread and the thXYMain.

'Model'-Thread: This thread works as dispatcher thread for Courier messages for the 'Model' component.

'thXYMain'-Thread: This thread mainly executes the rendering and works as dispatcher for Courier messages to the 'Controller' and 'View' component

As of now the thread configuration is only steered by using preprocessor defines. It is not planned to give control to the projects.

Pre-Rel

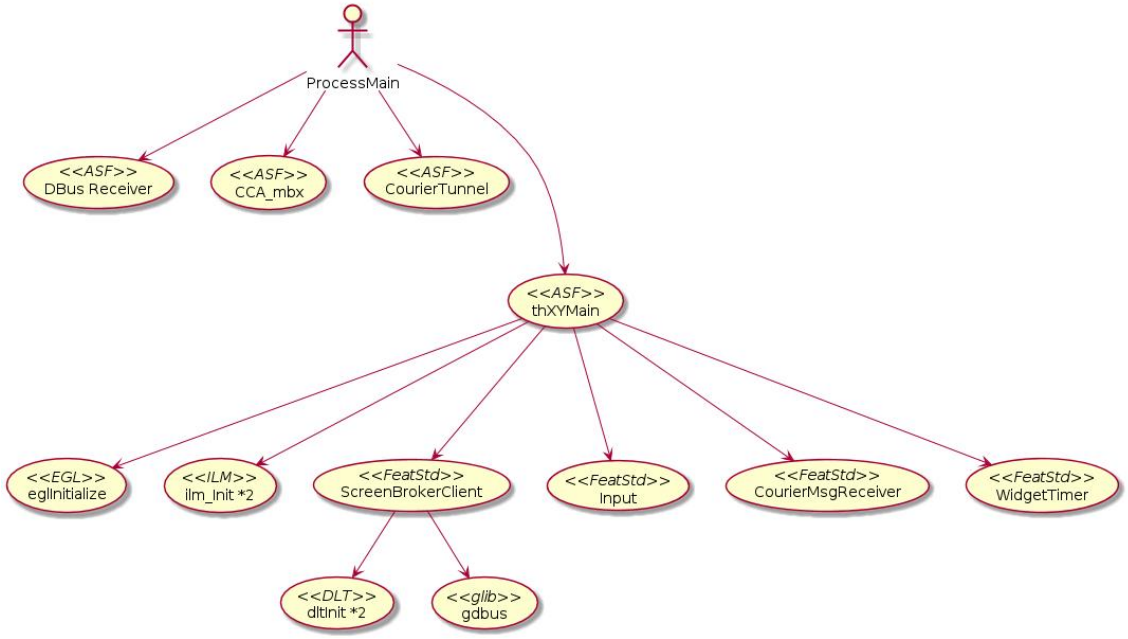


Figure 2.4.: Example for a thread setup of an HMI application

2.4.3. Common

There are several common parts that can be found in every HMI application. Some are encapsulated in base classes, some have to be implemented due to project specific needs. The following chapters will explain the parts that can be handled individually from project to project and where such parts have to be placed to meet a common structure for HMI applications.

Folder structure

In order to achieve a common structure for HMI applications and also to have the possibility for automating some things using scripts. The folder structure of an HMI application basically must look like below (example based on the reference application AppHmi_Playground)

```
|   AppHmi_Playground.xml
|
+---GUI
|   |   UserData.xml.default
|   |   AppHmi_Playground.scs
|   |
|   \---Solution
+---App
|   |   AppHmi_Playground_Trace.h
|   |   AppHmi_Playground_Trace.trc
|   |
|   +---Core
|   |       PlaygroundHall.h
|   |       PlaygroundHall.cmc
|   |       PlaygroundHall.cpp
|   |       PlaygroundGui.cmc
|   |       PlaygroundGui.cpp
|   |       AppHmi_Playground.cma
```

Pre-Release

```
|   |       PlaygroundGui.h
|   |
|   \---datapool
|           apphmi_playground_DpPersistentValues.xml
|           apphmi_playground_datapool_if.h
|
+---_scripts
|       SceneComposer_Start.cmd
|       SceneComposer_GenerateAssets.cmd
|       SceneComposer_StartSolution.cmd
|       GenContract.cmd
|       GenContract2VS.cmd
|
\---HMI
    +---HMIContract
    |   +---preliminary
    |   |
    |   \---config
    |           AppHmi_PlaygroundDatabinding.xhcdl
    |           AppHmi_PlaygroundViews.xhcdl
    |           AppHmi_PlaygroundMessages.xhcdl
    |           AppHmi_PlaygroundStateMachine.xhcdl
    |           AppHmi_PlaygroundWidgets.xhcdl
    |           AppHmi_PlaygroundConstants.xhcdl
    |
    +---DataModel
    |       HMIModelComponent.h
    |       HMIModelComponent.cpp
    |
    +---SM
    |   +---VisualStateGeneratedCode
    |   |
    |   \---config
    |           AppHmi_PlaygroundStateMachine.vnw
    |           AppHmi_PlaygroundStateMachine.stereotypes
    |           AppHmi_PlaygroundModule.vsr
    |           AppHmi_PlaygroundStateMachine.vtg
    |           AppHmi_PlaygroundPopUpModule.vsr
    |           AppHmi_PlaygroundStateMachine.vsp
    |
    \---CGIComponents
            CGIAppController.h
            CGIAppController.cpp
            CGIAppPopupTable.cpp
            CGIApp.h
            AppViewSettings.cpp
            AppViewSettings.h
            CGIApp.cpp
            CGIAppViewFactory.h
            CGIAppViewController_Playground.cpp
            CGIAppDefines.h
            CGIAppViewFactory.cpp
            CGIAppViewController_Playground.h
```

Short explanation of the content of the folders listed above:

- _scripts: Scripting related to code generation and Scene Composer, explained in detail in chapter 3.4 General Automation
  and chapter 3.4 General Automation
- App/datapool: definition of persistent datapool values to be used by this application
- App/Core: containing the definition of the ASF components and their implementation

- GUI: the Scene Composer solution of the particular application (aka the GUI model as generator base for the asset, chapter 3.1.1 The Asset)
- HMI/HMIContract: the definition of the HMI Contract as well as the output of the HMI Contract generator
- HMI/SM: the state model as well as the corresponding generated source code
- HMI/DataModel: implementation of the data model
- HMI/CGIComponents: some additional source code like the view factory, application controller and view controller for manual extensions (both empty by default), popup configuration table

Component and Sources

In addition to the sources explained in chapter 2.4.3 Folder structure , there are some common software parts that are part of every application due to the use of base classes and libraries of the framework:

- Proxy for HMIAppControl Service (see chapter 4.2.1 HmiAppControl Service)
- Basic connection to life cycle management (see chapter 4.2.3 Connection to LifeCycleManagement (LCM))
- receiver of input events from different sources, called WindowInputEventHook (see chapter 2.7 Input Handling) and ScreenBrokerClient (see chapter 2.6 Screen Broker)

or as part of the 'Apps/Common' folder placed in <project_repo>/products/<project_name>/

- Key IDs and mapping table
- Surface IDs
- Project settings to be used by the ScreenBrokerClient

The points of the second list will be described in detail in chapter 5 HowTo

2.4.4. AppHmi_Master

Whenever running more than one application providing some information on the screen, there will be a need for some kind of a 'Master' application, controlling the display access. Also some global functionality, e.g. hard key processing for application changes have to be handled there. These are the main reasons to have an AppHmi_Master process.

Please pay attention that this application has the same capabilities as a 'normal HMI app, but it has some responsibilities and skills in addition:

- As the only HMI application: Interface to GENIVI AudioManager, chapter 4.3 Audio Client
- Providing service to control all HMI applications, chapter 4.2.1 HmiAppControl Service
- Providing service with audio information/functionality to synchronize the same with the visual content and to route from/to AudioManager

Pre-Release

- Collector for events which were not consumed (and for sure routed frmo the initial receiver) by the individual statemachines of the other HMI apps
- Provider of main TTFis commands, chapter 4.5 Tracing/Debugging

Whenever AppHmi_Master is not running there is no application switch possible, but any application has to work in their domain as before. So please take care that there will be no strict dependencies to AppHmi_Master (and to other applications as well)

What AppHmi_Master is responsible for:

- Holding the state machine to decide for an application to come to foreground (related to its main surfcae, not for their popups)
- work as gateway for audio related request from any HMI app to GENIVI AudioManager
- provide a trace interface for generic trace broadcast commands
- instantiate the stub of HMIAppCtrl service
- receive forwarded events from other HMI applications that are not consumed there, e.g. hardkeys
- implementing logic that can not be delegated to a particular hmi app, e.g. as it is overlapping. Example can be a functionality that has to be realized all the time, independent from the situation on the screen and that is triggered by some user input like hardkey or so. As AppHmi_Master will finally receive all hardkeys that are not yet consumed it can react all the time on that (e.g. volume change, application switch).

What AppHmi_Master will *NOT* do:

- Having knowledge about internals of other HMI applications
- Controlling popups of other applications, see chapter 5.4.5 Screen Broker plug-ins and chapter 5.6 Popups to have an idea about popups and their management
- Dispatching user input to other HMI applications

2.5. Widgets

A graphical control element or widget is an element of interaction in a graphical user interface (GUI), such as a button, scroll bar or simply a textout. Widgets are software components of the GUI that allows a user to interact with the system (like loudness control) through direct manipulation of system values. Others act as containers that group the widgets added to them, for example panels, control bars and tabs.

The Bosch HMI Widget Repository library contain a collection of 2D graphical control elements and the logic to render these.

The CGI-Studio graphical user interface in combination with HMI-Contract automatically generates all the data structures and source code for the graphical control element from property configuration provided by the GUI developer.

Below you see a Widget render Sequence

Pre-Release

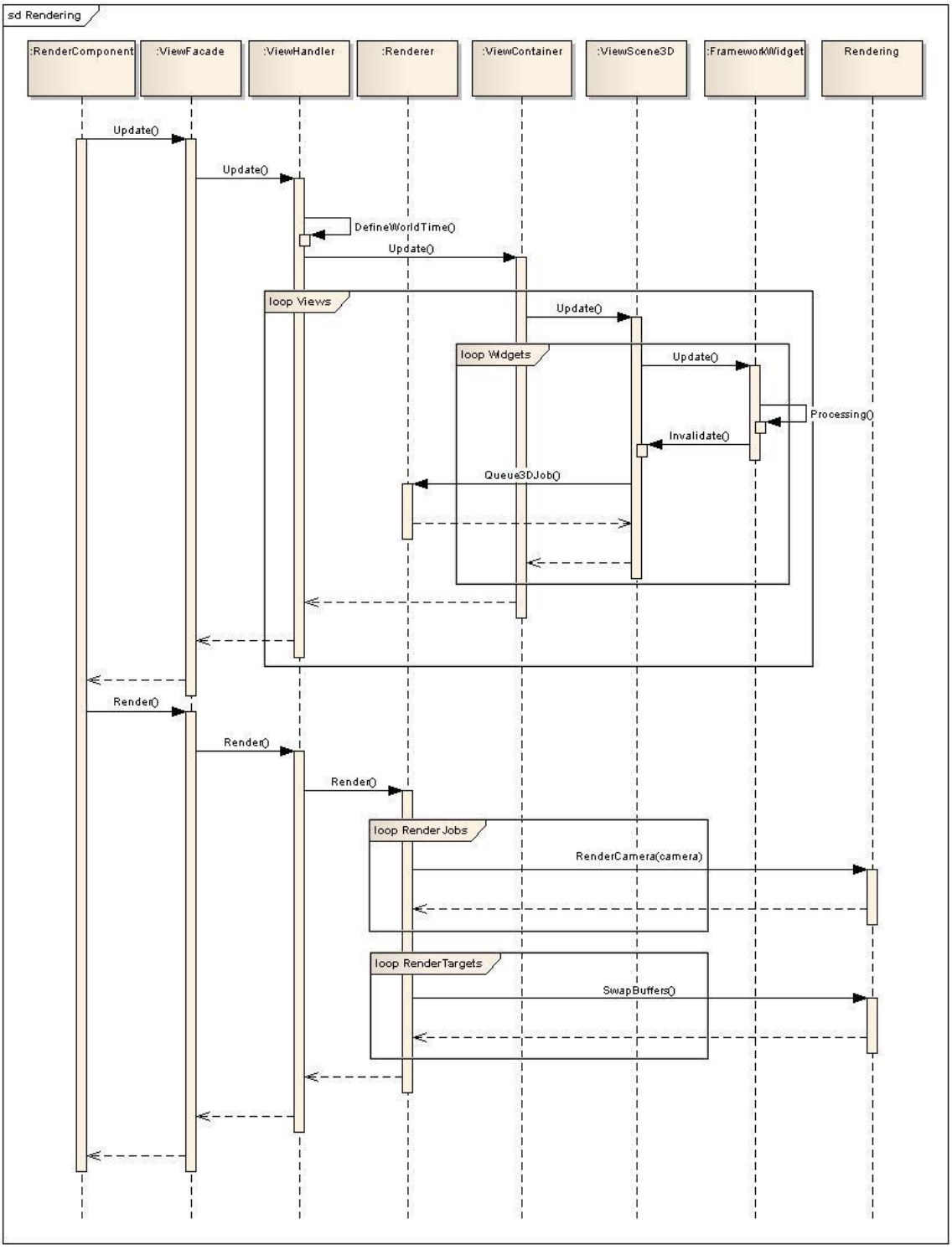


Figure 2.5.: Widget render Sequence

2.6. Screen Broker

In Gen2 ”X Window System” was used as window management system. In Gen3 instead the Genivi component ”Genivi Ivi Layer Management” (ILM) is used. The ILM supports several back-ends (as plug-ins). In Gen3 the ILM is used together with the Wayland plug-in.

The Screen Broker was developed by FEAT to let the CGI-Framework interact with ILM/Wayland. It is a component that is independent from the CGI-Framework but can be used as extension to it. The Screen Broker depends on the ILM and the Wayland plug-in.

The Screen Broker offers the following features:

- Handling of the resource display/screen
- Handling of popups

The Screen Broker is the single point of contact for all HMI-Applications to show a view or a popup on the screen. To make the corresponding content visible the Screen Broker uses ILM/Wayland-APIs. The ILM uses layer and surface ids, they can be defined freely. However, these ids must be unique across the complete system. So the developers of all applications have to agree on the ids. Please take into account that also applications like the map or the rear view camera (RVC) use surface and layer ids.

The Screen Broker is split into the ”core & client” and the ”plug-ins”. The plug-ins are project specific. HMI-Base delivers a basic set of Screen Broker plug-ins. If the customer projects need more/different features, they have to extend the Screen Broker plug-ins by project specific features.

In the next sections the core and the plug-ins are described in more detail.

2.6.1. Core

The Screen Broker core consists of:

- Screen Broker executable: screenbroker_out.out
- Screen Broker server: libscreenbrokerservice_so.so
- Screen Broker client: libscreenbrokerclient_so.so

Furthermore there exists an HMI-dependent interface, which is used by HMI-Apps and does all communication with Screen Broker client. This interface is part of this lib:

- AppBase: libapphmi_fw_appbase-asf_a.a

To enable an HMI-App to use the Screen Broker, it has to link against Screen Broker client and AppBase. All calls from HMI-App to Screen Broker go via AppBase to Screen Broker client. The Screen Broker server contains the components Screen Broker and Popup Manager. AppBase interacts with these components via the client side component Screen Broker client, which provides interfaces for requests, responses and notifications (see Figure 2.6).

The requests send to Screen Broker core contain the following attributes:

- Application defined request ID which is used to map corresponding responses to requests

Pre-Release

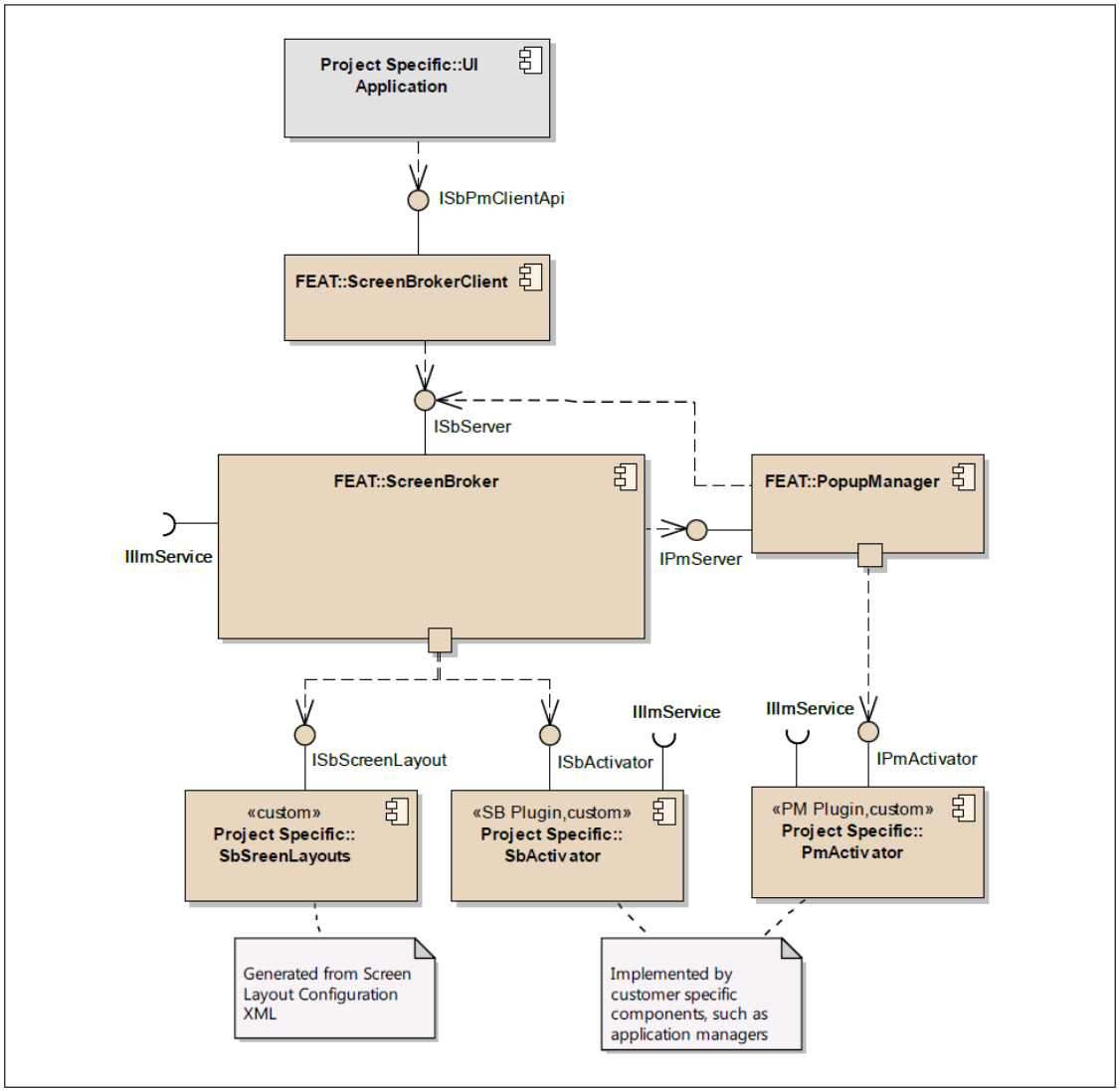


Figure 2.6.: Screen Broker and Popup Manager Component Design

- Application defined ID (AppID) which allows project specific Screen Broker plug-ins to identify the application.
- Application or project specific data (user data) which can be used for special processing instruction or other purpose in communication between client application and project specific Screen Broker plug-ins.
- Request specific attributes

Screen Broker core takes care to return the request ID of the request with a corresponding response but does not process or modify application ID or ”user data”. Therefore it is in responsibility of the plug-ins to provide according values in corresponding requests.

Screen Broker core sends information to clients in form of notifications and responses. Notifications are submitted unsolicited whenever an according event occurs on Screen Broker core side. Responses are

the immediate answers to requests and typically provide information requests from the Screen Broker core.

The Screen Broker client is communicating with Screen Broker server across boundaries which requires inter process communication (IPC). The concretely used IPC mechanism is abstracted from the applications. The used IPC mechanism in Gen3 is gdbus [Design Screen Broker and PM].

2.6.2. Plug-ins

Project specific functionality and behavior, such as arbitration and basic visualization configuration is required to be implemented in project specific plug-ins. A plug-in is a ”physically” dynamically linked library which is detected and loaded during start-up of Screen Broker core. It is possible to implement multiple interfaces within a single library. The Screen Broker core cannot work without the plug-ins, so the start-up will fail in this case.

The plug-ins are built into: libscreenbrokerplugins_so.so. This lib contains these plug-ins:

- Screen Layouter
- Screen Broker Activator
- Popup Manager Activator

Screen Layouter

This plug-in gets as input the *ScreenLayouts.xml*, which is placed in the source folder of this plug-in. It contains the configuration for layers and surfaces. For each surface it is defined which kind of input it can accept (keyboard, pointer, touch). Furthermore the display ID of the used HW display has to be added into this file. It is also possible to connect more than one display to the system at the same time. In this case all displays and the corresponding layers and surfaces have to be added to *ScreenLayouts.xml*. The display ID is defined by the ILM, here you find as example the extract from the file ilm_types.h (placed in /usr/include/ilm/):

```
/**
 * \brief Enumeration on possible displays
 * \ingroup ilmControl
 */
typedef enum e_ilmDisplays
{
        ILM_DISPLAY_HDMI = 0,
        ILM_DISPLAY_LVDS0 = 1,
        ILM_DISPLAY_LVDS1 = 2,
        ILM_DISPLAY_PARALLEL = 3,
        ILM_DISPLAY_UNKNOWN= 666
} ilmDisplayTypes;
```

At start-up the Screen Layouter creates with help of ILM the layers configured in *ScreenLayouts.xml* and attaches them to the corresponding display. The render order of the layers is given by the sequence in the XML document. The first layer is the bottom most layer, the last one is the top most layer. The XML format is described in HMI-BASE SW delivery in this file:

XML-FORMAT.README(4)

Early release

When switching between surfaces the Screen Broker uses the information regarding input acceptance of a surface from the XML to set the input acceptance for a surface and to set the pointer and keyboard focus at ILM.

There is a corresponding schema file (*ScreenLayouts.xsd*) against which the *ScreenLayouts.xml* has to be validated. This file is also placed in the source folder of this plug-in. On the Linux host PC the validation can be done with this command (*xmllint* must be installed):

```
xmllint --noout --schema ScreenLayouts.xsd ScreenLayouts.xml
```

This validation can also be done on the target by Screen Broker (at start-up of Screen Broker). Therefore the *ScreenLayouts.xsd* must be copied to target also and *xmllint* must be available on the target. The environment variable *PLUGIN_SCREENLAYOUTER_XSD* has to be set to holds the path to the corresponding schema file. However, it is not recommended to do this validation on target, since it is time consuming. Furthermore it makes only sense to do this once the xml file was changed. So a validation on the Linux host PC after changing the XML file is sufficient.

Screen Broker Activator

This plug-in is responsible for showing main views of an HMI-App on the screen. The requests to show a main view of an HMI-App are processed here, the result will be communicated back to the HMI-App via AppBase with a call to OnSurfaceStateChanged().

The method OnSurfaceStateChanged() has these parameters:

- ScreenBroker::UInt32 userData
- ScreenBroker::UInt32 surfaceId
- ScreenBroker::SurfaceState::Enum surfaceState
  - Registered
  - Unqueued
  - Queued
  - Unmapped
  - Mapped

As a result AppBase sends the Courier message SurfaceStateUpd into the HMI-App (see chapter 2.6.5 Application activation).

Popup Manager Activator

This plug-in encapsulates the popup handling. It offers showing/hiding of popups. Each popup has a defined set of properties. Depending on these properties this plug-in decides what to do with this popup (show, hide, queue, unqueue). This decision is also communicated back to the HMI-Application via AppBase with a call to OnSurfaceStateChanged(). As a result the AppBase sends the Courier message SurfaceStateUpd into the HMI-App (see chapter 2.6.6 Popup presentation). The description of the popup properties can found in HMI-BASE SW delivery in this file:

PROJECT_ScreenBrokerPLUGINS_POPUP_PROPERTIES.README(2)

Pre-Release

Diagnosis features

The Screen Broker plug-ins support a diagnosis command, which prints the status of all active surfaces (incl. popups and permanent surfaces). This command can be send to Screen Broker service daemon with: `kill -SIGUSR2 'pgrep screenbroker'`

Furthermore there is a generic interface available for testing purpose: RequestCurrentStatus(UInt32 requestId). It provides the possibility to request various current status from the plug-ins in a generic way.

RequestCurrentStatus() is used to make a status request against the Screen Broker plug-ins. The nature of the request is identified by the given requestId. A status request is always sent to all attached Screen Broker plug-ins, which then may decide to answers such a request or not. The communication return path is not defined by the given request, as this depends on the result, respectively on the given surfaceId, which is part of the CurrentStatus notification. This may cause one application triggering a status request, another application receiving its result. Furthermore, it is even not guaranteed, that any result is delivered, as either no plug-in reacts on the request, or there is no valid result (e.g. Request the top popup, while no popup is currently active). If a result is available it is notified in the client application in AppBase via the ClientApi interface OnCurrentStatus(). The protocol (resp. the kind of requests) has to be defined between client applications and the plug-ins. As payload a 32bit status and a 32bit userData are available. In HMI-Base plug-ins these requests are supported:

Request ID	*Description*
ActivePopups	All active popups (application and system)
TopActivePopup	Only the top active popup (application or system)
TopActiveVisiblePopup	Only the top active popup (application or system) that is visible (in case a popup filter is active, otherwise the result is the same as for TopActivePopup).
TopActiveSurfaceNonPermanent	Top main view surface, that is not permanent
TopActiveSurface	Top active surface (incl. permanent surfaces)

These request IDs can be found in every HMI-Base delivery in this file:

ProjectSettings.h(1)

Reset of Screen Broker plug-ins

It is possible to reset the internal state of Screen Broker plug-ins. This can be useful e.g. on shut down of the system. The reset for activator plug-ins can be triggered via the external signal SIGUSR1, which is sent to the Screen Broker service daemon e.g. like this: `kill -SIGUSR1 'pgrep screenbroker'` The reset clears all internal states (incl. Popup Manager's timer list) and request to hide all active surfaces (incl. popups and permanent surfaces) toward layermanager.

Pre-Release

2.6.3. Threads

In Screen Broker these threads are created:

- Popup Manager (Screen Broker core): Popup timer thread
- Screen Broker plug-ins: Animation timer thread

2.6.4. Start-up

The Screen Broker server has to be started after ILM service because it relies on functionality of the same during its own start-up phase. To prevent client-side communication in early system states, clients/application processes shall be started after start-up of Screen Broker is completed.

Screen Broker core loads associated plug-ins and instantiates internal classes including classes of Popup Manager component. During initialization phase screen areas are set up from Screen Layouter Plug-in which creates and configures corresponding ILM layers. Screen Broker sets up and applies the render order of the layers defined as z-order of areas by *ScreenLayouts.xml*.

Afterwards Popup Manager related classes are instantiated and server-side communication channel is established.

HMI-Apps as clients to Screen Broker are started by the system after start-up of ILM and Screen Broker is completed. The CGI based HMI-App requests loading its visual content and UI via Candera Graphic Engine in the same way as for server-less configurations. This request is sketched by invocation of method loadAssetLibrary() on "Candera 2D/3D" component.

Candera loads appropriate assets from the asset library and instantiates its render targets based on this information. The render targets are implemented in a candera device package suitable to work with ILM. Candera registers as Wayland client for this display if not yet done and creates corresponding graphic buffers as EGL surfaces. The display ID, layer ID and surface ID are read from the asset.

ILM device package registers the created EGL surfaces as ILM surface at ILM service using ILM client library. Afterwards it registers the ILM surface at Screen Broker via Screen Broker client with additional attributes. These additional attributes include the layer ID, surface ID and surface specific hinting information to provide special, project specific tags or other kind of information for processing by the project specific Screen BrokerActivator plug-in implementation [Design Screen Broker and PM].

Surface on demand registration

Normally at start-up of the HMI-Applications, each application registers its surface(s) at Screen Broker. In some use cases it might be useful to avoid too much early communication between HMI-Apps and Screen Broker. In this case the surface registration on demand can be activated at Screen Broker with setting this define during build of Screen Broker plug-ins: SCREENLAYOUTER_READ_SURFACE_LAYER_MAPPING to 'ON' or '1' in buildScreenBrokerPlugins.sh(5) . In the *ScreenLayouts.xml* the static surface to layer mapping has to be done by adding *Surface* tags to the *ScreenArea* tags accordingly. If no surface registration was done by the HMI-App, this mapping in the XML file is used to perform the registration. This fall-back solution is applied if the client API interfaces

- ActivateApplication()

Pre-Release

- ShowPopup()
- HidePopup()

are called and the surface referenced in these interfaces is not yet registered. The owner of the surface registration is then the client which triggered one of the three mentioned client interfaces (first). To have this feature proper working, the applications have to take care, that the surfaces are created prior to these calls.

External layers

The Screen Broker normally creates and destroys all layers mentioned in *ScreenLayouts.xml*. Additionally the layer render order per display is set. But if there are external layers (e.g. for Map or LSIM-facia), it might be useful to let Screen Broker only set the render order. In this case the creation and destruction are done by an external application (creation needs to be done before Screen Broker is started) and not by the Screen Broker. Marking which layers are "external" is possible in *ScreenLayouts.xml*. The tag *ScreenArea* has the property *managed*. If this property is set to "false", the Screen Broker uses this layer only for setting the render order. For further details please have a look here:

XML-FORMAT.README(4)

2.6.5. Application activation

HMI-Apps as clients to Screen Broker need to be "activated" to be presented on screen. If an application wants to be activated, it submits an activation request with help of AppBase via Screen Broker client to Screen Broker. The HMI-App can call at AppBase these methods:

- Show(const Courier::ViewId & viewId, ScreenBroker::UInt32 surfaceId)
- Hide(const Courier::ViewId & viewId, ScreenBroker::UInt32 surfaceId)
  - This method is not used for "normal" main view surfaces, only for the special case of *permanent* main view surfaces it is used (see chapter 2.6.5 Permanent surfaces)

Please note that an HMI-App may only call these methods for its own surfaces. AppBase internally checks if this surface is a "main view" or a "popup". In case it is a "main view", it submits the corresponding request ActivateApplication() to Screen Broker client for activating or deactivating a main view with the following properties:

- Surface ID
- Request ID (Bit0 indicates if this is a show (1) or hide (0) request.)
- App ID
- Project specific data (in HMI-Base plug-ins: user data = hash of view ID)

For more information regarding the project specific protocol for the request ActivateApplication(), please find more details in HMI-BASE SW delivery in this file:

PROJECT_ScreenBrokerPLUGINS_PROTOCOL.README(3)

Screen Broker server receives the request and delegates it to Screen Broker Activator plug-in. Besides the data received from the client, it looks-up the screen area (layer) and the corresponding layer ID

Pre-Release

to complete the request for the plug-in. The Screen Broker Activator plug-in needs to arbitrate the request by considering surface hinting information and data provided by the request. If the request is accepted, it maps the surface with the named ID on the indicated layer. It is also free to position or modify surface attributes according to project needs. [Design Screen Broker and PM].

HMI-Apps are deactivated automatically if a different HMI-App is activated. Screen Broker notifies clients for each of their surfaces being presented or concealed. Clients shall react on such notifications by enabling or disabling rendering onto the respective surface. The notification sent to AppBase in this case is OnSurfaceStateChanged(). AppBase sends then the Courier message SurfaceStateUpd to the HMI-App. This message contains one of these surface states:

- Invisible
- Visible
- Registered
- Unqueued (in case of popup)
- Queued (in case of popup)

Permanent surfaces

In terms of Screen Broker there are two different types of surfaces:

- Main view surfaces
- Popup surfaces

Regarding the main view surfaces, only one surface at a time can be visible. The same applies for the popups, all other requested popups are queued or closed.

It is also possible to have *permanent* main view surfaces. If a surface is *permanent* it will not be deactivated on application switch. It can be activated from the HMI-App the surface belongs to like a "normal" main view surface (with calling Show() at AppBase). It can be deactivated with calling Hide() at AppBase.

The configuration which main view surface is *permanent* is done in ProjectSettings.cpp. This file can be found in this path: ProjectSettings.cpp(6) .

An example for a *permanent* surface is a status bar. The status bar should be visible in most of the views and should not be deactivated on an application switch. If in some views, it should not be visible, it can be deactivated in the corresponding views.

2.6.6. Popup presentation

Independent of the current activation state of the HMI-App, it needs to be able to present popups. Presentation of popups is requested via a dedicated popup presentation request at Screen Broker client

Pre-Release

providing additional attributes:

Popup property	*Description*
Popup ID	ID
Priority	0 (low) to 255 (high)
Presentation time	for automatic deactivation
Minimum presentation time	to avoid flickering
Validity period	after which the popup shall be discarded in any case
Close on superseded	If the popup should be closed or queued when it is superseded
Close on app leave	If an app/local popup should be closed once the corresponding app is left
Alignment	alignment information
Scaling	scaling information

More information regarding the properties can be found in every HMI-Base SW delivery in this file:

PROJECT_ScreenBrokerPLUGINS_POPUP_PROPERTIES.README(2)

Screen Broker queries its internal registry objects to find the screen area (layer) the popup is registered for. It stores the request data within a dedicated "PopupState" object. It forwards the request and corresponding data to the Popup Manager Activator plug-in. The plug-in needs to arbitrate the request and (in case of presentation is accepted) issue all required steps for visual presentation at ILM. The plug-in can access the popup queue of the corresponding screen area (layer) to access information and states of the already queued popups. After the plug-in finishes arbitration and presentation procedure, it submits the result of the request to the requesting AppBase via Popup Manager and Screen Broker (via OnSurfaceStateChanged()). AppBase sends then the Courier message SurfaceStateUpd to the HMI-App. This message contains on of these surface states:

- Invisible
- Visible
- Registered
- Unqueued
- Queued

The HMI-Apps need to be able to conceal any presented popup if the cause for presentation is not valid or existing anymore. Screen Broker provides an interface for applications to request popup concealing. This request processing is similar to the one for processing the presentation request. The differences are basically that the object representing the popup state already exits and need to be cleared and removed from the list of requested popups.

Pre-Release

Popups define time-based behaviour, such as automatic concealing or expiration. Because the calculation of this behaviour is project specific, it is performed by the PM activator plug-in each time a popup is requested to be presented. The plug-in stores the timestamp of the next event inside the popup state object. PM calculates every time a popup is added or removed the timestamp for the next evens and resets a timer accordingly. If the system timer expires Screen Broker server is notified about the timeout. This is sketched as "OnTimer" invocation. It notifies PM to request updates of the state of all popups. PM walks through all screen areas (layers) which are configured to present popups and iterates over all popup states having a "next update time" equal to or less than the timestamp of the timer event. For each of those popup states it requests an update of the state at the plug-in. The plug-in calculates the new state of the popup, presents or conceals popups as needed and submits notifications for every popup surface that entered a different state to the HMI-Apps via Screen Broker and Screen Broker client [Design Screen Broker and PM].

The HMI-App can call at AppBase these methods to show or hide a popup:

- Show(**const** Courier::ViewId & viewId, ScreenBroker::UInt32 surfaceId)
- Hide(**const** Courier::ViewId & viewId, ScreenBroker::UInt32 surfaceId)

Please note that an HMI-App may call these methods only for its own surfaces. AppBase internally checks if this surface is a "main view" or a "popup". In case it is a "popup", it submits the corresponding request ShowPopup() or HidePopup() to Screen Broker client to show or hide a popup with the following properties:

- ShowPopup()
  - Surface ID
  - Request ID
    - (Bit0 indicates "close on superseded" is true(1) or false(0). see chapter 2.6.7 Close on superseded)
    - (Bit1 indicates "close on app leave" is ture(1) or false(0). see chapter 2.6.7 Close on app leave)
  - App ID
  - Project specific data (in HMI-Base plug-ins: user data = hash of view ID)
  - Popup presentation arguments
    - Priority
    - Modality
    - Presentation time
    - Minimum presentation time
    - Validity period
    - Horizontal alignment
    - Vertical alignment
    - Scaling

Pre-Release

- HidePopup()
  - Surface ID
  - Request ID
  - App ID
  - Project specific data (in HMI-Base plug-ins: user data = hash of view ID)

For more information regarding the project specific protocol for the requests ShowPopup() and HidePopup () please find more details in HMI-BASE SW delivery in this file:

PROJECT_ScreenBrokerPLUGINS_PROTOCOL.README(3)

To use the popup handling of the HMI-Base Screen Broker plug-ins in an HMI-App some requirements regarding modelling of state machine and GUI for the HMI-App must be fullfilled. Furthermore a popup table has to be created for every HMI-App that contains the popup properties for all popups of this application. The details are described here: How To Popups wiki and chapter 5.6 Popups .

2.6.7. Popup features

The popup handling of the HMI-Base Screen Broker plug-ins offers these features:

- Modality handling
- Priority handling
- Close on superseded
- Close on app leave
- Close on touch
- Timing handling
- Popup filter

Modality handling

A popup has one of these modalities:

- Application
- System

"Application" means that this is an application-local popup. It can only be visible when the corresponding application is also shown (showing its main view). "System" means that this popup is global. It can be shown on top of every application. The modality is checked in different situations:

- On show popup request (see Figure 2.7)
- On application switch (see Figure 2.8)

Pre-Release

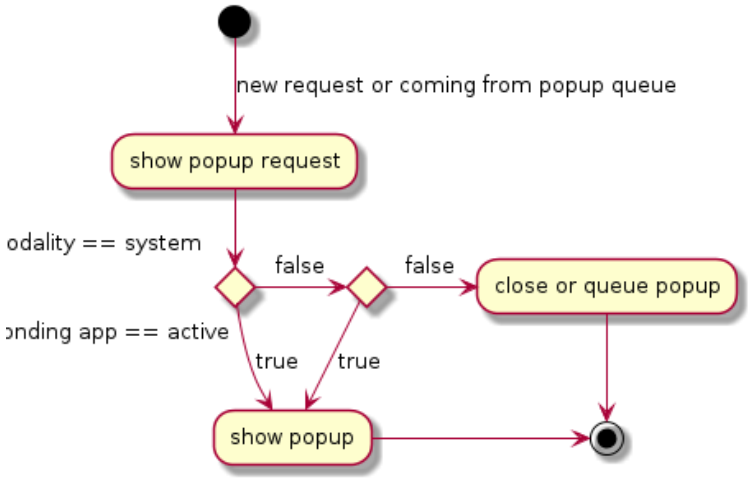


Figure 2.7.: Activitiy diagram modality handing - show popup

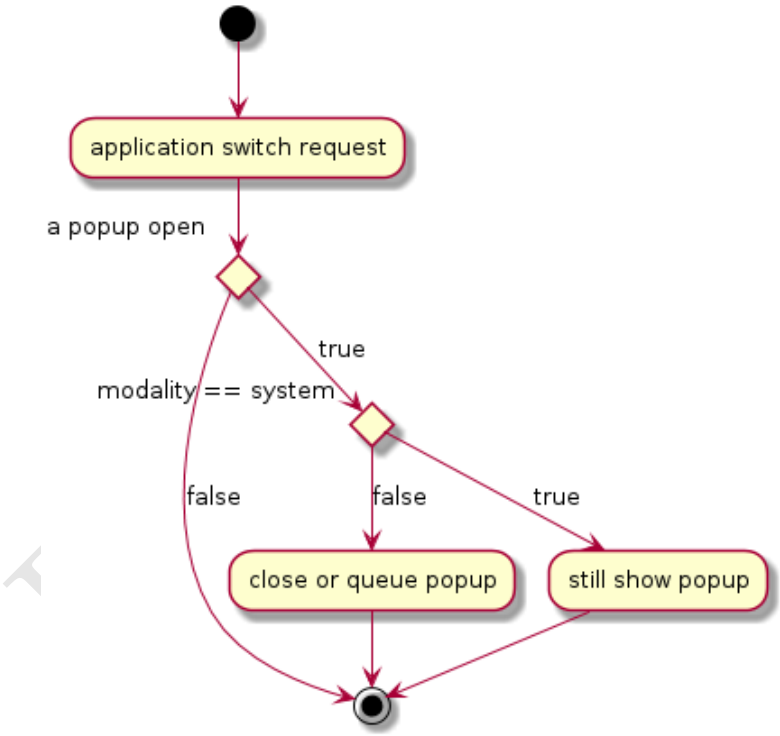


Figure 2.8.: Activitiy diagram modality handing - switch application

Priority handling

Every popup has a priority between 0 (low) and 255 (high). The priority decides on the order in which popups are shown, if several popup are open at the same time. One popup at a time can be visible. All other popups are closed or queued (depending on close on superseded). The popup with the highest priority gets shown.

Close on superseded

Popup A with priority 10 is shown, now it is requested to show popup B with priority 100. As a result Popup B gets visible, for popup A this decision has to be taken:

- *Close on superseded* = true: Popup A is closed
- *Close on superseded* = false: Popup A is queued

Close on app leave

Application A is active and on top of it an application popup of app A is shown. Now the app is switched to app B. For the application popup this decision has to be taken:

- *Close on app leave* = true: The popup is closed
- *Close on app leave* = false: The popup is queued

Close on touch

If a popup is open, the user can touch outside of touchable widgets (e.g. buttons) to close this popup. To use this feature for a popup, some changes have to be done in state machine and view controller.

State machine:

- Call action *gRegisterForCloseOnTouchSession* when entering a state that should be closed when touched (and no touchable widget) is hit.
- Define a message (based on *CloseOnTouchMsg*) in the *Statemachine.xhcdl of your application, on which your process closes your scene (example see below).
- Call action *gDeregisterForCloseOnTouchSession* when leaving the state.

```
<message messageRef="CloseOnTouchReqMsg" uid="{958d808e-7260-4c8c-8d30-3467a2b4174d-sat2hi}"
    >
  <scope moduleRef="AppHmi_PlaygroundPopUpModule"/>
  <map nameRef="ViewId" valueRef="AppHmi_PlaygroundPopUpModule/PUScenes/PopupScene_Glob_1"
      />
</message>
```

View controller:

- Derive from *CloseOnTouchViewHandler* instead of *Courier::ViewHandler*.
- Use macro *COURIER_MSG_MAP_END_DELEGATE(CloseOnTouchViewController)* instead of *COURIER_MSG_MAP_END* (see example below).

```
#include "CgiExtensions/CloseOnTouchViewController.h"
class CGIAppViewController_PopupScene_Glob_1 : public CloseOnTouchViewController
{
    courier_messages:
        COURIER_MSG_MAP_BEGIN()
        COURIER_MSG_MAP_END_DELEGATE(CloseOnTouchViewController)
```

Release

For more details please have a look in the HMI-Base reference application AppHmi_Playground and see here: Close on touch wiki.

Timing handling

A popup can be closed due to a trigger from the user (e.g. user touches a soft-button), due to a trigger from the system (e.g. phone stops ringing, HMI-App sends an HidePopup() request to Screen Broker to remove phone popup) or due to timing.

For every popup a timeout can be defined after which the popup has to be closed (*presentation time*). If the presentation time is '0' the timeout handling is deactivated.

Once the HMI-App sends with help of AppBase the ShowPopup() request to the Screen Broker, a timer with this timeout is started. If the popup cannot be displayed immediately (e.g. since a popup with a higher priority is shown), the popup gets queued and can only be shown once the higher priority popup is closed. In this case the popup is then only shown for the remaining presentation time. If the timeout is elapsed the popup is closed or removed from the popup queue automatically.

In case the remaining presentation time is very less, it could come to a flickering on the screen. Therefore the *minimum presentation time* can be defined per popup. It defines the time the popup must be minimum shown on the screen. This means for the case described above (the popup cannot be shown immediately and is queued for some time): If the remaining presentation time is < minimum presentation time, the popup is not shown but closed.

There is a third parameter in timing handling: *validity period*. It defines the period during which it is valid to show the popup. But the validity period will not close a popup, it is a passive timing. It will be checked once the state of a popup surface changes from queued to mapped. The validity period is only used if the presentation time is != '0'. If the validity period is set to '0', it is ignored. If the popup cannot be shown immediately, but later it can be shown, the validity period is checked. Only if the popup is still valid, it will be shown. Otherwise it will be closed.

Here is an example for the different timing values:

- Popup A should be visible maximum 15s, minimum 3s. But if this popup is not visible after latest 10s, it should not be made visible.
- Corresponding timing values:
  - Presentation time = 15s
  - Minimum presentation time = 3s
  - Validity period = 10s

Popup filter

There are use cases in which it is required to not show (some of the) active popups. E.g. if the rear view camera (RVC) is shown, no popups have to be shown on top. Otherwise the user cannot see the RVC picture (which is a safety risk). To handle this use case the RVC HMI-App can set a popup filter at Screen Broker. These filters are available:

- Disable all popups

Pre-Release

- Disable all popups with a priority < n

An HMI-App can set and clear a popup filter. To do so an HMI-App can use these methods of AppBase to send the request via Screen Broker client to Screen Broker:

Method	*Description*
SetPopupFilterAll(void)	Used to disable all popups.
SetPopupFilterPrio(ScreenBroker::UInt32 prio)	Used to disable all popups with a priority < prio.
ClearPopupFilter(void)	Used to disable the popup filter.

Regarding popup filter these rules apply:

- An HMI-App is only allowed to set/clear a popup filter, if it is active. Active means its non-permanent main view surface is visible on the screen (e.g. tuner main view for tuner application).
- If an HMI-App that is currently not active sets/clears a filter, this filter request will be stored and used once this HMI-App gets active.
- It is allowed to set one filter at a time. No combination of filters are possible.
- If a filter is set for the active HMI-App and it is switched to another HMI-App, the filter is cleared by the Screen Broker plug-ins.
- The filter has only an impact on the popup visibility, the order of the popup stack is not touched.

2.7. Input Handling

In Gen3 the handling of these input device types is supported:

- Touch
- Encoder
- Key

All input events are written into linux device files. They can be found on the target under: /dev/input/eventN, in which 'N' is the number of the device. The Wayland plug-in (as part of ILM) reads at start-up all available device files and creates these devices internally.

Once input events are received through these files, they will reach Wayland. Depending on the active surface on which this input occured, the input is sent to the corresponding HMI-Application to which this surface belongs to (via callbacks). Details regarding the platform concept of Gen3 input handling can be found here: G3g Input Handling.

In the CGI framework there is a component that does the basic input handling from HMI point of view (amongst others): CGI Courier AddOns. It does the registration at Wayland for input handling and offers a base class (WaylandEventHook) for receiving the Wayland callbacks for input.

Pre-Release

The AppBase library (libapphmi_fw_appbase-asf_a.a) contains a class that is derived from WaylandEventHook which is called WindowInputEventHook. Each HMI-Application links against AppBase and uses this class for receiving input events from Wayland.

2.7.1. Touch

As touch devices single and multi touch devices can be used. Since Wayland supports only real multi touch devices, the single touch devices are recognized as pointer devices (which is normally e.g. a mouse) at Wayland. That is why the single touch input events are received in the HMI as pointer callbacks. The multi touch input events are received as touch callbacks. In the two tables below you find the different entry points for touch input in the HMI and how they are handled.

Once a touch is received the Courier message TouchMsg with these parameters is sent:

- TouchMsgState (Down, Move or Up)
- X coordinate
- Y coordinate
- pointer id ('0' in case of single touch, number of touch contact on the surface in case of multi touch).
- surface id of touched surface

Single touch

The single touch input events are received in the HMI at these callbacks.

Method	*Description*
OnPointerEventEnter()	This method is called once the touch reaches the surface, it is used to store the surface id.
OnPointerEventButton()	This method is called once a touch down or up is received on a surface. The Courier message *TouchMsg* is sent with the parameter *TouchMsgState* = Up or Down.
OnPointerEventMotion()	This method is called once a touch contact is moved on a surface. The Courier message *TouchMsg* is sent with the parameter *TouchMsgState* = Move.

Pre-Release

Multi touch

The multi touch input events are received in the HMI at these callbacks.

Method	*Description*
OnTouchEventDown()	This method is called once a touch down is received on a surface. The Courier message *TouchMsg* is sent with the parameters *TouchMsgState* = Down.
OnTouchEventUp()	This method is called once a touch up is received on a surface. The Courier message *TouchMsg* is sent with the parameters *TouchMsgState* = Up.
OnTouchEventMotion()	This method is called once a touch contact is moved on a surface. The Courier message *TouchMsg* is sent with the parameters *TouchMsgState* = Move.

2.7.2. Encoder

Since Wayland is mainly used on personal computers till now, encoders like we have them in IVI systems, were not supported when Gen3 was started. That is why in Gen3 the pointer axis event is used for the encoders. Normally it is used for the mouse wheel. But it fits perfect to encoders since it transfers the axis (which encoder in our case) and the number of steps the wheel (in our case the encoder) is turned.

The mapping of the different axis (coming from the encoder hardware) to the HMI encoder codes has to be done by every project in the project specific file KeyMapping.cpp(7) . The mapping is used by the method OnPointerEventAxis(). The encoder input events are received in the HMI at this callback.

Method	*Description*
OnPointerEventAxis()	This method is called when an encoder is turned. Which encoder is turned is determined via the parameter *axis*. The Courier message *EncoderMsg* is sent with the parameters *EncCode* (which is the axis) and *EncSteps* (which are the numbers of steps the encoder is turned). The algebraic sign of the steps indicates the turning direction (+/-).

2.7.3. Key

Since keys (coming from the input hardware) are project specific, they have to be defined in the project specific file KeyMapping.cpp(7) . The mapping to HMI key ids has to be done in the same file. This file is used by the method OnKeyboardEventkey(). The key input events are received in the HMI at this

Pre-Release

callback.

Method	*Description*
OnKeyboardEventKey()	This method is called when a key is pressed. With a key mapping mechanism the corresponding HMI key code is retrieved. The Courier message *HardKeyMsg* is sent with the parameters *HKCode* and *HKState* (e.g. Down or Up).

Pre-Release

3. Toolchain

The CGI Studio tool chain from FEAT and the VisualState from IAR is currently chosen by HMI-Base as the HMI-Tool-Chain for the HMI development in the CM-AI domain and uses the MVC approach.

Data Component (Model)

- Filtering and preprocessing of dynamic data.
- Acts as the interface to the contents/data for the View (e.g. speed, temperature).
- Project Specific and should be decided based on the system requirements (e.g. xml based, database etc.).

UI Component (View)

- The display and the UI part of a system.
- Responsible for graphics, animations, texts etc.
- Software development platform for 2D and 3D graphical interfaces.
- CGI Studio by FEAT containing CGI Scene Composer, CGI Player, CGI Analyzer.

Logic Component (Controller)

- State Machine based logic which enables user to switch between the various scenes in the View.
- IAR visualSTATE is the tool to design state charts.

CGI Studio: Fujitsu CGI Studio is Fujitsu's software development platform for development of hybrid 2D and 3D graphical interfaces (HMI/GUI) for Automotive Systems. Besides the 'Candera 2D' engine and the 'Candera 3D' engine, Fujitsu provides a continuous tool chain (Scene Composer, Analyzer, Translator, Interaction Framework) for the development of hybrid 2D and 3D graphical interfaces.

IAR visualSTATE: IAR visualSTATE is a set of highly sophisticated and easy-to-use development tools for designing, testing and implementing embedded applications based on state machines. IAR visualSTATE creates a graphical overview of the application, with direct graphical feedback on various levels of detail. It allows you to outline your application and then step by step add functionality at a more detailed level.

Pre-Release



3.1. Tools

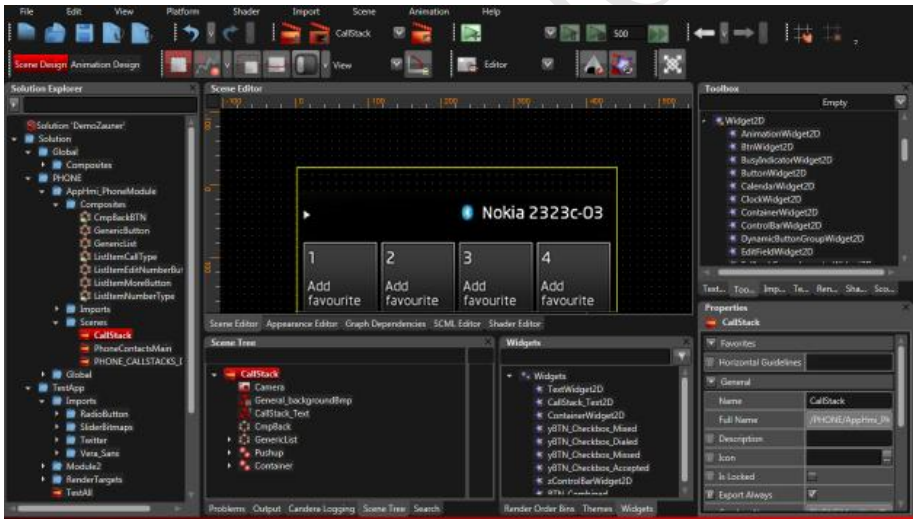
3.1.1. CGI-Studio

Introduction

The CGI Studio HMI tool chain enables system developers to seamlessly implement a graphics application from concept to actual embedded implementation.

You have on the one hand the Candera Graphic Engine that provides you a mature and intuitive 2D/3D graphic application programmable interface (API) and on the other hand Courier Interaction Framework which allows to apply modern messaging concept and data binding mechanism to your application development.

SceneComposer



As its name implies, the Scene Composer module composes 2D and 3D scenes, and allows combining 2D and 3D content, enabling the seamless process flow. Within a scene, 2D and 3D content can be separated into distinct layers to address their respective needs. The application can have any combination of graphics: 2D only, 3D only, or both at the same time.

it covers ...

- Import design from artist
- Development on host
- Verification host

Release

- Convenient deployment to target

Input to this tool by import is the graphic resource, such as models and geometry in FBX format, bitmaps, textures, 2D TrueType fonts, and 3D fonts. These assets could be generated using industry-standard 3D and 2D authoring tools like Photoshop, Cinema3D, Paint. Another key input to the Scene Composer is the widgets, that could control button states, slider positions, or simple text output on the screen. Other inputs to the tools are animations and the shader programs for the GPU. As mentioned Scene Composer allows import 2D/3D graphics resource and widgets to be dragged and dropped. This enables easy 2D/3D scene composition and object configuration. Controls for light and camera are available. The tool is capable of creating animations and also configuration and assignment relative to 2D/3D widget properties. It has a mechanism to export its output for running on either the host or the target.

Active scene is always rendered using a WYSIWYG approach. As a result, it is possible to visually inspect the imported graphics assets early in the development. Moreover, object properties can be dynamically changed in the active scene to get immediate results and timely evaluation. Finally Scene Composer exports the content for the target hardware as described in chapter 3.1.1 The Asset .

Candera Engine

This module runs the output of the Scene Composer and is part of the framework as part of Scenec-composer itself. The Candera Engine can run either on the host or on the actual target. It can support development of both 2D and 3D graphics. The Candera 2D engine supports dynamic scene graph and 2D animations, including smooth rotation, scaling, and translation of bitmaps. In order to enable seam-less integration, the 2D engine supports interaction with the 3D engine in several ways. For example, it supports render to texture; it can post-process 3D images and combine scenes with 3D. It also sup-ports hardware layers, multiple displays, alpha blending, 2D widgets, and text rendering. The Candera 3D engine is based on OpenGL-ES 2.0 and is fully compliant with the standard specification. It has been designed to be OS, application, and hardware agnostic. Its key functions are screen and scene management, 3D object handling and processing, animation framework, and asset management in the embedded systems memory. It also optimizes render order and state management.

Courier Interaction Framework

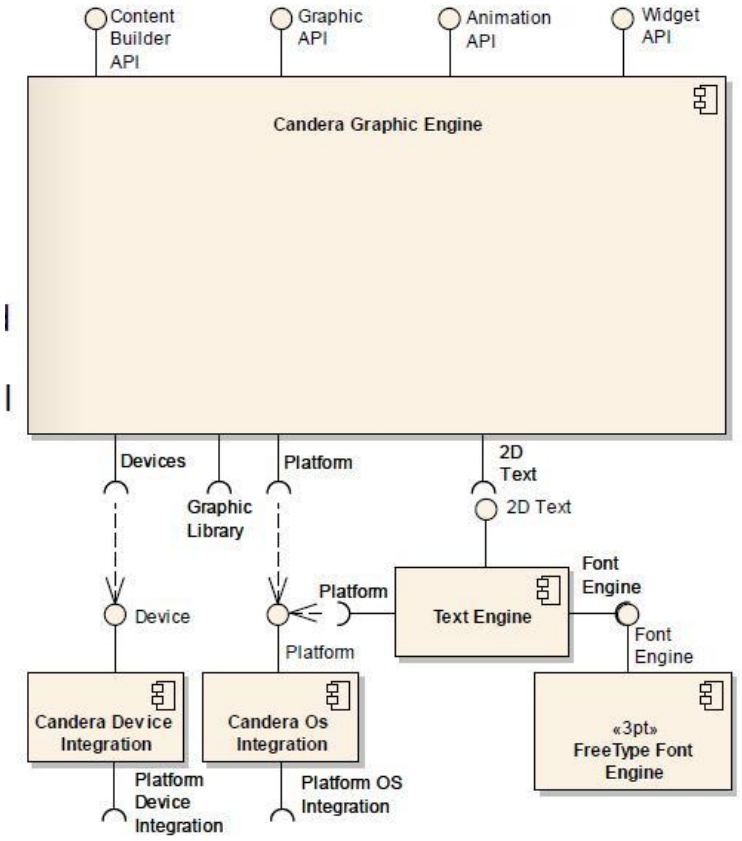
The Courier Interaction Framework is used for message handling and data binding between CGI Studio (View), any state machine (chapter 3.1.2 VisualSTATE from IAR) and the application logic HALL. Means, Courier connects the different MVC components via configurable messaging environment.

Globalization

CGI Globalization Support is a tooling environment for language and culture management in embedded systems.

Internationlisation need more text

Pre-Release



The Asset

Whenever dealing with HMI there is a need to have a certain layout of scenes, including resources like images and static (globalized) strings and their positioning/scaling/blending etc. Also there is a need to have some controll elements on a scene the so called Widgets (see chapter 2.5 Widgets for details). With the help of the SceneComposer tool a binary container containing all graphic related stuff will be generated. This is called 'Asset', which will be loaded from the HMI process during runtime from the working directory of the process.

What can be found in the binary container

- Shader
- Images
- Scene layouting
- Widgets to be instanciated and their property values

see also chapter 3.1.5 Asset Shaper



3.1.2. VisualSTATE

Along with CGI Studio from FEAT for the graphics part VisualSTATE from IAR is used for modeling the state machine. A special State Machine Connector allows CGI Studio and HMI Contract (see chapter 3.1.3 HMIContract) via a plugin a direct connection to the modeling tool VisualSTATE.

VisualSTATE is a toolset for design, test, formal verification (model checking), code generation, and high-level hardware debugging of state/event systems in general and state machines in particular. VisualSTATE is based on the state machine subset of UML (Unified Modeling Language).

IAR VisualSTATE creates a graphical overview of the application, with direct graphical feedback on various levels of detail. It allows you to outline your application and then step by step add functionality at a more detailed level. The system structure is represented with a high-level, hierarchical perspective that enables you to maintain the overview of complex applications.

VisualSTATE consists of following fully integrated tools:

Navigator

The Navigator is used for the overall handling of visualSTATE projects, from model design over test and simulation to code generation and documentation of projects.

With the Navigator you access and activate the other modules of the visualSTATE software, and set options for the Verificator, Coder and Documenter.

After creating a project in the Navigator you are ready to go to the Designer to draw your state machine models.

Designer

The Designer is a tool for designing the state machines. The tree browser allows to overview and navigate through the embedded project. All elements of the system are visible and accessible through the browser, and a double-click on the desired element brings up the appropriate configuration dialog box on the screen.

Verificator

A test tool for dynamic formal verification of models created with the Designer.

How do you test that your state machine design model and embedded application does not contain any of the following problematic properties?

- State, Local and System wide dead lock conditions Conflicting transitions between states

Pre Release

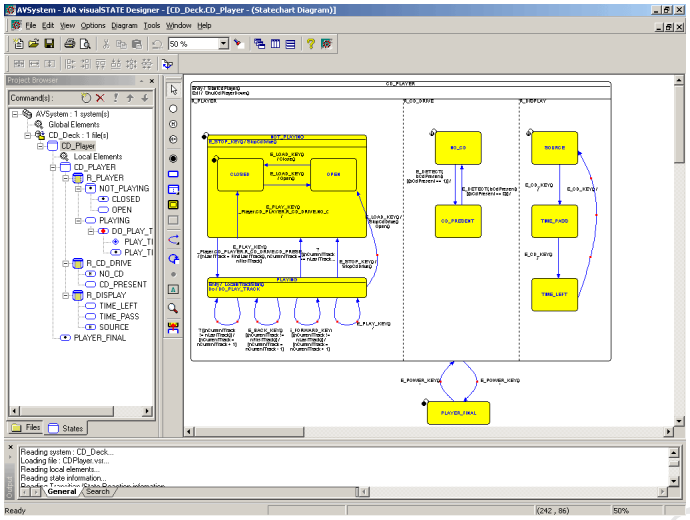


Figure 3.1.: IAR Designer GUI

- Unreachable states, i.e. states that cannot be entered by any sequence of events from the environment
- Unused events or signals, i.e. stimuli to the system that is not acted upon
- Unused transitions, i.e. transitions that will never fire, regardless of the event sequence fed into the system
- Unused actions or assignments
- Unused variables, parameters and constants

Validator

An application for simulating, analyzing, and debugging models created with the Designer. With the Validator you can test the functionality of your design. Using te RealLink facility, you can test visualSTATE models in a target application, and using the C-SPYLink plug-in, you can perform high-level state machine debugging of visualSTATE applications in the IAR Embedded Workbench C-SPY Debugger.

Coder, the code generator

The Coder can automatically generate code on the basis of models created with the Designer. The automatically generated code must be combined with a visualSTATE application programming interface (API) and manually written code.

Documenter

With the Documenter you can easily create an up-to-date documentation report on your visualSTATE project. The report includes information on design, validation, simulation, verification, generated code

Pre-Release

and implementation. All relevant project information is collected from the other visualSTATE modules and organized into a structured document in Rich Text Format or HTML.

VisualSTATE offical documentation

The official documentation can be found in the installation folder of VisualState.

VisualState Usecases is another document in Docushare and cotains information 'From Developer To Developer' (see link to docuhare).

3.1.3. HMIContract

The HMI contract should be the starting point when developing a new HMI application. This contract deals as the interface between all involved parties (Model/HALL, View, Controller). It defines constants, messages, structures of the datamodel for later databinding (to be done in SceneComposer), incoming events and outgoing actions of the statemachine, scenes to be modeled in SceneComposer and to be controlled in VisualState Statemodel and last but not least Widget definition (mainly to be used for widget development).

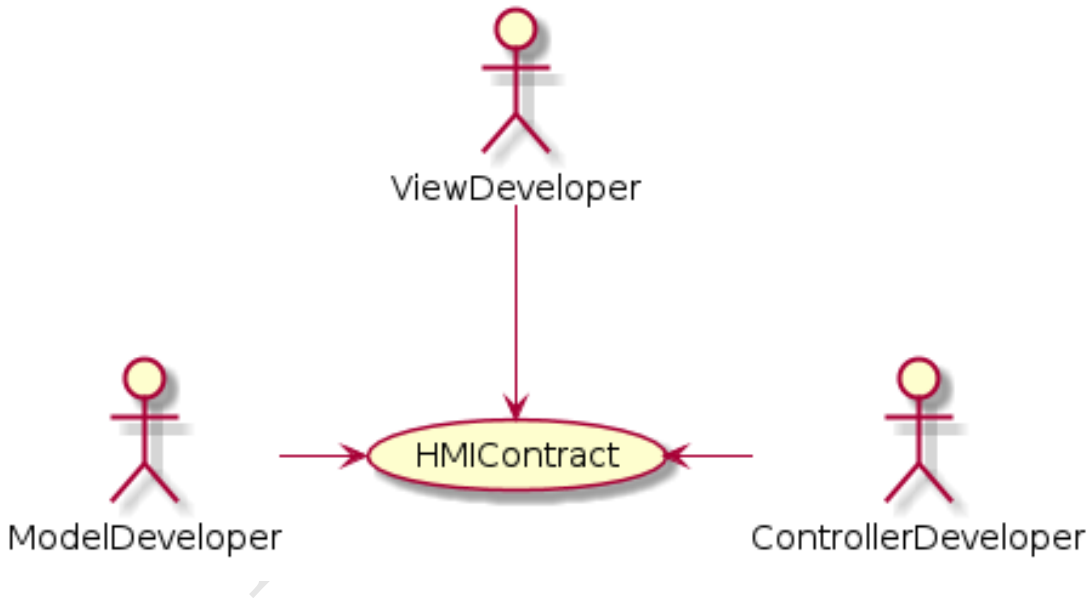


Figure 3.2.: HMIContract as development base

Contract files use the file extension 'xhcdl' and are based on schema files as it is common for xml dialects. Please find a detailed documentation about all the key words and some examples under XHCDL documentation.

Almost all elements of the contract are identifiable by a unique id

```
<message name="DummyChangedUpdMsg" uid="{CAB96F02-129B-4227-ACBC-DF8C00C4C11I}">
<subscribers>
  <subscriber id="Model" />
</subscribers>
```

```
<member name="DummyString" type="Candera::String" uid="{932ADB57-97CD-484A-8DE1-9
    BE59F3BACI7}"></member>
<member name="DummyValue" type="Courier::UInt32" uid="{932ADB57-97CD-484A-8DE1-9
    BE59F3BACJ7}"></member>
    </message>
```

When writing a contract, these uids can be filled in manually, but it can also be done during the generation by the generator. Therefore the above shown definition has to look like below, what makes life much easier and less error prone:

```
<message name="DummyChangedUpdMsg" uid="">
<subscribers>
  <subscriber id="Model" />
</subscribers>
<member name="DummyString" type="Candera::String" uid=""></member>
<member name="DummyValue" type="Courier::UInt32" uid=""></member>
    </message>
```

ATTENTION: Do never touch already existing UIDs, otherwise the synchronization of the elements with SceneComposer and VisualState will fail!!!

3.1.4. Code Generators

Based on the HMI contract files, there are mainly two generator running with different aims.

HMI Contract Generator

First of all the content of the HMI contract has to be available as some kind of source code. Therefore the executable 'HMIContract.exe' is used. It produces C++ source code containing different classes, functions and definitions to be used later on in the different components of the application.

As the executable provides different configuration items, a script should be used to run the generator, so that the configuration can be controlled at a central place. This script is provided by HMI-Base team, see chapter 3.4 General Automation .

HMI Contract to VisualState Generator

The second generator that is present is the 'HMIContract2VS.exe'. This one will not produce source code, but will be used to synchronize the VisualState project with the HMI contract files. That means whenever a new element has been added to the contract (e.g. a new scene or a new statemachine action) or an already existing element has been renamed, the generator will update the project files accordingly.

3.1.5. CGI Studio Tools

CGI Analyzer

The CGI Analyzer tool allows system benchmarking and performance analysis before the application is deployed on the target. It provides valuable feedback regarding the hardware feasibility of the system. The Analyzer helps to assess application performance, detect bottlenecks and optimize the final design. In offline mode it is possible to examine the applications performance log at a later point in time.

Pre-Release

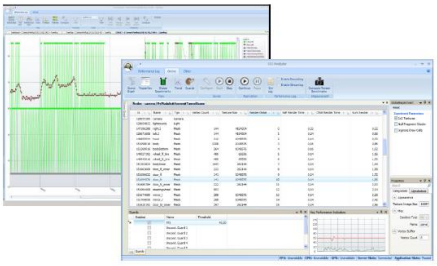


Figure 3.3.: CGI Analyzer

Asset Shaper

The CGI Asset Library Shaper allows creation, management and partitioning of several Asset files based on an XML rule set, which can be set individually for each project.

Partitioning an Asset Library provides the following advantages:

- Ease of variant handling
- Reduction of flash time for partitioned updates
- Faster boot up time

The Asset Library Shaper is also required for multi language support. Additionally the CGI Asset Library Verification component is a helper tool, which supports the user in managing his asset files in larger environments or projects with multiple variants. It provides means for verification of matching asset library / widget / application versions and allows to assign asset libraries to their originating solution.

3.2. Build System and Setup

3.2.1. System Requirements

Windows 7/Vista/XP 6 GB System RAM Consumer GPU with full OpenGL 2.x or higher support. This is actually any graphic card that supports DirectX 10 or higher (Example: Radeon hd 4350, NVidia GeForce 8600 or better).

3.2.2. Installation

Installing CGI Studio: Once you have the zip file, unzip it to the desired installation root directory (No separate installers required). To get an overview of CGI_Studio directory, click here.

Add link to "click here" in Toolchain/Build System/Installation

Installing the Dongle License: The folder "License" contains the licensing software required for running Scene Composer. Click on the following link for the license: License.

Pre-Release

Note: Execute runme.bat from delivered license folder. This will install License Software from HASP. Insert the dongle to an USB port. Invoke Scene Composer from the CGI Studio installation directory. Scene Composer shall start up.

As the executable provides different configuration items, a script should be used to run the generator, so that the configuration can be controlled at a central place. This script is provided by HMI-Base team, see chapter 3.4 General Automation .

3.2.3. Setting up Environment

Add content to: How To/Build System/Setting up Environment

3.3. Simple Application Sample

Add content to: How To/Simple Application Sample or is this section not needed?

3.4. General Automation

In order to achieve consistent code generator results, some scripting is available to prevent the user from using the graphical frontend of the generator and to change the settings there. It is only allowed to use the script in the '_scripts' folder of an application. The only exception to this rule is the code generator of VisualState, which can directly be started from the VisualState Navigator panel (pressing F9).

Available scripts in applications '_scripts/' folder:

- GenContract.cmd -> Genereate the contract files in HMI/HMIContract/preliminary
- GenContract2VS.cmd -> Synchronize the VisualState project in HMI/SM/config with the HMI contract content
- SceneComposer_StartSolution.cmd -> start the SceneComposer from ai_hmi_cgi/tools/CGI_2.10.0 with the latest SCHost from ai_hmi_cgi/tools/CGI_2.10.0/SCHost and open the solution of the application
- SceneComposer_Start.cmd -> start the SceneComposer from ai_hmi_cgi/tools/CGI_2.10.0 with the latest SCHost from ai_hmi_cgi/tools/CGI_2.10.0/SCHost only
- SceneComposer_GenerateAssets.cmd -> Generate the asset out of the solution

Pre-Release

4. Framework Features

4.1. Introduction

This section will describe features that are (basically) supported by the HMI application framework. Base for feature support will be the availability of all required components and infrastructure within the AI-Distribution. There will be no project specific implementation supported from framework side.

4.2. Application Framework

4.2.1. HmiAppControl Service

In order to control HMI applications from a central point, what is the AppHmi_Master, the service called *HmiAppCtrl* provides some basic functionality. Figure 4.1 shows simplified the sequences that are supported

Pre-Release

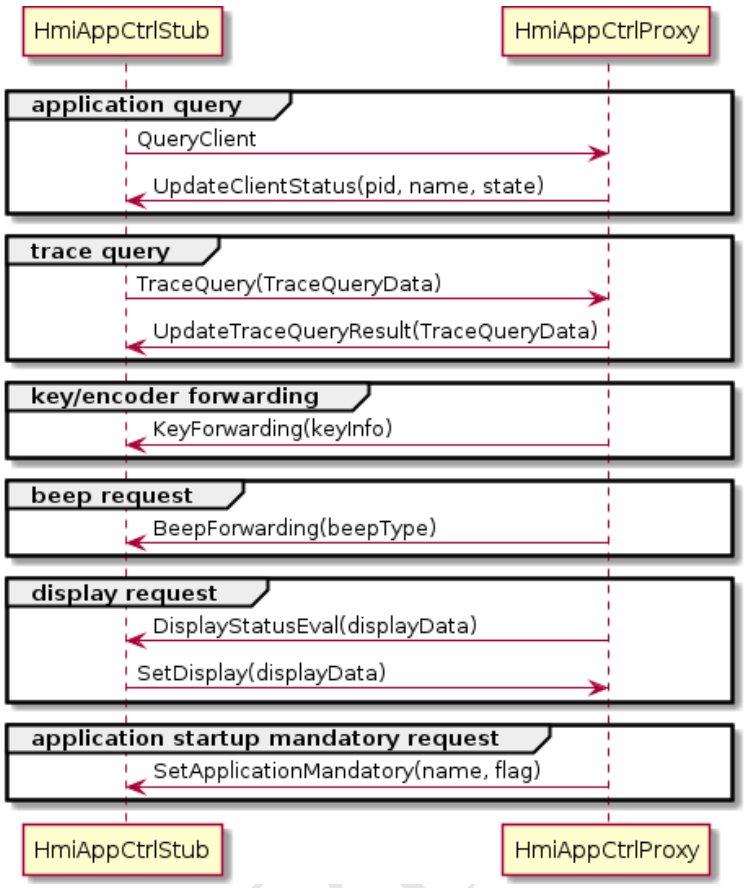


Figure 4.1.: HmiAppCtrl methods and signals

The service is realized as Franca-DBus service.

Signal/Method	Description
QueryClient/UpdateClientStatus	The stub sends out a query broadcast message and a client has to answer with UpdateClientStatus transmitting its process id, a maningful name and a state property, indicating if the application is ready to work or still initializing or not available due to other reasons.
TraceQuery/UpdateTraceQueryResult	The stub sends a broadcast transmitting an id for the trace query. The signal can also transport a process id, when only a particular client has to answer the query, but this is optional. The client has to answer with UpdateTraceQueryResult, it will again transmit the process id of the client, the trace query id originally received and a free text, which will be traced out by the stub.
keyForwarding/encoderForwarding	These messages can be used to forward (unprocessed) key and encoder events from an HMI application to the AppHmi_Master application.
beepForwarding	This can be used to forward a beep request through AppHmi_Master to Genivi AudioManager.
display request	Display related methods and signals to request the display or at least parts of it, the exact sequence will be explained on an example, see chapter 5.5 Status Bar .
application startup mandatory request	

Add info about ”application startup mandatory request” in Framework Features/Application Framework/ HMIAppControl Service

Figure 4.2 shows a simplified class diagram on how to instanciate the required objects to use the HmiAppCtrl service. Please pay attention that AppHmi_Master has a special role in this setup, as it instanciates a proxy as well as the (one and only) stub. This has the advantage that there has to be not specialized ways to communicate from stub to AppHmi_Master as it is the same as for all other applications.

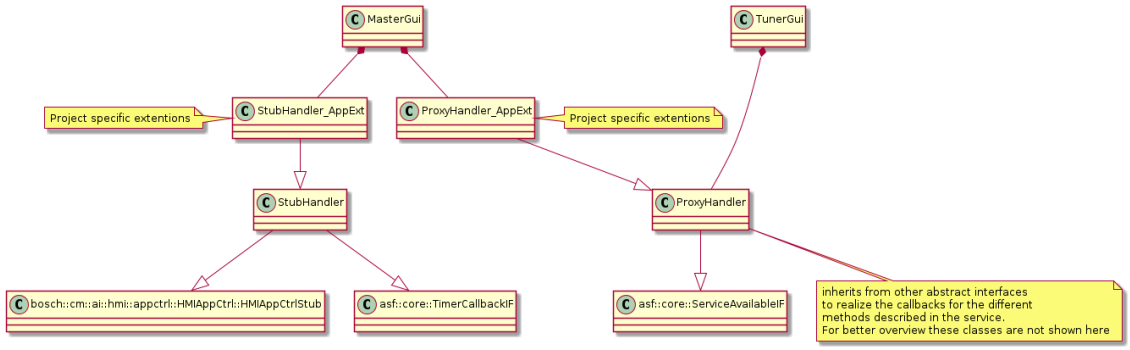


Figure 4.2.: Simplified class diagram of HmiAppCtrl usage

4.2.2. Language Change

A basic functionality of HMI is the language change. As of now there is no support/middleware component in the AI-Distribution to distribute the system language on a Gen3 platform. Anyhow this topic should not be left out here.

Figure 4.3 shows a language change via GUI menu in principal.

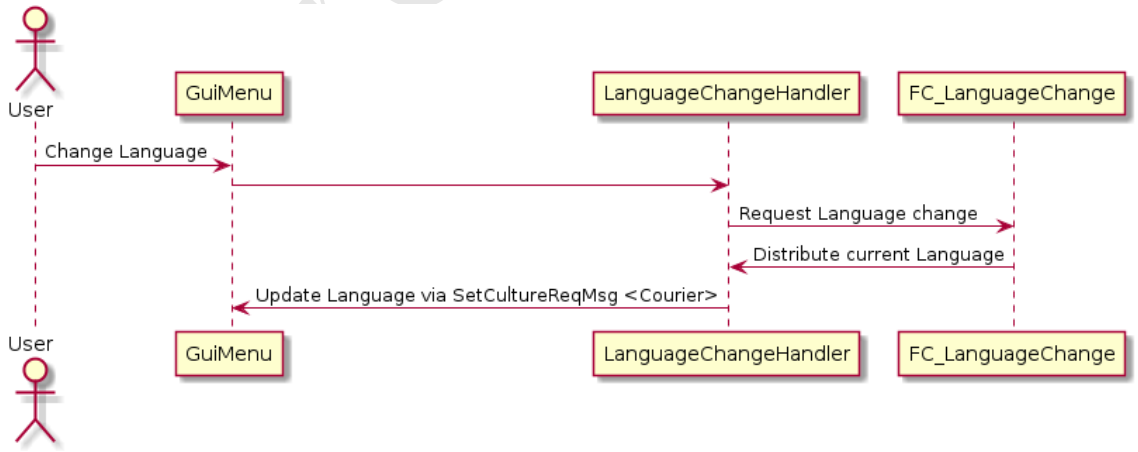


Figure 4.3.: Distribute language in HMI

As long as the service as well as the middleware component is not part of the platform, this feature

has to be implemented in the project and can not be supported by the HMI framework (except the Courier message definition, which is already there).

4.2.3. Connection to LifeCycleManagement (LCM)

As an HMI application will be set up as ASF application, there is no connection to the power management by default, as it was well known for CCA-Applications. In order to be controllable by the LifeCycleManager, HMI framework provides a basic implementation to satisfy the basic needs of the LCM. Therefore every application has to setup an own DBus Service.

Unfortunatly, the configuration of ASF applications and components do not support inheritance as of now(ASF V1.8) of base sets for such. So this has to be done by each application. Please find below an example snippet of the application cma file, showing how the LCM connection has to be configured.

```
// Application controlserver xml
import "../../../../../../../ai_g3g/components/api/NodeStateManager/org.genivi.NodeStateManager.Consumer.
    xml"
// Application controlclient xml's
import "../../../../../../../ai_g3g/components/api/NodeStateManager/org.genivi.NodeStateManager.
    LifecycleConsumer.xml"
import "../../../../../../../ai_g3g/components/api/NodeStateManager/org.genivi.NodeStateManager.
    LifecycleControl.xml"
application My_App version 1.0.0
{
 DBusConnector dbusSessionConnector
 {
    busType = Session
 }


   thread thHmiMaster
 {
    immediate MyGui myGui
    immediate MyHall myHall
 }

 // interface of LCM to register the application interface
 org.genivi.NodeStateManager.Consumer LcmConsumer = dbusSessionConnector.importByName ("org.bosch.cm
     .lcm.LcmAsf", "/org/genivi/NodeStateManager/Consumer")
 bind myHall.ConsumerPort to LcmConsumer
 org.genivi.NodeStateManager.LifecycleControl LcmLifecycleControl = dbusSessionConnector.importByName ("
     org.bosch.cm.lcm.LcmAsf", "/org/genivi/NodeStateManager/LifecycleControl")
 bind myHall.LifecycleControlPort to LcmLifecycleControl
   // application interfaces for LCM
 dbusSessionConnector.exportByName(myHall.LifeCycleConsumerPort, "/org/genivi/NodeStateManager/
     LifeCycleConsumer/My_App")
}
```

```
// Application controlserver xml
import "../../../../../../../ai_g3g/components/api/NodeStateManager/org.genivi.NodeStateManager.Consumer.
    xml"
// Application controlclient xml's
import "../../../../../../../ai_g3g/components/api/NodeStateManager/org.genivi.NodeStateManager.
    LifecycleConsumer.xml"
import "../../../../../../../ai_g3g/components/api/NodeStateManager/org.genivi.NodeStateManager.
    LifecycleControl.xml"
```

```
component MyHall version 1.1.0
{
 // Application control ports for LCM
 // requires interfacename
 requires org.genivi.NodeStateManager.Consumer ConsumerPort
 requires org.genivi.NodeStateManager.LifecycleControl LifecycleControlPort
   // provides interfacename
 provides org.genivi.NodeStateManager.LifeCycleConsumer LifeCycleConsumerPort
   requires CFC_SPM_FI spmFiPort
}
```

A class to handle these things is instanciated in the Hall base implementation, see Figure 4.4

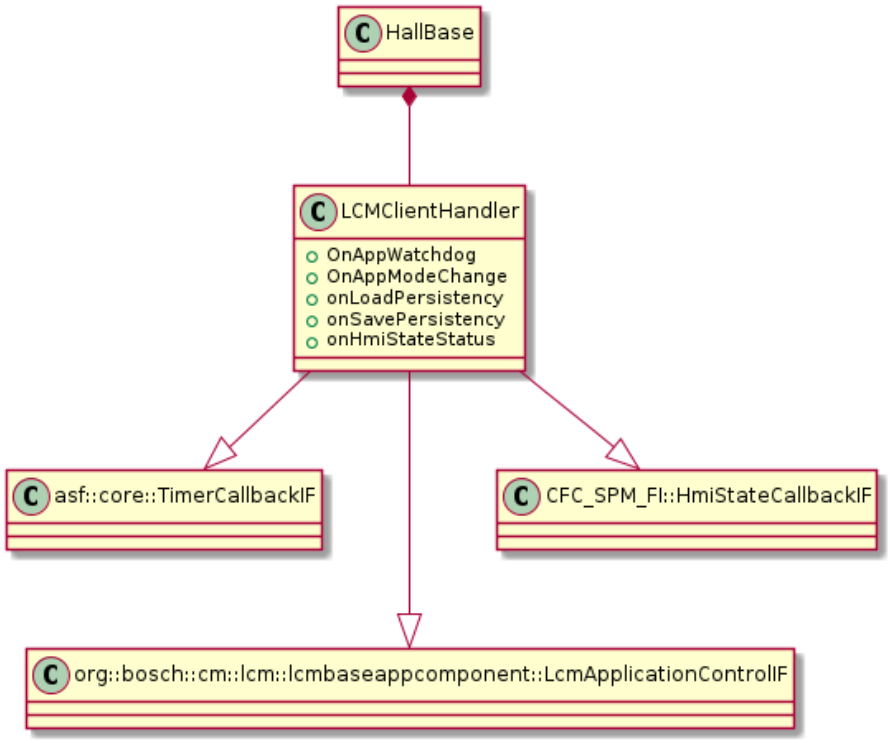


Figure 4.4.: Lcm client handler

.

As of now the implementation is doing only the minimum to satisfy LCM (Watchdog is answered as well as application mode changes are confirmed, Save/LoadPersistency not yet supported). This has to be extended later on.

Additionally there is a CCA property from SPM for historical reasons that indicates a so called HMISubState. This is still useful, as the new LCM is made for DBus applications and these only know the powerstates ON and OFF. But for automotive it is required to differentiate also some states in between. Therefor the old SPM property will be reused to. This will provide some sub power states that are only of interest for HMI (HMI_Off, HMI_On, HMI_Restricted etc.). Whenever an update is received on this property, a Courier message is posted to the controller component of the application that may react on it. Its up to the application developer if there is a need to do so.

The message definition is done in the base contract and so it is available for all the applications, that can react on that in their corresponding statemachine.

```
<message name="HMISubStateChangedUpdMsg" uid="">
<subscribers>
  <subscriber id="Controller" />
</subscribers>
<member name="SubState" type="HMIStateEnum" uid=""></member>
</message>
```

4.2.4. Screenshot

Screenshot functionality to be implemented

4.3. Audio Client

4.3.1. Overview

The HMI framework brings a basic support for audio source switching and control of different sound properties. The concept of audio control from HMI application basically looks like Figure 4.5 .

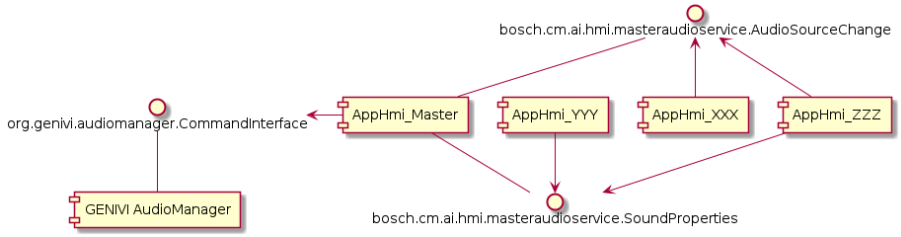


Figure 4.5.: Component diagram of audio issues controlled by HMI applications

To minimize synchronization problems between audio sources and visual feedback there is only one HMI application connected to the GENIVI AudioManager DBus interface. This application is the AppHmi_Master, because this one will also take care about general HMI application switches and so both threads can be synchronized into a single statemachine. To give also other HMI applications the possibility to request audio related things, the AppHmi_Master will provide two services as well for audio source switching and sound properties. So these can be used by other HMI applications instead of directly talking to GENIVI AudioManager.

4.3.2. Audio Source Control Service

This service encapsulated all topics from the AudioManager interface related to audio source switching. The service itself is set up as Franca-DBus service. HMI framework brings an implementation for basic source change, that can be stimulated via this service. A more detailed explanation on how to control source switching will be available in chapter 6.2.1 AppHmi_Master (the service stub) and chapter 6.2.2 AppHmi_Tuner (the service proxy).

Pre-release

Supported features by this service:

- source activation/deactivation -> activate source using the source id (with additional device id for media sources that are dynamically assigned by the AudioManager component, by default it is -1)
- request for current source list -> source id, device id, source availability and availability reason
- indication about source activated
- indication about changed source list

A source switch in general looks like shown in Figure 4.6 .

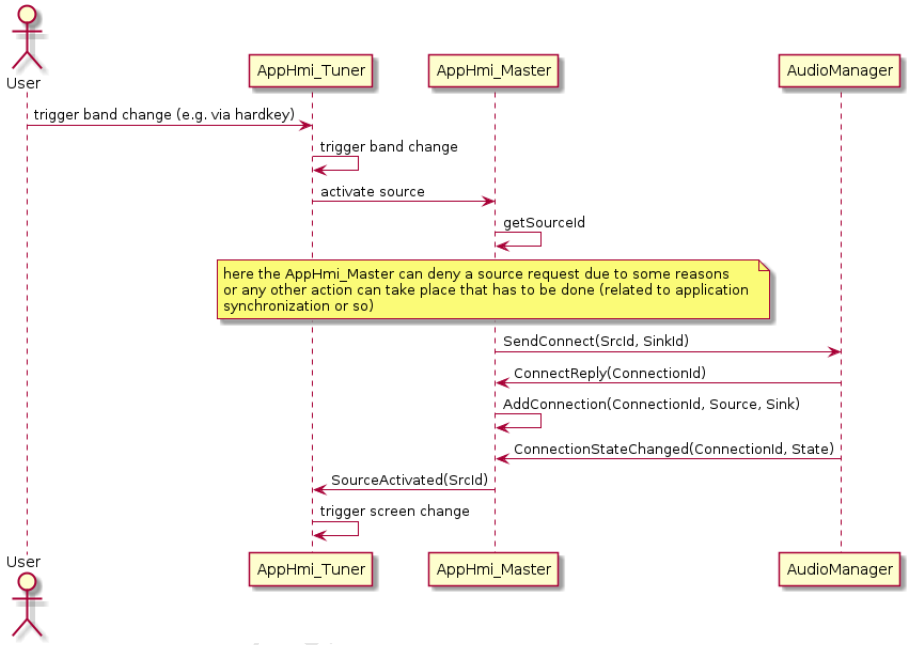


Figure 4.6.: Simplified audio source switch initiated by user

4.3.3. Sound Settings Service

Additionally to the audio source switch functionality, there is a service for sound settings available that will provide sound related functionality. The reason to have a separated service and not to use the original sound interface from AudioManager is the same as already described in chapter 4.3.2 Audio Source Control Service .

The functionality of this service is currently restricted to the basiscs

- distribute current volume value
- distribute current mute state
- request mute state change

Functionality of Sound Settings Service has to be extended to cover the whole AudioManager functionality

4.4. Startup Animations

Generally a device will always have a startup animation to cover startup times. The support of the same is by intent NOT part of the here described HMI application framework. In the following table some Pros and Cons for implementing a startup animation within an HMI application are listed to give a recommendation finally.

PRO:

- all possibilities related to graphic, due to the availability of OpenGL and LayerManager

CONTRA:

- usually HMI is started late
- wide range of dependencies (OpenGL, EGL, LayerManager, etc.)
- long lasting startup due to graphic initialization

So the recommendation would be to implement a startup Animation in a tiny application with less dependencies, which can be started much earlier and will not have big impact on the startup of other components/processes.

4.5. Tracing/Debugging

For tracing and debugging the standard mechanisms of CM-AI (ETG, DLT and GDB) are used. General informations can be found in different links

ETG Manual

DLT in G3g

Using gdbserver and Eclipse

For an HMI process there are several trace ids and channels predefined. Originally these can be found in ai_osal_linux/components/bosch_includes/trace/mc_trace.h. HMI framework makes use of some of them and provides the rest for HMI applications. Therefor refer to the implementation of the reference applications and there especially to the header *Project_Trace.h*.

Pre-Release

4.5.1. Tracing the framework

All trace defines related to HMI framework are available in *Framework_Trace.h*.

Trace Class	Description
TR_CLASS_HMI_FW	general information from HMI framework
TR_CLASS_HMI_FW_INPUT	information related to input
TR_CLASS_HMI_FW_COMMON_APPCTRL	master trace class of the HmiAppCtrl service, chapter 4.2.1 HmiAppControl Service , to be enabled in addition to an application specific class related to HmiAppctrl proxy, chapter 5.4.2 Tracing
TR_CLASS_HMI_FW_APPCTRL_STUB	to be enabled to see the stub side of HmiAppCtrl
TR_CLASS_HMI_CGI	master trace class for all CGI related trace information, chapter 5.4.2 Tracing
TR_CLASS_HMI_LISTPROVIDER	master trace class for list related informations
TR_CLASS_HMI_LISTPROVIDER_DEFAULT	to be removed, application has to provide a trace class for list tracing by its own
TR_CLASS_HMI_PERFORMANCE_MP	to be enabled to see performance related measure points
TR_CLASS_HMI_LANGUAGE_HANDLER	not activated as of now
TR_CLASS_HMI_FW_AUDIO_IF	general information about audio related topics described in chapter 4.3.2 Audio Source Control Service
TR_CLASS_HMI_FW_SBCLIENT	information regarding communication to/from Screen Broker
TR_CLASS_STDOUT_TO_TTFIS_*	ASF logs from stdout routed to TTFis. This trace class has to be established for each application individually. For detailed information about ASF logs, please have a look into the ASF User Guide

Additionally to the above mentioned trace classes, which will be used for TTFis output, the Screen Broker (SB) component uses DLT as trace mechanism. These traces are not switchable and can be seen all the time in DLT viewer or TTFis (using Gen3Flex@DLT). If the DLT traces are de-activated by default on the target, you can activate DLT traces for SB with sending these commands via the TTFis channel Gen3Flex@DLT:

Pre-Release

```
DLT_SET_LOG_LEVEL sb kh DLT_LOG_VERBOSE
DLT_SET_LOG_LEVEL sb pm DLT_LOG_VERBOSE
DLT_SET_LOG_LEVEL sb sb DLT_LOG_VERBOSE
DLT_SET_LOG_LEVEL sb sl DLT_LOG_VERBOSE
DLT_SET_LOG_LEVEL sb ut DLT_LOG_VERBOSE
DLT_SET_LOG_LEVEL sb wl DLT_LOG_VERBOSE
DLT_SET_LOG_LEVEL sb act DLT_LOG_VERBOSE
DLT_SET_LOG_LEVEL sb ani DLT_LOG_VERBOSE
DLT_SET_LOG_LEVEL sb lma DLT_LOG_VERBOSE
DLT_SET_LOG_LEVEL sb plm DLT_LOG_VERBOSE
DLT_SET_LOG_LEVEL sb pma DLT_LOG_VERBOSE
DLT_SET_LOG_LEVEL sb sba DLT_LOG_VERBOSE
DLT_SET_LOG_LEVEL sb set DLT_LOG_VERBOSE
DLT_SET_LOG_LEVEL sb sta DLT_LOG_VERBOSE
```

Use this command to store the changes permanently: DLT_STORE_CONFIG Best is to put them in a script and execute this in TTFis.

You can also enable the log of all DLT components with this command: DLT_SET_DEFAULT_LOG_LEVEL DLT_LOG_VERBOSE These commands work only with TTFis 4.8 onwards, if you are using an older version you can set the default log level also with changing this file on the target: */etc/dlt.conf*. In this file change the 'ApplicationDefaultLogLevel' to '6' (verbose):

```
###################################
# Application Specific settings #
###################################
# The defaul log level to be set for applications during startup (Default: 4)
# DLT_LOG_OFF = 0, DLT_LOG_FATAL = 1, DLT_LOG_ERROR = 2, DLT_LOG_WARN = 3,
# DLT_LOG_INFO = 4, DLT_LOG_DEBUG = 5, DLT_LOG_VERBOSE = 6
ApplicationDefaultLogLevel = 6
```

For more information regarding DLT, please contact TTFis-Team at ECO.

The HMI framework also supports the following trace commands to ask for a certain state or infor-

Pre-Release

mation:

Trace Command	Decription
HMI_Get_Visible_Screen	Show the name of the current visible main scene on screen
HMI_Get_Top_Popup	Show the name of the current top popup scene.
HMI_Get_Top_Visible_Popup	Show the name of the current visible top popup scene on screen. In case a popup filter is active, this popup might not be the top popup. In all other cases the result will be the same as with HMI_Get_Top_Popup.
HMI_Get_Popup_Stack	Show the names of all popup scenes on stack
HMI_Get_Top_Active_Surface	Show the id of the current active and topmost main surface
HMI_Get_Top_Active_Surface_Non_Permanent	Show the id of the current active and topmost main surface except non permanant surfaces
HMI_SMInfo	broadcasts signal to all HmiAppCtrl proxies to trace out their current statemachine situation
HMI_StartupInvestigation	Summary of all measurements related to HMI startup
HMI_FEATMemoryStatistics	memory allocation statistics via FEAT macros. Feature switches in FeatStd/Config.h have to be enabled • FEATSTD_SYSTEM_MEMORY_STATISTIC_ENABLED • FEATSTD_SYSTEM_MEMORY_STATISTIC_FILE_AND_LIN

Pre-Release

Part III.

HowTo: Hmi Application

Pre-Release

5. HowTo

5.1. Main Goals

This part describes a general way to setup an HMI application with help of the mechanisms provided by HMI-Base. It will give an impression what to do to configure the application.

5.1.1. Workflow

The principal workflow is shown in Figure 5.1 .

5.2. Guidelines and Naming Conventions

5.2.1. SourceCode

For source code (cpp and h/hpp) the generic coding rules for Gen3 have to be followed, Coding rules

5.2.2. HMI-Contract

Constants and Types

1. Constants and enum values have to be in UPPERCASE
2. Do not use any prefix for data type
3. Use a meaningful name
4. Use underscore as separator
5. Recommendation: To avoid name clashes use namespaces before extending the constant/enum/-value name

Example

```
<constants>
  <const name="RED" type="uint8" uid="" value="1" />
  <const name="GREEN" type="uint8" uid="" value="2" />
  <const name="BLUE" type="uint8" uid="" value="3" />
  <const name="LIGHT_GREY" type="uint8" uid="" value="4" />
</constants>
```

Pre-Release

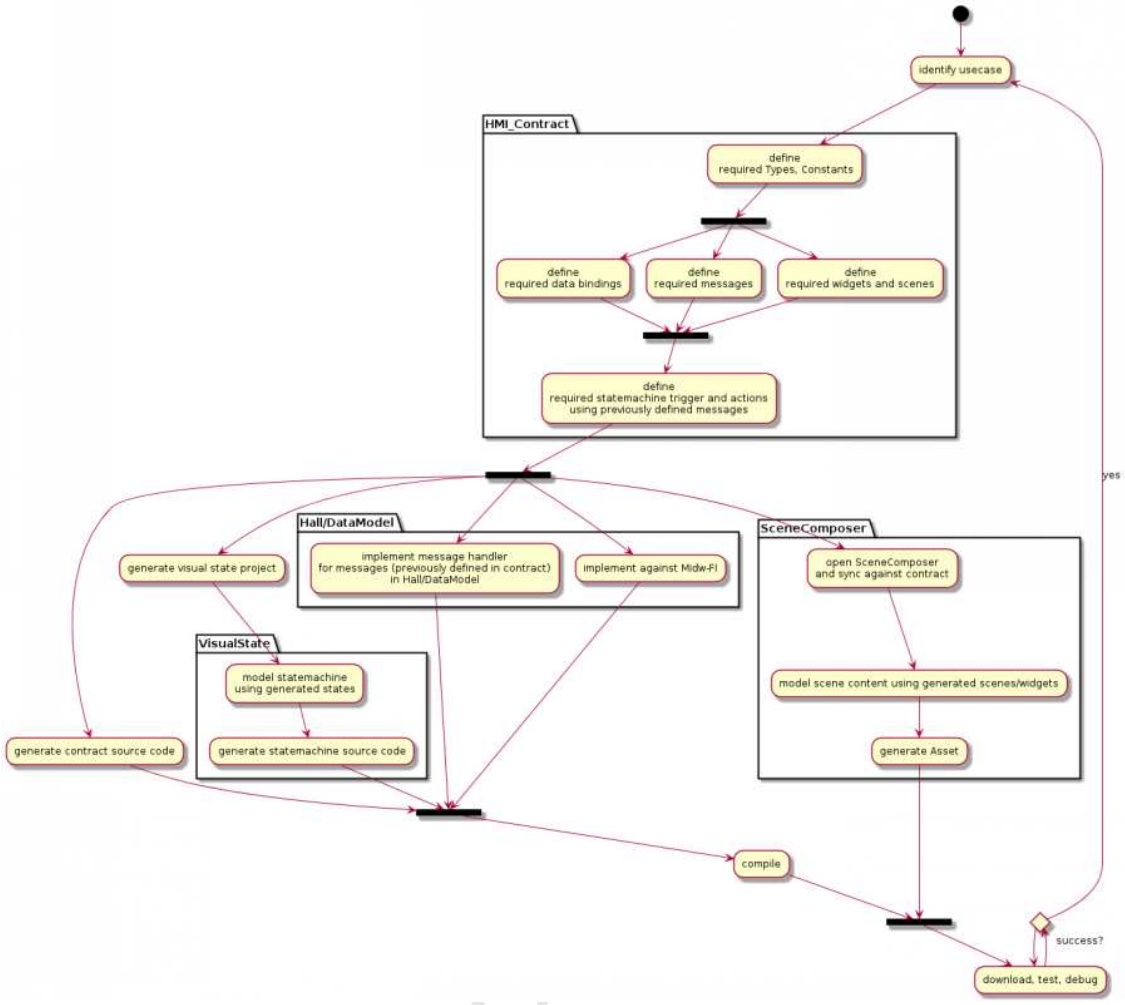


Figure 5.1.: Workflow

```
<types>
  <enum name="Colors" uid="">
    <define name="BLACK" value="1" uid=""/>
    <define name="WHITE" uid=""/>
    <define name="VIOLETT" uid=""/>
  </enum>
</types>
```

Views

1. Modulename have to be CamelCase and as short as possible (max ˜8 chars), first letter uppercase, no underscore
2. View name -> refer to HMI Spec, normally UPPERCASE

3. widget names have to be CamelCase, starting with widget type (without widget suffix 'Widget2D' and 'Widget3D'), first letter uppercase, when more than one instance of a type is used, use underscore and meaningful name decribing the functionality

4. Recommendation: Use widget prefix when name is not clear. Prefix should be lowercase and max 3 chars

Example

```
<modules>
  <module name="M1" uid="" regionPolicy="unify" exportIdentifier="">
    <view name="Scenes" uid="" regionPolicy="unify">
      <view2D name="DUMMY_MAIN" uid="" exportIdentifier="">
        <!-- no prefix as the name is clear-->
        <widget name="Speller" typeRef="SpellerWidget2D" uid="" />
        <!-- prefix as the name is not clear-->
        <widget name="Button_Next" typeRef="ButtonWidget2D" uid="" /-->
      </view2D>
    </view>
  </module>
</modules>
```

Messages

1. Use suffix 'ReqMsg'/ResMsg/UpdMsg' all over the place, no other suffix allowed
   - Commands: Starting with meaning full verb in present tense, suffix 'ReqMsg'
   - Confirmations: same like command but with suffix 'ResMsg'
   - Status messages: meaningful verb in past tense directly before suffix 'UpdMsg'

2. Message names have to be CamelCase, first letter uppercase, no underscore

3. Message member names have to be CamelCase, first letter uppercase, no underscore, no data type prefix

ATTENTION: The naming of message member names contradicts to the C++ coding rules, as a generator from an external company is processing the message definition. So the contradiction is done by intend.

Example

```
<message name="ChangeSomethingReqMsg" uid="">
  <subscribers>
    <subscriber id="Model" />
  </subscribers>
  <member name="DummyString" type="Candera::String" uid=""></member>
  <member name="DummyValue" type="Courier::UInt32" uid=""></member>
</message>

<message name="ChangeSomethingResMsg" uid="">
  <subscribers>
    <subscriber id="Controller" />
  </subscribers>
  <member name="DummyValue" type="Courier::UInt32" uid=""></member>
```

Pre-Release

```
</message>

<message name="DummyValueChangedUpdMsg" uid="">
  <subscribers>
    <subscriber id="Model" />
  </subscribers>
  <member name="DummyValue" type="Courier::UInt32" uid=""></member>
</message>
```

DataBinding

1. Data binding names have to be CamelCase, first letter uppercase, no underscore
2. Data binding item names have to be CamelCase, first letter uppercase, no underscore, no data type prefix

Example

```
<bindingSources>
  <bindingSource name="DummyDataSet" readonly="false" uid="">
    <item name="DummyValue" type="Courier::Int8" defaultValue="0" readonly="false" uid=""/>
    <item name="DummyString" type="Candera::String" defaultValue="0" readonly="false" uid=""/>
  </bindingSource>
</bindingSources>
```

State Machine

1. Messages: prefix 'on' plus message ref name plus optional parameter value. When parameters are predefined, these can be listed afterwards, separated by underscore, use alias names for parameters, when message name becomes unhandy because of the length
2. Actions: prefix 'ac' plus message ref name, prefix 'gac' for action defined outside of the application contract (e.g. in the project contract). When parameters are predefined, these can be listed afterwards, separated by underscore, use alias names for parameters, when message name becomes unhandy because of the length. Use varible parameters whereever possible.
3. Functions: prefix 'fc' plus meaning full name in CamelCase, first letter (after prefix) uppercase, no underscore
4. Function parameters: CamelCase, first letter lowercase, no underscore, no data type prefix
5. Dataguards: prefix 'dg' plus meaning full name in CamelCase, first letter (after prefix) uppercase, no underscore

Example

```
<message name="onSomethingChanged" messageRef="SomethingChangedUpdMsg" uid=""/>

<message name="onSomethingChanged_RED" messageRef="SomethingChangedUpdMsg" uid="">
  <map nameRef="Value" valueRef="RED" />
</message>

<action name="acDoSomething" messageRef="DoSomethingReqMsg" uid="" />
```

Pre-Release

```
<function name="fcDoSomeOtherThings" returnType="void" uid="">
  <arg name="myParam" type="uint16" uid="" />
</function>

<dataGuard name="dgMyValue" itemRef="BlaBla/Dummy" returnType="bool" uid="" />
```

5.2.3. VisualState

1. Internal state: CamelCase with meaningful name, no underscore
2. Variables: UPPERCASE, use underscore as separator
3. Signals: prefix 'si', CamelCase, first letter (after prefix) uppercase, no underscore

5.2.4. SceneComposer

1. widgets: see chapter 5.2.2 Views
2. composites: CamelCase, first letter uppercase, start with type name ()like 'Button', 'List', 'Speller' etc), use underscore to add additional information about different variants, Figure 5.2

5.3. HMI-Contract as central development tool

As Figure 5.1 already shows, the HMI contract and its definition is the base for further development. A detailed description of all elements/keywords can be found in HMI-Contract schema. In the following some basics an also important key words should be shown, to use the contract in an efficient way.

5.3.1. Motivation to have a proper contract

The definition of a proper contract helps the developer team to work in parallel and so goals and milestones can be achieved faster. It will also ease up the work with the different modeling tools, as the main skeleton can be described in a text form and so refactoring (e.g. renaming object) will be done automatically. Additionally the contract offers the possibility to have a good documented and easy to extend API.

5.3.2. HmiContract Editor

As long as there is no HmiContract editor available, it is recommended to use an editor that supports auto-completion based on xml schema file, e.g. Microsoft VisualStudio 2010 or 2012.

5.3.3. Key features

PARADIGMA: Define everything in the contract that is intended to be exchanged between the components (MVC), to be used by the statemachine project or the SceneComposer solution

The major things to be defined in the contract are

Pre-Release

- Constants
- Data types like enums
- Databinding structures
- Messages to be send between MVC (inner process communication)
- Scenes and containing widgets instances
- Widgets

Generic scope

Nearly all elements have to have a uid. This id will be used by the generators to identify the element later on in the already generated text files of SceneComposer and VisualState. Please make use of the XML validator feature of your editor to identify if the element needs a uid keyword or not.

```
<message name="ChangeDummyReqMsg"uid=""/>
```

The value of 'uid' has not to be filled manually, since the scripting to generate the contract is doing so automatically. Therefore the value is left open. Whenever an uid has been assigned, do not change it later on. This will break the synchronization functionality for this object.

Sometimes it might be helpful to add some additional includes to the code to be generated or to encapsulate it in a specific namespace (to be used where it makes sense to avoid name clashes). This can be achieved with the section *generator*.

```
<generator location="MyContract/generated">
  <namespace namespace="mynamespace" folder="false"/>
  <header>
    <include file="MyHeader.h"/>
  </header>
  <source>
    <prolog file="MyFile.txt" />
  </source>
</generator>
```

The code above shows the possibilities of the *generator* section. A default location for the generated source code can be defined, as well as a namespace for the generated content, which in turn can be placed in a separate folder. Additionally some other key words can be used in the child sections *header* and *source* to modify the header and source file. Please have a look in the schema documentation for further key word explanation, HMI-Contract schema.

Scope 'Constants'

The section 'constants' gives you the possibility to define constant values of standard data types.

```
<constants>
  <const name="MY_DUMMY_CONSTANT" type="int8" value="5" uid=""/>
</constants>
```

This will later on be visible as C-define in the generated code.

Scope 'Types'

Use the 'types' section to identify enums to be used later on e.g. in messages.

```
<types>
  <enum name="myEnum" uid="">
    <define name="RED" value="0" uid=""/>
    <define name="GREEN" uid=""/>
    <define name="BLUE" uid=""/>
    <define name="UNDEFINED" value="255" uid=""/>
  </enum>
</types>
```

Scope 'DataBinding'

The definition of data binding sources is mandatory to fill dynamic data into the widgets in SceneComposer.

```
<bindingSources>
  <bindingSource name="MyBindingSource" access="asynchronous" uid="">
    <item name="MyString" type="Candera::String" defaultValue="0" readonly="true" uid=""/>
    <item name="MyValue" type="int8" defaultValue="0" uid=""/>
    <string name="myString" defaultValue="noop" uid=""/>
    <array name="myArray" type="int8" length="5" validateType="true" uid=""/>
  </bindingSource>
<bindingSources>
```

A *bindingSource* will group a couple of data items/strings/lists/arrays. All these can be marked as read-only using the keyword *readonly* with value 'true'.

Please have a look in the schema documentation to get a more detailed idea about the power of bindingSource: HMI-Contract schema.

Clarify: What is the impact of 'asynchronous' bindingSource?

Scope 'Messages'

In order to exchange data or to trigger actions, messages have to be defined. Use the keyword 'message' to define a message. Other important keywords are 'subscriber' and 'member' defining a subscriber for the message (Model, View, Controller, Custom) and members of any type. Also self defined types from the 'types' section can be used.

```
<messages>
  <message name="ChangeDummyReqMsg" uid="">
    <subscribers>
      <subscriber id="Controller"/>
    </subscribers>
    <member name="DummyMember" type="int8" defaultValue="0" uid=""/>
  </message>
</messages>
```

Pre-Release

The schema for messages contain a lot more keywords that might help you with your specialized problem. Please have a look in CourierMessageTypes.xsd and the documentation HMI-Contract schema.

Scope 'Widgets'

Ask Helmut to fill this section

Scope 'View'

The code example below shows in principle a module with a view definition:

```
<modules>
  <module name="MyModule" uid="" regionPolicy="unify" exportIdentifier="">
    <view name="Scenes" uid="" regionPolicy="unify">
      <view2D name="MAIN" uid="" exportIdentifier="">
        <widget name="btnNext" typeRef="ButtonWidget2D" uid="" />
      </view2D>
      <view3D name="SETTINGS" uid="" exportIdentifier="">
        <widget name="btnBack" typeRef="ButtonWidget3D" uid="" />
      </view3D>
    </view>
  </module>
</modules>
```

The most important keywords here are

- *module*: define a module that will finally group the containing views in SceneComposer, resp. States in VisualState
- *view/view2D/view3D*: define a view/a collection of views
- *widget*: define a widget instance inside of a view
- *exportidentifier*: guides the generator to generate an identifier for the element, so it can later be referenced easily from the state model (recommended e.g. view create/show/hide/destroy etc.). This has not to be filled as it will later contain a uid and in the state model dialog, the original name of the lement will become selectable
- *regionPolicy*: to be used for the statemachine. If set to unify, no extra region will be defined for corresponding state, means the state is not running in parallel. Default is 'separate', what will have the impact, that there will be a separat region in VisualState. This will make sense, if some concurrent states have to be modelled

Scope 'Controller'

In the 'controller' section the trigger and actions for the statemachine can be defined based on the definition already done in the sections described in the preceding chapters.

```
<controller>
  <message name="onChangeDummy" messageRef="ChangeDummyReqMsg" uid="">
    <map nameRef="DummyMember" valueRef="5" />
  </message>
```

Pre-Re

```
  <action name="acDoSomething" messageRef="DoSomethingReqMsg" uid="">
    <set nameRef="ViewAction" valueRef="Courier::ViewAction::Create" />
    <set nameRef="ShallLoadScene" valueRef="true" />
    <arg nameRef="ViewId" type="identifier" />
  </action>
</controller>
```

The code example shows a simple statemachine trigger, where a message *ChangeDummyReqMsg* with member value *5* is defined. This will later be visible in the VisualState trigger selection dialog. Also an action is defined where a call to *acDoSomething*(again selectable via action dialog in VisualState) will post a Courier message *DoSomethingReqMsg*. The parameters of the message will be filled with the values also defined in the 'action' section. Keyword 'set' identifies a simple assignment of the given value to the parameter, keyword 'arg' denotes a variable value, that can be assigned in the VisualState Designer tool directly.

add example for statemachine dataguard definition in contract

Scope 'InterProcessCommunication - IPC'

Courier is currently not used for interprocess communication, so therefore this section will not be explained here.

5.4. Setup applications

A skeleton generator to produce an application skeleton is not yet available

As long as there is no skeleton generator to produce a naked application, here are some manual steps to be done to setup an application initially.

When setting up a new VOB/Project some initial steps have to be done once before the work on the application can begin.

1. Create folder structure

```
+---config
|   +---packages
|   \---xml
+---products
|   \---MYPROJECT
|       \---Apps
|           \---Common
\---tools
    \---Contract
```

2. Create a folder structure like shown above. Copy content for folder *Common* from the reference applications and modify it to your needs, chapter 5.9 Configuration .
3. Create your *.prod and *.pack file in xml and add them in the corresponding file list of ai_projects, chapter 5.4.1 The 'pack and prod' xml .
4. Copy scripts GenContract*.cmd from *ai_hmi_cgi/tools/Contract* to the tools directory and adopt the parameters of the scripts, chapter 5.4.1 The 'pack and prod' xml .

Pre-Release

For a new application it is recommended to take a copy of the folder AppHmi_Skeleton and place it in the application folder of the project

<VOB-Name>/products/<Project-Name>/Apps, e.g.: *ai_nissan_hmi/products/NCG3/Apps*

Now there are some tasks to be done to have your 'naked' application compilable and linkable.

1. Renaming inside application folder
   a) Rename the folder to a meaningful name (*AppHmi_<yourname>*)
   b) Search and replace 'Playground' case sensitive with the new application name in all files and folders
   c) Search and remove the suffix *_reference-g3g* in all files
2. Adapt project header and config files, see chapter 5.4.2 Common and chapter 5.9 Configuration :
   a) Adapt the project specific header files *Project.h* and *Project_Trace.h* and enter your applications name and the corresponding trace header
   b) Assign new ids for the layers and surfaces to be used in the project base contract, located in folder *Common*
   c) Add new layer id to ScreenLayouts.xml
3. create *_pack.xml and *_prod.xml, chapter 5.4.1 The 'pack and prod' xml
4. create a download package, chapter 5.4.1 Definition of the download package

Now you have a basic setup to start with. Please find more detailed description on the steps mentioned above in the following chapters.

5.4.1. Bosch Build XML

The 'pack and prod' xml

pack.xml

The file *pack.xml basically defines a productgroup and products included, as well as the types of the product (if it will be an executable, a static or dynamic library etc.).

HMI applications have to be defined in the product group *apphmi_asf_applications*, see chapter 5.4.1 prod.xml for more details why this is the name. The code example is for reference:

```
<productgroup name="apphmi_asf_applications">
  <product name="AppHmi_MyApp"/>
    <targetdefinition resulttype="Executable" name="Gen3ArmMake" customerproject="myproject"/>
    <targetdefinition resulttype="Executable" name="Gen3X86Make" customerproject="myproject"/>
  </product>
</productgroup>
```

It is recommended that each target definition should have a customerproject to filter settings based on this later on.

In addition to the definition of the application itself, there has to be a project dependent common part where all HMI processes link against. To create such, a product *apphmi_app_prj_common* has to

Pre-Release

be defined in the productgroup *apphmi_asf_applications*. This library finally has to contain the project contract definition and other common but project specific parts.

```
<!-- entry in *_pack.xml -->
<productgroup name="apphmi_asf_applications">
  <product name="apphmi_app_prj_common" variant="apphmi_app_prj_common">
    <targetdefinition resulttype="StaticLibrary" name="Gen3ArmMake" customerproject="myproject"/>
    <targetdefinition resulttype="StaticLibrary" name="Gen3X86Make" customerproject="myproject"/>
  </product>
</productgroup>
```

prod.xml

In order to have a common way to define include paths, defines, dependencies and so on for HMI applications all over the projects, HMI-Base as well as AID uses the mechanism of so called snippets to provide a predefined set for all these. Whenever a project has to setup an HMI application it has to define its HMI application products in the productgroup *apphmi_asf_applications* which includes all the necessary snippets.

In addition to that, each HMI application has to link against a widget library and an application configuration lib. Example below show the default libraries from HMI-Base, refer also to chapter 5.4.2 Common and chapter 5.9 Configuration .

Finally the definition can look like below:

```
<productgroup name="apphmi_asf_applications" customerprojectfilter="myproject">

  <compilersettings>
    <include path="%_SWNAVIROOT%/products/MYPROJECT/Apps"/>
  </compilersettings>

  <dependencies>
    <library name="apphmi_cgi_widgets_standard-g3g_a"/>
    <library name="apphmi_app_config-myproject_a"/>
  </dependencies>

</productgroup>
```

Now the common settings are complete and a concrete product can be defined. Therefore the section of AppHmi_Playground can be copied and renamed.

```
<productdeclaration name="AppHmi_MyApp">
  <description responsible="the responsible" text="a meaningful description"/>
  <basepath path="/generated/components/asf/asf/AppHmi_MyApp/cma/src-gen"/>
  <basepath path="/generated/components/buildversion"/>
  <basepath path="/products/MYPROJECT/Apps/AppHmi_MyApp"/>
  <compilersettings>
    <include path="%_SWBUILDROOT%/generated/components/asf/asf/AppHmi_MyApp/cma/src-gen"/>
    <include path="%_SWNAVIROOT%/products/HMIBASE/Apps/AppHmi_MyApp"/>
    <include path="%_SWNAVIROOT%/products/HMIBASE/Apps/AppHmi_MyApp/HMI/CGIComponents"
        />
    <include path="%_SWNAVIROOT%/products/HMIBASE/Apps/AppHmi_MyApp/HMI/HMIContract/
        preliminary"/>
    <include path="%_SWNAVIROOT%/products/HMIBASE/Apps/AppHmi_MyApp/HMI/SM/
        VisualStateGeneratedCode"/>
```

```
    <precreaterule name="SM_TTFis.py" value="python %_SWNAVIROOT%/../ai_hmi_cgi/tools/Trace/src/
        SM_TTFis.py
                                                -d %_SWNAVIROOT
                                                    %/../ai_hmi_cgi/
                                                    products/
                                                    HMIBASE/Apps/
                                                    AppHmi_MyApp/
                                                    HMI"
    <precreaterule name="asf_gen.sh" value="%_SWNAVIROOT%/../ai_projects/tools/prj_build/asf/asf_gen.
        sh
                                                -e
                                                -v asf
                                                -a cma
                                                -n AppHmi_MyApp
                                                -m %_SWNAVIROOT
                                                    %/../ai_hmi_cgi/
                                                    products/
                                                    MYPROJECT/
                                                    Apps/
                                                    AppHmi_MyApp
                                                -s %_SWNAVIROOT
                                                    %/../ai_hmi_cgi/
                                                    components/
                                                    Framework/
                                                    Services
                                                -c %_SWNAVIROOT
                                                    %/../ai_hmi_cgi/
                                                    products/
                                                    MYPROJECT/
                                                    Apps/
                                                    AppHmi_MyApp/
                                                    App/Core/
                                                    AppHmi_MyApp.
                                                    cma"/>
    <!-- use the application id also as datapool id -->
    <define name="DP_DATAPOOL_ID==0x0815"/>
    <define name="APPHMI_MYAPP"/>
  </compilersettings>
  <dependencies>
  </dependencies>
</productdeclaration>
```

For each HMI application there have to be two precreate steps to be done, before the application can be build:

- generate the 'main' using the asf generator (asf_gen.sh)
- add trace statements to the already generated statemachine files (SM_TTFis.py)

According to the changes in *_pack.xml, there has to be a section for the application configuration also in the corresponding *_prod.xml:

```
<!-- entry in *_prod.xml -->
<productgroup name="apphmi_app_prj_common">
  <productdeclaration name="apphmi_app_common">
    <description responsible="" text="projectdependent configuration items and common code parts for all
        hmi applications"/>
```

```
    <basepath path="/products/MYPROJECT/Apps/Common" customerprojectfilter="myproject"/>
  </productdeclaration>
</productgroup>
```

For detailed information about the content of this library, see chapter 5.9 Configuration .

Definition of the download package

To have an application running on the target, there has to be a download package to be defined, including all the necessary things for such.

- the process itself
- the corresponding asset binary
- a service file for systemd to start the process

In addition to that, there are some parts that are already bundled in a package from HMI-Base

- the Screen Broker process
- shared objects of the screenbroker
- Default Screen Broker plug-ins
- Default ScreenLayouts.xml

Nevertheless the project can define their own ScreenLayouts.xml as well as their own Screen Broker plug-ins to be downloaded, see chapter 5.4.5 Screen Broker plug-ins

An example for a donwload package can look like below

```
<packages>
 <pkg>
    <Package> myproject-hmi-apps </Package>
    <Version> ${_SWNAVIROOT}/../ai_myproject_hmi/cc_label.txt </Version>
    <Maintainer> </Maintainer>
    <Architecture> armv7a,x86 </Architecture>
    <Section> Bosch </Section>
    <Priority> mandatory </Priority>
    <Depends> preparesys </Depends>
    <Source> N/A </Source>
    <Homepage> N/A </Homepage>
    <files>
      <file source="${_SWNAVIROOT}/../ai_mycustomer_hmi/products/MYPROJECT/Apps/
          AppHmi_MyApp/ProcessStartup/StartupScripts/rbcm-apphmi_myapp.service" destdir="/lib/
          systemd/system"/>
      <file source="${_SWBUILDROOT}/generated/bin/${COMPILERENV}/${MODE}/apphmi_myapp_out.
          out" destdir="/opt/bosch/base/bin"/>
      <!-- optionally copy your own plugins and screen layout description --/>
      <file source="${_SWNAVIROOT}/../ai_mycustomer_hmi/products/MYPROJECT/ScreenBrokerPlugins/
          ScreenLayouter/ScreenLayouts.xml" destdir="/opt/bosch/base"/>
      <file source="${_SWBUILDROOT}/generated/bin/${COMPILERENV}/${MODE}/
          libscreenbrokerplugins.so" dest="/opt/bosch/base/lib"/>
```

Release

```
    </files>
    <folders>
      <folder source="${_SWNAVIROOT}/../ai_mycustomer_hmi/products/MYPROJECT/Assets/2.10/" dest
          ="/opt/bosch/base/bin" filter="*Asset.bin"/>
    </folders>
  </pkg>
</packages>
```

5.4.2. Common

The following common rules for HMI application develepment have to be followed

- use HMIContract as central specification and development tool as already described in chapter 5.3 HMI-Contract as central development tool
- use interface header provided by the framework
  - gui_std_if.h
  - hall_std_if.h
  - sys_std_if.h
  - hmi_trace_if.h

  always as first include in your cpp file
- reduce includes in header files to a minimum, better use forward declaration
- use predefined product group in your prod/pack file to define your product

Tracing

Tracing in HMI applications has to be done using ETG. Wherever a trace is needed, the header *hmi_trace_if.h* has to be included, not more, not less. This header brings everything required for tracing, also including a header called *Project_Trace.h* that has to be provided by each project. The intention is that every HMI application should provide a trace interface header, defining all the trace classes for that application. These application specific trace definitions will be bundled as includes in *Project_Trace.h*.

In order to have a common setup of trace classes and ids, there is a set of classes that have to be

Pre-Release

defined by each application. (Prefix: TR_CLASSOFFSET_APPHMI_)

Trace Class	Description
<PREFIX>_MAIN	main trace class for a particular application
<PREFIX>_HALL	trace class for the HALL component
<PREFIX>_CGI_USER	clarify scope needs trace class TR_CLASS_HMI_CGI to be enabled in addition to see the traces
<PREFIX>_CGI_CANDERA	tracing Candera related issues, needs trace class TR_CLASS_HMI_CGI to be enabled in addition to see the traces
<PREFIX>_CGI_MESSAGING	tracing Courier messaging issues, needs trace class TR_CLASS_HMI_CGI to be enabled in addition to see the traces
<PREFIX>_CGI_DEBUG	tracing all CGI issues, needs trace class TR_CLASS_HMI_CGI to be enabled in addition to see the traces
<PREFIX>_SM_EVENTS	tracing state machine events
<PREFIX>_SM_ACTIONS	tracing state machine actions
<PREFIX>_SM_STATES	tracing state machine state changes
<PREFIX>_APPCTRL_PROXY	tracing the proxy of HmiAppCtrl service, TR_CLASS_HMI_FW_COMMON_APPCTRL has to be enabled in addition to see the traces
<PREFIX>_GUI	main trace class for the GUI component
<PREFIX>_WIDGET	tracing widget issues
<PREFIX>_DM	tracing the datamodel
<PREFIX>_RESERVED1 <PREFIX>_RESERVED2 <PREFIX>_RESERVED3 <PREFIX>_USER1	from here the user can implement own trace classes if required

In order to make life easier and to avoid copy/paste mistakes, HMI-Base provides a macro that is defining all the default trace ids using the offsets from above.

APPHMI_STANDARD_TRACE_SET(_hmi_base_trace_id_, _component_name_in_capital_letters)

Pre-Release

Use this define in *Project_Trace.h* to define the basic set of trace ids as well as a trace channel for your application. Please have in mind, that the value of the first parameter should never be used more than once, otherwise more than one application will trace on the same id.

Example:

```
// Project_Trace.h
#ifdef APPHMI_MASTER
APPHMI_STANDARD_TRACE_SET(MASTER, MASTER)
#include "App/AppHmi_Master_Trace.h"
#endif

#ifdef APPHMI_SKELETON
APPHMI_STANDARD_TRACE_SET(01, SKELETON)
#include "App/AppHmi_Skeleton_Trace.h"
#endif

// somewhere in your code
#ifdef VARIANT_S_FTR_ENABLE_TRC_GEN
#define ETG_DEFAULT_TRACE_CLASS TR_CLASS_APPHMI_SKELETON_HALL
#define ETG_I_TRACE_CHANNEL TR_TTFIS_APPHMI_SKELETON
#endif
```

Don't forget to adapt your trc file according to the base id you had choosen. HMI Trace will be mapped to the ids already defined in the platform *ai_osal_linux/components/bosch_includes/trace/mc_trace.h*.

Trace id for output:

- TR_COMP_UI_APP_HMI_MASTER
- TR_COMP_UI_APP_HMI_01 - TR_COMP_UI_APP_HMI_18

Trace channel for input:

- TR_TTFIS_UI_APP_HMI_MASTER
- TR_TTFIS_UI_APP_HMI_01 - TR_TTFIS_UI_APP_HMI_18

With the standard define above the following trace ids become available for the application:

```
// trace classes AppHmi_SKELETON
#define TR_CLASS_APPHMI_SKELETON_MAIN (TR_COMP_UI_APP_HMI_01+
    TR_CLASSOFFSET_APPHMI_MAIN )
#define TR_CLASS_APPHMI_SKELETON_HALL (TR_COMP_UI_APP_HMI_01+
    TR_CLASSOFFSET_APPHMI_HALL )
#define TR_CLASS_APPHMI_SKELETON_CGI_USER (TR_COMP_UI_APP_HMI_01+
    TR_CLASSOFFSET_APPHMI_CGI_USER )
#define TR_CLASS_APPHMI_SKELETON_CGI_CANDERA (TR_COMP_UI_APP_HMI_01+
    TR_CLASSOFFSET_APPHMI_CGI_CANDERA )
#define TR_CLASS_APPHMI_SKELETON_CGI_MESSAGING (TR_COMP_UI_APP_HMI_01+
    TR_CLASSOFFSET_APPHMI_CGI_MESSAGING )
#define TR_CLASS_APPHMI_SKELETON_CGI_DEBUG (TR_COMP_UI_APP_HMI_01+
    TR_CLASSOFFSET_APPHMI_CGI_DEBUG )
#define TR_CLASS_APPHMI_SKELETON_SM_EVENTS (TR_COMP_UI_APP_HMI_01+
    TR_CLASSOFFSET_APPHMI_SM_EVENTS )
#define TR_CLASS_APPHMI_SKELETON_SM_ACTIONS (TR_COMP_UI_APP_HMI_01+
    TR_CLASSOFFSET_APPHMI_SM_ACTIONS )
```

Pre-Release

```
#define TR_CLASS_APPHMI_SKELETON_SM_STATES (TR_COMP_UI_APP_HMI_01+
    TR_CLASSOFFSET_APPHMI_SM_STATES )
#define TR_CLASS_APPHMI_SKELETON_APPCTRL_PROXY (TR_COMP_UI_APP_HMI_01+
    TR_CLASSOFFSET_APPHMI_APPCTRL_PROXY )
#define TR_CLASS_APPHMI_SKELETON_GUI (TR_COMP_UI_APP_HMI_01+
    TR_CLASSOFFSET_APPHMI_GUI )
#define TR_CLASS_APPHMI_SKELETON_WIDGET (TR_COMP_UI_APP_HMI_01+
    TR_CLASSOFFSET_APPHMI_WIDGET )
#define TR_CLASS_APPHMI_SKELETON_DM (TR_COMP_UI_APP_HMI_01+
    TR_CLASSOFFSET_APPHMI_DM )

// trace input channel
#define TR_TTFIS_APPHMI_SKELETON = TR_TTFIS_UI_APP_HMI_01;
```

If there is a need to create more trace ids for the application, start from:

TR_CLASSOFFSET_APPHMI_USER

#define TR_CLASS_APPHMI_SKELETON_USER1 (TR_COMP_UI_APP_HMI_SKELETON + TR_CLASSOFFSET_APPHMI_USER +0)

5.4.3. Application

In order to set up a new application, *AppHmi_Skeleton* from ai_hmi_cgi/products/HMIBASE/Apps can be copied to your new folder. This includes a more or less naked application. Then the following steps have to be done (

application setup has to be done manually as of now :-(, scripted solution will follow

)

Example: the new app should be a tuner application

- copy *AppHmi_Skeleton* from *ai_hmi_cgi/products/HMIBASE/Apps* to *ai_mycustomer_hmi/products/MYPROJECT/Apps/AppHmi_Tuner*
- rename files **Skeleton** to **Tuner**
- case sensitiv search and replace over all files in the new folder
  - *Skeleton -> Tuner*
  - *skeleton -> tuner*
  - *SKELETON -> TUNER*
- add new application name in *Project.h*
- map free trace id and channel in *Project_Trace.h*
- change trace id and channel in AppHmi_Tuner/App/AppHmi-Tuner.trc according to the newly mapped id/channel
- map free layer and surface ids in Apps/Common/ProjectBaseContract/config/ProjectBase-Types.xhcdl

Pre-Release

- generate contract, statemachine project and statemachine code using the scripts in *_scripts*

ASF Configuration

HMI applications have to be setup as ASF applications. Therefore there are the files *.cmc to define the components *GUI* and *HALL* and one *.cma file to define the application. Detailed information about ASF and application configuration can be found in the ASF User Guide.

The HMI framework requires some common interfaces for all applications (powermanagement, HmiAppCtrl, CourierTunnelService, AudioManagment for AppHmi_Master). As of now (ASF 1.8) there is no possibility of inheritance for these configuration files. Therefore do not remove things from the template, only extend them to your needs (mainly add new ports to the HALL component and import middleware interfaces in the cma file). Please get in contact with HMI-Base team before making changes in the GUI component.

File structure of an application

The picture below shows the basic folder structure of an application and the listing give a small abstract on the most important content:

```
AppHmi_Skeleton
|   AppHmi_Skeleton.xml
|
+---GUI
|   |   AppHmi_Skeleton.scs
|   |
|   \---Solution
|
+---ProcessStartup
|   \---StartupScripts
|           rbcm-apphmi_skeleton.service
|
+---App
|   |   AppHmi_Skeleton_Trace.h
|   |   AppHmi_Skeleton_Trace.trc
|   |
|   +---Core
|   |       SkeletonHall.cmc
|   |       SkeletonGui.cmc
|   |       AppHmi_Skeleton.cma
|   |       SkeletonGui.cpp
|   |       SkeletonGui.h
|   |       SkeletonHall.cpp
|   |       SkeletonHall.h
|   |
|   \---datapool
|           apphmi_skeleton_DpPersistentValues.xml
|           apphmi_skeleton_datapool_if.h
|
+---_scripts
|       SceneComposer_Start.cmd
|       SceneComposer_GenerateAssets.cmd
|       SceneComposer_StartSolution.cmd
|       GenContract.cmd
|       GenContract2VS.cmd
|
\---HMI
    +---HMIContract
    |   +---preliminary
```

```
|   |
|   \---config
|           AppHmi_SkeletonDatabinding.xhcdl
|           AppHmi_SkeletonWidgets.xhcdl
|           AppHmi_SkeletonStateMachine.xhcdl
|           AppHmi_SkeletonMessages.xhcdl
|           AppHmi_SkeletonConstants.xhcdl
|           AppHmi_SkeletonViews.xhcdl
|
+---DataModel
|       HMIModelComponent.h
|       HMIModelComponent.cpp
|
+---SM
|   +---VisualStateGeneratedCode
|   |
|   \---config
|           AppHmi_SkeletonStateMachine.vdi
|           AppHmi_SkeletonModule.vsr
|           AppHmi_SkeletonStateMachine.vtg
|           AppHmi_SkeletonStateMachine.vnw
|           AppHmi_SkeletonStateMachine.vsp
|           AppHmi_SkeletonStateMachine.stereotypes
|
\---CGIComponents
        CGIAppViewController_Skeleton.cpp
        CGIAppViewController_Skeleton.h
        CGIAppController.h
        CGIAppController.cpp
        AppViewSettings.cpp
        AppViewSettings.h
        CGIApp.cpp
        CGIApp.h
        CGIAppPopupTable.cpp
        CGIAppViewFactory.cpp
        CGIAppViewFactory.h
        CGIAppDefines.h
```

- _scripts/: different scripts for contract and statemachine generation/synchronization, SceneComposer solution start and asset generation
- App
  - *Trace.h: Mapping of the generic trace ids provided by the framework to application specific names
  - *Trace.trc: the one and only trc file for the application, including branch rules only (rest is done by ETG trace generator)
- App/Core/: the application core definition
  - *.cma -> the application definition for ASF
  - *.cmc -> the component definitions for ASF
  - *Gui -> the GUI component implementation
  - *Hall -> the HALL component implementation
- App/Datamodel/: the datamodel of the application
- App/datapool/: contains datapool definition for persistent data storage (optional)
- HMI/HMIContract/config/: contains the HMIContract description for the application

Pre

- GUI: contains the Gui model (SceneComposer solution)
- HMI/HMIContract/preliminary/: conatin the HMIContract generated files. Do not change the content manually, as it will be overwritten next time the generator runs.
- HMI/SM/config/: contains the VisualState statemachine model
  - *.vwn -> the 'Navigator' file, to be opened to generate statemachine from VisualState GUI
  - *.vsp -> the 'Designer' file, this is the main project file
  - *.vsr -> each module in VisualState will have a .vsr file, which might have a limited scope. The .vsp is refering to (all) .vsr files.
- HMI/SM/VisualStateGeneratedCode/: contains the generated code of the statemodel. Do not change the content manually, as it will be overwritten next time the generator runs.
- HMI/CGIComponents/: contain files for manual adaptation as model extention as well as the view factory and the popup configuration
  - AppViewSettings -> Provides popup presentation arguments and request ids to show/hide a main view or popup. This information is used by the HMI interface to Screen Broker client to communicate with Screen Broker. If no project specific implementation exists, the implementation of the base class from the framework is used.
  - CGIApp -> instanciation of the main GUI components, requires the statemachine name only, rest is done in base class provided by the framework
  - CGIAppController -> place for extentions of the controller component, so each event not consumed by the controller model will be received here and can be handled by handcrafted code
  - CGIAppViewController_Skeleton -> has to include a class definition/declaration with at least the scene name and path (to locate the scene in the asset). This class can also be extended to handle messages in addition to the widgets (view component)
  - CGIAppPopupTable -> whenever a popup is required, place its configuration (priority, presentation time, etc.) inside the makros in this file
  - CGIAppViewFactory -> the view factory should include a list of all view controller classes already defined in CGIAppViewController_Skeleton
- ProcessStartup: conatins the startup script to be used by systemd

5.4.4. AppHmi_Master

The AppHmi_Master is from its structure basically identical to all other HMI applications. But it has some extended responsibilities and therefore additional interfaces.

From the component diagram the additional required and provided interfaces in comparison to a standard application can be seen. There is the additional interface usage of GENIVIs audiomanager 'CommandInterface' as well as the provided interfaces for AudioSourceChange and SoundSettings (to be used by other HMI applications, chapter 4.3 Audio Client). Also AppHmi_Master provides the service for HmiAppCtrl (and uses it as well).

Pre-Release

Extend HmiAppCtrl functionality within AppHmi_Master

The framework provides some basic functionality for HmiAppCtrl. Standard applications require only the proxy functionality and there normally no extention is needed as the base implementation provided by the framework covers the application needs. For AppHmi_Master the situation is slightly different, as it might have to extend the proxy (this will be explained with an example in chapter 5.5 Status Bar) and has to instantiate and extend the stub.

The Stub extension is mainly used to add names of hmi applications under control of HmiAppCtrl service. Therefore a task array has to be filled in the constructor of StubHandler_Prj with the names of the applications of the project:

```
StubHandler_AppExt::StubHandler_AppExt()
{
    // these applications are expected to be started
    _taskArray.addTask(hmi::apps::appHmi_Master, AppHmi_Master, true);
    _taskArray.addTask(hmi::apps::appHmi_Tuner, AppHmi_Tuner, true);
    // playground is not mandatory to start, it is not part of procconf.xml by default (anyway can be started
        manually via 'systemctl start rbcm-apphmi_playground')
    _taskArray.addTask(hmi::apps::reference::appHmi_Playground, AppHmi_Playground, false);
}
```

Additionally some other functionality can be extended, e.g. the key forwarding which is originally maintained by the framework, but sometimes there might be a need to forward a key received by application X, not processed there and so forwarded automatically to AppHmi_Master, to another application Y.

Activate and extend AudioSourceCtrl functionality within AppHmi_Master

By default there is a basic audio functionality provided by the framework. In order to extend the audio control functionality to the project specific needs, you can create project specific implementation of the connection, sound and/or resource controller class using inheritance. In order to initialize the audio related software parts (with the project specific or the default implementations or maybe also a mixture of both), you simply have to use the following macro and but the class in that you require it:

```
MasterHall::MasterHall()
{
    // do something

    // initialize the audio related controller with default and/or project specific implementations
    APPHMI_MASTER_INIT_AUDIO_COMPONENTS(ConnectionControllerPrj, SoundControllerPrj,
        ResourceController)
}
```

Responsibility of the statemachine

In addition to the normal responsibility of an application statemachine (create/destroy/show/hide main scenes and popups), the statemachine of AppHmi_Master has to handle the application switch logic. Whenever an application has to be switched on (has to come to foreground), the statemachine has to take the decision and finally has to trigger an action to send a signal via HmiAppCtrl service to the corresponding application to activate its main surface (this is done by the ProxyHandler base

implementation). Due to the fact that AppHmi_Master is the audio master for HMI applications also the audio source switch logic (strongly project specific) can be modelled here (or manually coded in the controller classes, see chapter 5.4.4 Activate and extend AudioSourceCtrl functionality within AppHmi_Master)

Nevertheless the statemachine of AppHmi_Master can interact like any other statemachine, there are no restrictions compared to the others. And therefore it has to handle functionality, which has to be available in the system, independent from the current situation on screen (mainly related to functionality that is triggered by some key input, e.g. changing volume via encoder and showing the corresponding popup). Also general functionality e.g. a clock screen, when system is paused via On-Tipper can be placed in AppHmi_Master.

Please have in mind, that popups are a little bit special and can be shown all the time, independent from the current application on screen. So there is no need to implement every global popup in AppHmi_Master, but in the application, where they belong to (related to their domain), see chapter 5.6 Popups .

5.4.5. Screen Broker plug-ins

The project can use the SB plug-ins delivered by HMI-Base. They are described in chapter 2.6.2 Plug-ins . If the project needs features not covered in the plugins, it can create own plug-ins (e.g. based on the HMI-Base SB plug-ins). The project specific plug-ins have to be placed in this folder: folder for project specific SB plug-ins(8) .

Furthermore the project can create its own *ScreenLayouts.xml* file. This file contains all layers and surfaces the project wants to use in its system. If the project wants to do so, it can copy this file from the HMI-Base plug-ins: HMI-Base ScreenLayouts.xml(9) . The project specific *ScreenLayouts.xml* has to be stored here: project ScreenLayouts.xml(10) . If the HMI-Base plug-ins are used, the project just has to create the necessary empty folders (”ScreenBrokerPlugins” and ”ScreenLayouter”) to place its *ScreenLayouts.xml* in its own project folder. If the project later decides to create its own SB plug-ins it will store it in the same location.

As mentioned in chapter 5.4.1 Definition of the download package the location of the *ScreenLayouts.xml* has to be given in the download package. The download package file has to be placed here: package file(11) . If own project SB plug-ins are used, they also have to be added to the download package file.

5.4.6. Guidelines

HALL development

- main class *Hall should not implement abstract interface methods from asf directly, create handler classes instead
- split up the handling of asf callbacks into small parts that belong together, use separated classes

GUI application development

Add content to How To/ Setup/ Guidelines/ Gui application development

Pre-release

2207 **Development with Scene Composer**

2208 Add content to How To/ Setup/ Guidelines/ Development with Scene Composer

2209 5.5. Status Bar

2210 A status bar is an area in which different applications can show information in (phone signal strength,
2211 clock time, temperature, etc.). With the approach that every HMI-Application runs in its own process,
2212 there are different ways how to implement a status bar:

2213 - Status bar in one HMI-App, layer is bottom most
2214 - Status bar in one HMI-App, layer is top most
2215 - Status bar as part of every HMI-App

2216 The status bar surface must be a permanent surface (in terms of Screen Broker) otherwise it gets
2217 invisible on an application switch (since only one main view surface can be visible at a time (see also
2218 chapter 2.6.5 Permanent surfaces).

2219 5.5.1. Status Bar in one HMI-App (layer bottom most)

2220 The status bar surface is implemented only once in one HMI-App. This application holds the status
2221 bar surface on which the status bar is rendered. This surface is added to an own status bar layer. This
2222 layer is put in the layer render order in the very back, so that it is behind all other HMI views. At
2223 start up of this application the status bar surface is activated via Screen Broker (with sending Show())
2224 for the status bar surface.

2225 The views of all HMI-Apps in which the status bar should be visible must be transparent at the area
2226 of the status bar. If this is the case the status bar from the bottom most layer is visible through the
2227 HMI view. Views in which the status bar should not be visible (e.g. list views) are opaque at the area
2228 of the status bar, so the status bar is not seen there.

2229 5.5.2. Status Bar in one HMI-App (layer top most)

2230 This approach is similar to chapter 5.5.1 Status Bar in one HMI-App (layer bottom most) except the
2231 layer render order and the surface activation. The status bar layer is put in the layer render order to
2232 the very top, so that it is in front of all HMI main views. Normally you would want that popups are
2233 not covered by status bar, so you would put the status bar layer in the render order between the main
2234 view layer and the popup layer.

2235 If an HMI-App switches to a view that requires the status bar, it has to send a request for showing
2236 the status bar to the HMI-Master. The HMI-Master evaluates if passing the request is ok (only the
2237 current active main application is allowed to enable/disable the status bar). If the evaluation fails,
2238 the HMI-Master has to remember this request for that HMI-App as pending and has to send it out
2239 at that point of time when the requesting application becomes active. If the evaluation succeeds the
2240 HMI-Master sends the request to the HMI-App to which the status bar surface belongs to. Once this
2241 HMI-App receives the request, ii activates the status bar surface with sending Show() to Screen Broker.
2242 This sequence is shown in Figure 5.6 .

Pre-Release

In case an HMI-App switches to a view that does not require the status bar, it has to send a request for removing the status bar to the HMI-Master. The sequence is the same as described in Figure 5.6

except for the last call, there Hide() is called instead of Show().

5.5.3. Status Bar in every HMI-App

In this approach the status bar is implemented in every HMI-App, so the application itself has the control to show or hide the status bar. Every application holds an own status bar surface. But since only one HMI-App is visible at a time all status bar surfaces can be added to the same status bar layer. Regarding the layer render order of the status bar layer you can decide to use the render order from chapter 5.5.1 Status Bar in one HMI-App (layer bottom most)

or chapter 5.5.2 Status Bar in one HMI-App (layer top most) . The HMI-App itself has to take care to activate the status bar surface when it has to be shown and to deactivate it in all other cases (e.g. when the App is left).

required extention of ProxyHandler class to be described here

5.6. Popups

In this section the popup handling that is implemented in the HMI-Base Screen Broker plug-ins is described (chapter 2.6.2 Plug-ins).

5.6.1. Surface and Layer IDs

Every application can have several popups. A popup can be an *application* or a *system* popup, this has an impact on the visibility of the popup. *Application* popups are only visible if the owning application is also visible. *System* popups are visible on the entire system. For more information please refer to chapter 2.6.7 Popup features .

For popups every application needs two ILM surfaces, one surface for *application* and one surface for *system* popups. These popup surfaces can be assigned to different layers (one for *application*, one for *system* popups, dependent of your layer setup). It is recommended to use one layer for all *application* popup surfaces (of all applications) and one layer for all *system* popup surfaces (of all applications). The surface and layer ids have to be unique across the system, please refer to chapter 2.6 Screen Broker .

The surface and layer ids need to be added in these places:

- Render Targets in Scene Composer
- ScreenLayouts.xml
- ProjectBaseTypes.xhcdl

The corresponding render targets for *application* and *system* popups have to be created in Scene Composer (with the surface and layer ID). Furthermore the surface and layer IDs have to be added to *ScreenLayouts.xml* (see chapter 2.6.2 Screen Layouter , chapter 5.4 Setup applications), which has to

Pre-Release

be stored here: project ScreenLayouts.xml(10) . Finally the surface and layer ids have to be added to *ProjectBaseTypes.xhcdl* which has to be stored here: ProjectBaseTypes.xhcdl(10) .

5.6.2. HMI-Contract

In addition to the normal procedure to describe a new scene some information has to be added to *AppHmi_<AppName>Constants.xhcdl*. For every popup add the below shown line, which holds the name and the id (*value*) of the popup: <const name="PID_VOLUME_MAIN"type="uint8"uid=""value="1"/>

5.6.3. Popup Table

Every application has a popup table that defines the popup properties for all popups of this application (chapter 2.6.7 Popup features). This file is called *CGIAppPopupTable.cpp* and is placed in the folder *"HMI/CGIComponents"* of every application.

Please find here an example from HMI-Base playground application:

```
POPUP_TABLE_BEGIN() // popup ID, modality, priority, presentation time, min. presentation time, validity
    period, close on superseded, close on app leave, path in asset
  POPUP_TABLE_ENTRY(PID_PU_APPL_1, ScreenBroker::Modality::Application, 100, 0, 0, 0, false, false, "M2
      #PUScenes#PopupScene_Appl_1"),
  POPUP_TABLE_ENTRY(PID_PU_APPL_2, ScreenBroker::Modality::Application, 100, 0, 0, 0, false, false, "M2
      #PUScenes#PopupScene_Appl_2"),
  POPUP_TABLE_ENTRY(PID_PU_GLOB_1, ScreenBroker::Modality::System, 10, 0, 0, 0, false, false, "M2#
      PUScenes#PopupScene_Glob_1"),
  POPUP_TABLE_ENTRY(PID_PU_GLOB_2, ScreenBroker::Modality::System, 100, 0, 0, 0, false, false, "M2#
      PUScenes#PopupScene_Glob_2"),
POPUP_TABLE_END()
```

5.6.4. State Machine

To use the popup handling of the HMI-Base SB plug-ins in an HMI-App some requirements regarding modelling of state machine for the HMI-App must be fullfilled. Please find the state design in Figure 5.7 .

A popup can be in one of these states:

- Invisible
- Visible
- Queued

Once the state *Popup* is reached the corresponding view has to be created. Once the state is left, the view has to be destroyed. Initially a popup is in state *Invisible*. If the Screen Broker gets the request to show this popup (via gDisplayPopup), it answers it with this message OnSurfaceStateMsg. The parameter can be:

- Invisible
- Visible

Pre-Release

- Registered (needs not to be handled in popups)
- Unqueued
- Queued

Dependent on this message the popup changes to state *Visible*, *Queued* or *Invisible*. On entering the state *Visible* gViewShow has to be called for the corresponding view to show it on the screen. On leaving this state, gViewHide and gViewClear has to be called for the corresponding view.

To hide a popup gHidePopup has to be called for the corresponding popup. The Screen Broker answers again with the message OnSurfaceStateMsg.

Additional information can be found here: How To Popups wiki. As reference the popup implementation of the HMI-Base playground application can be taken.

5.7. Animations

In general we have to distinguish betwenn different types of animations as there are

1. Widget animations
2. Screen transition animations
3. Application change animations (Screen Broker animations)

The following sections will give some deeper knowledge on when and how to use these animation types

5.7.1. Widget Animations

5.7.2. Screen Transition Animations

Screen transition animations can be used/configured to change between different scenes within a application. Currently the following animation types are supported

- Fade
- ZoomIn/Out
- Prev/NextLevel
- PageFlipFront/Back
- DualViewLeft/Right

To have these animations working the following configuration work has to be done for all types of animations. These settings are sufficient for the animations 'PageFlip', 'Zoom' and 'Level'. 'Fade' and 'DualView' need some additional configuration effort in the scen

Pre-Release

Configuration work in SceneComposer

Common

Add a TransitionMarkerWidget to the WidgetTree, *do not change its name* and assign the scene camera to the widget

Additionally the Shader program *Global#ShaderPrograms#RefTrans_RefTex* has to be part of the asset.

Additional configuration for Fade Animation

To realize a Fade animation the following actions have to be performed in the SceneComposer

Actions in both involved scenes (the one to be fade out and the one to be fade in)

- add another TransitionMarkerWidget to the WidgetTree
- rename it to *FadeMarkerWidget* and assign the node to be faded to it. Use a groupnode if more than one node needs to be animated

Additional action to be done in the configuration of the scene to be entered

- add another TransitionMarkerWidget to the WidgetTree
- rename it to *BackGroundMarkerWidget*
- assign the node containing the background image (the content that should not be faded)

Additional configuration for DualView Animation

The following steps have to be done for all views to be involved in the animation. Mean to say the scene to be left has to have these settings as well as the scene to be entered.

- add an additional camera to the SceneTree
- assign a proper viewport and position
- add the camera to the same render target as the primary camera
- assign value 'Scope3' to the ScopeMask property of the second camera
- assign a different scope mask value (not the default!) 'to the first camera and all nodes
- add two more TransitionMarkerWidget to the WidgetTree
- rename them to *DualViewTransitionMarkerWidgetForNode* and *DualViewTransitionMarkerWidgetForCamera*
- assign the node to be animated and the second camera to these widgets

Pre-Release

State Model

The current animation approach contradicts a little with the configuration for states that will not be animated.

Another approach is under investigation, where the animation trigger is part of the transition and also not animated transition will be executed by the animation framework. This will lead to the fact that an animation can easily be applied/removed without changing the state content, only the trigger has to be removed/added from/to the transition

SceneFactory

The scene map has to contain two dummy scenes to be used for animation.

```
SCENE_MAPPING_BEGIN(aScenes)
// width height cameradistance
SCENE_MAPPING_ENTRY_TRANSITION( 400, 240, 200.0f), // for any scene transition animation
SCENE_MAPPING_ENTRY_PAGEFLIP( 400, 240, 200.0f), // for pageflip scene transition animation
// add your scenes as usual below
SCENE_MAPPING_END()
```

The parameters width and height are usually the display resolution. The camera distance can be set to any value (check differnt value to find the best for your use case).

5.7.3. Application Change / Screen Broker Animations

sat2hi/gs77hi: add howto for SBAnimations when screenbroker version is integrated, that solved Mantis 4496

5.8. SCHost and Widget Reporitory

5.9. Configuration

A project has to have somehow a kind of configuration or common software parts.

Mandatory configurations are done in the files *Project_Trace.h*, see chapter 5.4.2 Tracing , and *Project.h*. This file includes only a particular name for each application (used to identify the application in different DBus messages, as well as for tracing purpose) and the display properties like resolution and connection.

```
namespace hmi
{
    namespace apps
    {
        // AppHmi_Master is predefined in the framework, see app_base_defines.h
        //static const char* appHmi_Master = "AppHmi_Master";
        static const char* appHmi_Tuner = "AppHmi_Tuner";
        static const char* appHmi_Skeleton = "AppHmi_Skeleton";
```

Pre-Release

```
    }

    namespace main_display_config
    {
        static const int id = 3; // 0 – HDMI, 1 – LVDS0, 2 – LVDS1, 3 – Parallel
        static const int width = 800;
        static const int height = 480;
    }
}
```

Some more configuration/common stuff can be done in the folder *MYPROJECT/Apps/Common*. This is optional and only examplary described in the following chapters (what can be done there for example).

5.9.1. KeyMapping

In the key mapping class it is defined which key or encoder send by the input device driver (via Wayland) is mapped to which key event or encoder event in the HMI. The project specific key mapping class has to be derived from KeyMappingBase class which is part of the HMI-FW. It has to implement the methods defined by the base class. As example the key mapping implementation of HMI-Base can be taken: HMI-Base KeyMapping.cpp(7)

Furthremore a mapping mechanism has to be implemented, e.g. the same mechanism used in HMI-Base KeyMapping can be taken (see here: HMI-Base KeyMapping.cpp(7)). There at startup std::maps for keys and encoders are filled.

Here is an example of the mapping mechanism:

```
_mKeyMap[KEY_FAC_RADIO] = HardKeyCode_HK_FM_AM;
_mKeyMap[KEY_FAC_MEDIA] = HardKeyCode_HK_MEDIA;
_mKeyMap[KEY_FAC_CAR] = HardKeyCode_HK_CAR_MENU;
_mKeyMap[KEY_FAC_SETUP] = HardKeyCode_HK_MENU;
_mKeyMap[KEY_FAC_PHONE] = HardKeyCode_HK_PHONE;
...
_mEncoderMap[REL_HWHEEL] = EncCode_RIGHT_ENCODER;
_mEncoderMap[REL_WHEEL] = EncCode_LEFT_ENCODER;
```

5.9.2. Project specific base controller class

Sometimes it is necessary to have a common implementation/handling of messages in the controller. This can easily be done by using a Cpp Implementation, which will be executed, when a particular message is not processed by the controller model. Therefor you simply have to extend **class** CGIAppControllerProject in *MYPROJECT/Apps/Common/CgiAppController*. Use the already known macros to extend the message map of that class or write your own message handler, instanciate the same in CGIAppControllerProject and delegate messages to the object.

```
class CGIAppControllerProject : public CGIAppControllerBase
{
public:
    COURIER_MSG_MAP_BEGIN()
        COURIER_MSG_DELEGATE_TO_REF(myHandlerRef)
        COURIER_MSG_DELEGATE_TO_OBJ(myHandlerObj)
    COURIER_MSG_MAP_END_DELEGATE(CGIAppControllerBase)
```

Pre-release

```
private:
    myHandlerClazz1 myHandlerRef;
    myHandlerClazz2* myHandlerObj;
};
```

5.10. Setup Automation

In chapter 3.4 General Automation the general setup and availability of scripts is explained. When a new application is set up, the content of '_scripts' folder has to be adapted (when previously copied from a reference application) to the new name of the application.

5.10.1. The 'tools' directory

The folder content of the '_scripts' directory of an application is shown in chapter 2.4.3 Folder structure . So in there are several scripts that contain application specific names. These names have to be adapted to the correct one. In order to use the generic scripts of ai_hmi_cgi/tools, it is recommended to create a tools/HMIContract folder in the VOB the application is located in. There some small wrapper scripts have to be placed to call the generic scripts from HMI-Base with parameters, that are common for application in this VOB.

Finally there should be two wrapper sripts in <VOB-Name>/tools/HMIContract

- GenContract.cmd/GenContract2VS.cmd -> calling directly GenContract.cmd/GenContract2VS.cmd adding the name of the VOB and the project name as parameter

Provide script/archive to create such wrapper scripts

When this has been set up, the complete code generation shall be started only by the scripts.

Do never use the executables of HMIContract and HMIContract2VS instead!!!

Pre-Release

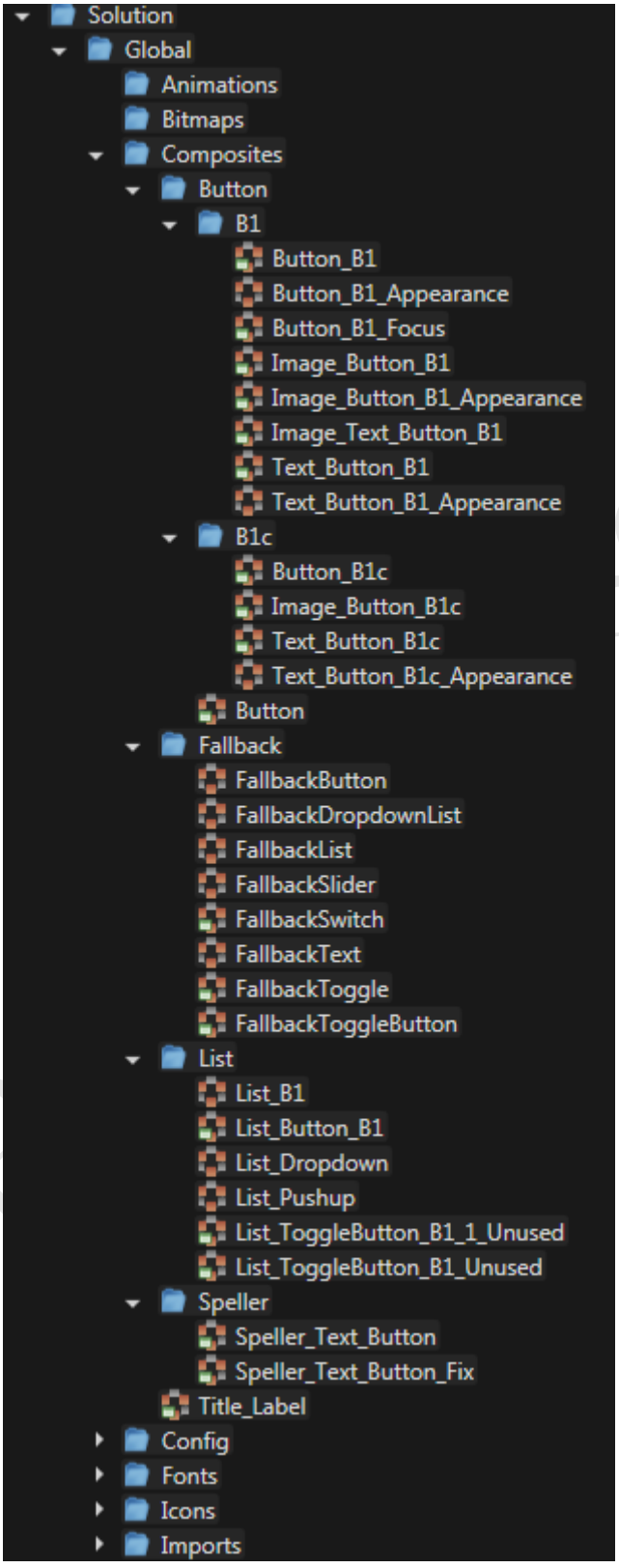


Figure 5.2.: Composite naming convention

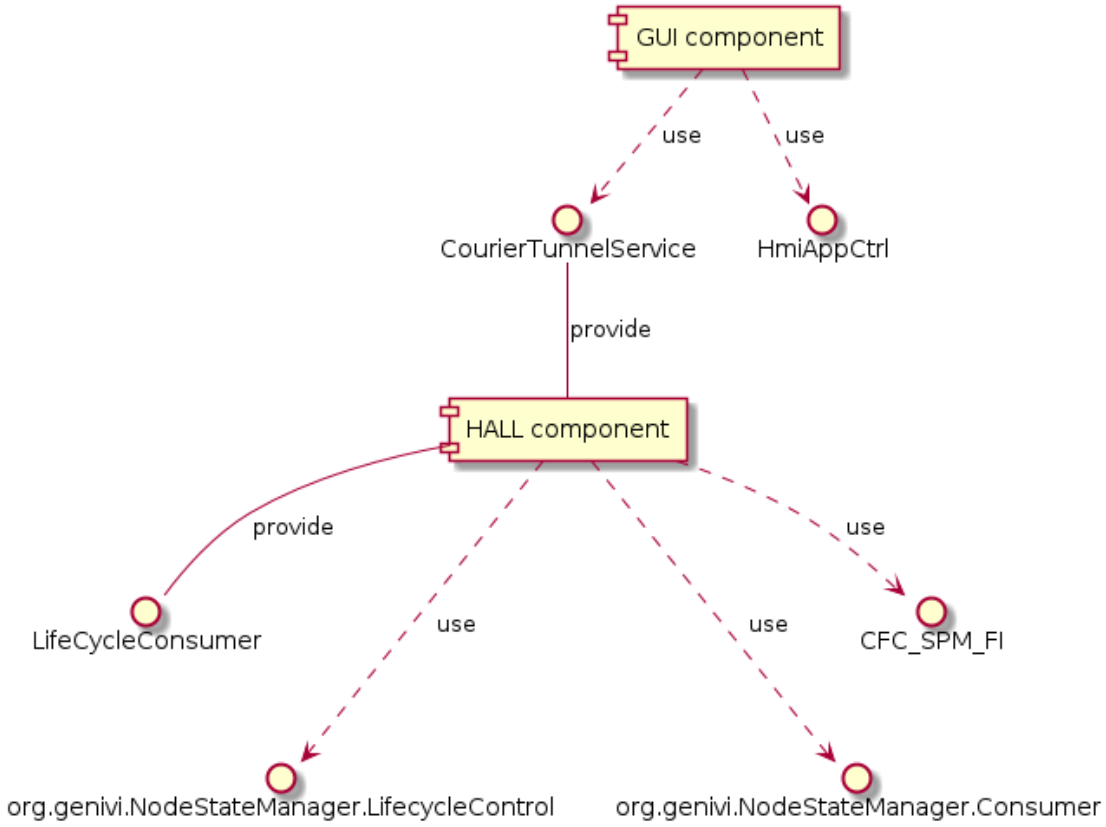


Figure 5.3.: Component diagram of a standard HMI application

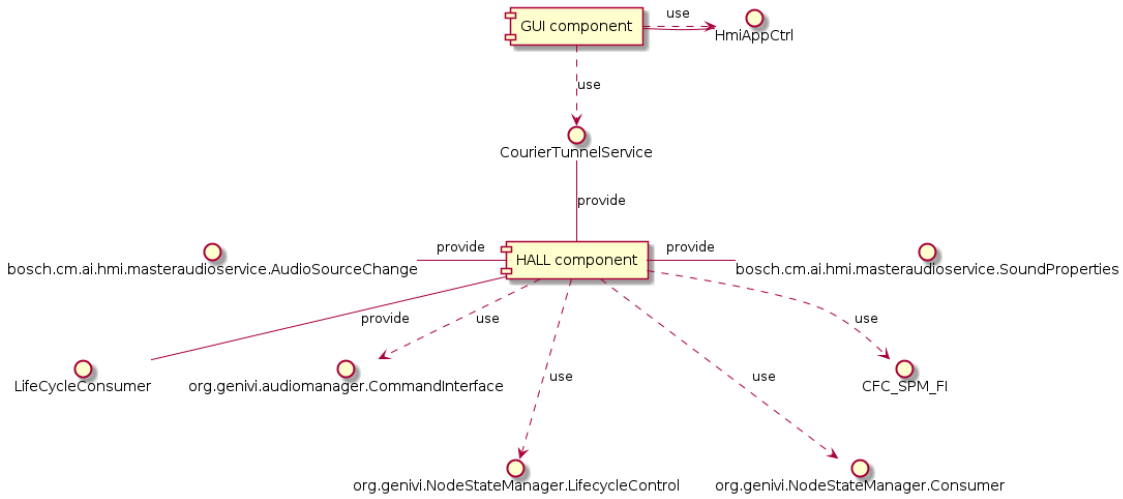


Figure 5.4.: Component diagram of a master HMI application

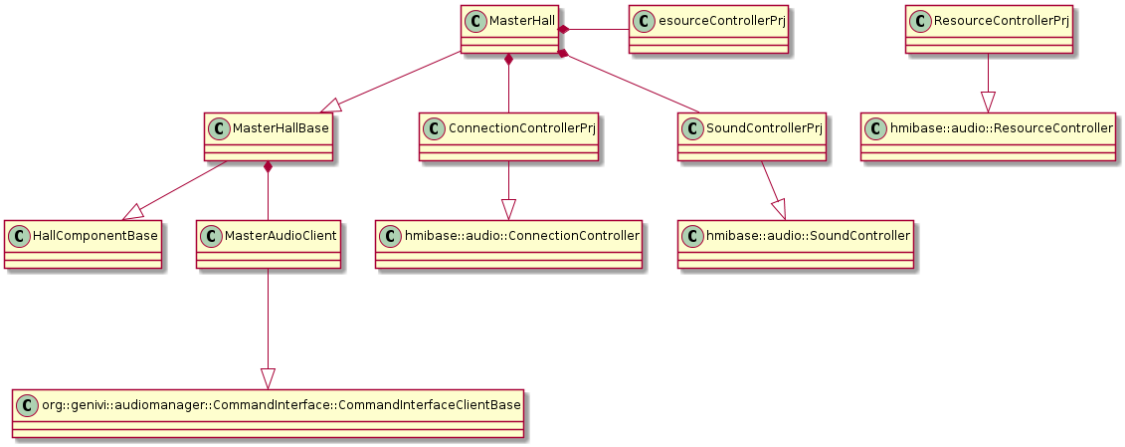


Figure 5.5.: Simplified class diagram of master HALL component

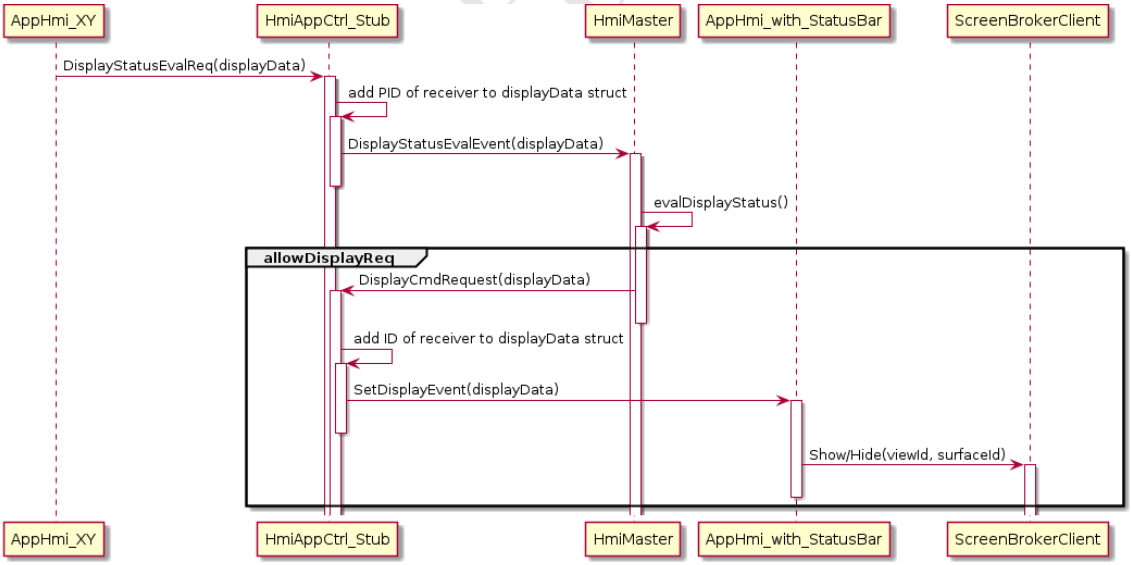


Figure 5.6.: Sequence diagram of de-/activating the status bar

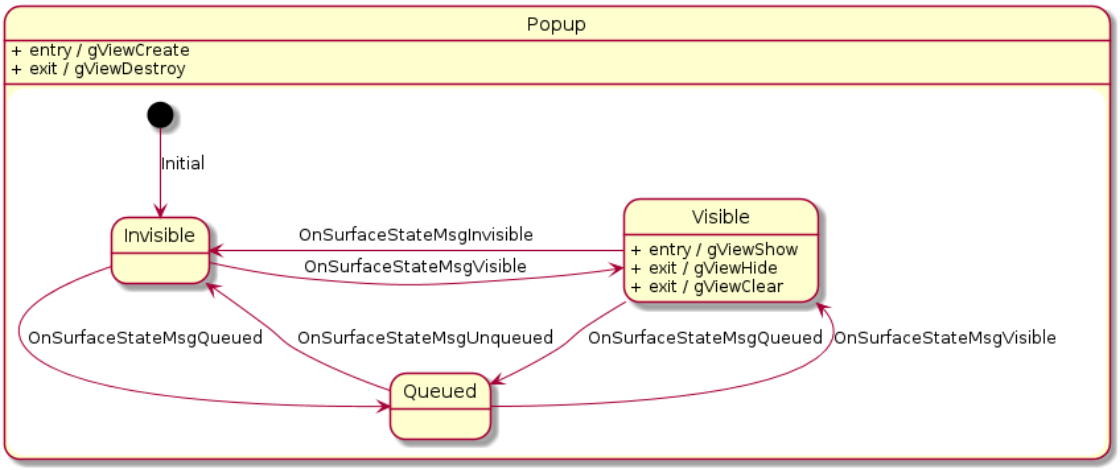


Figure 5.7.: Popup state machine

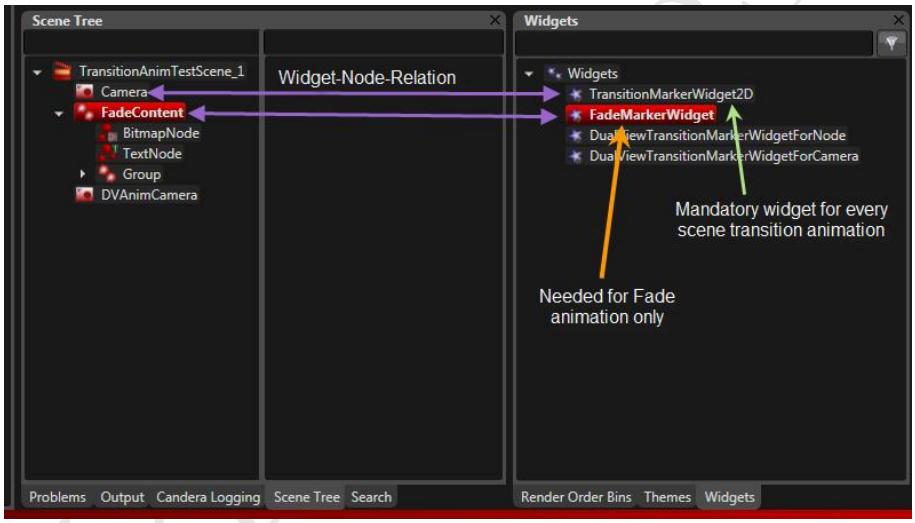


Figure 5.8.: Fade animation configuration in SceneComposer

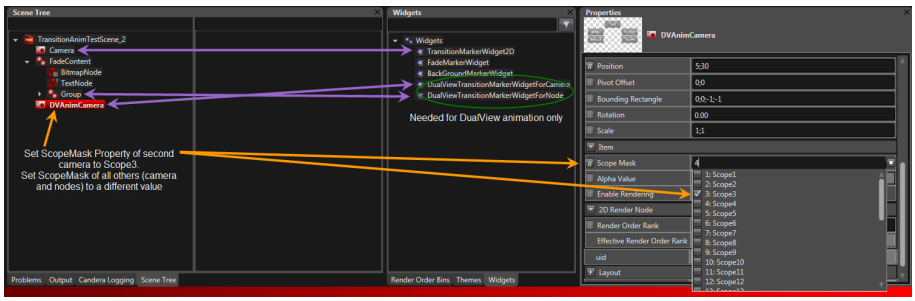


Figure 5.9.: DualView animation configuration in SceneComposer

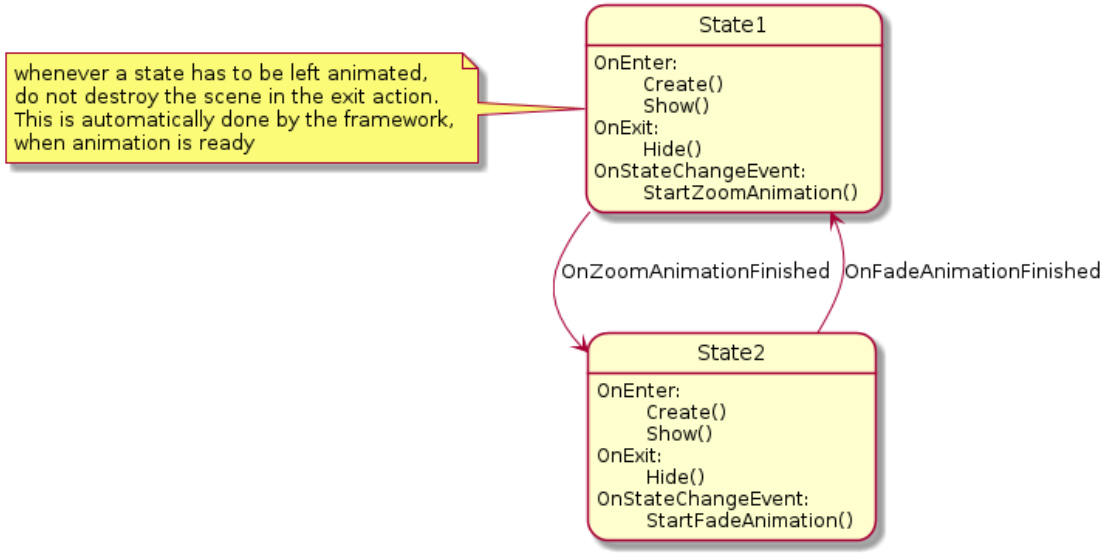


Figure 5.10.: state model for transition animation

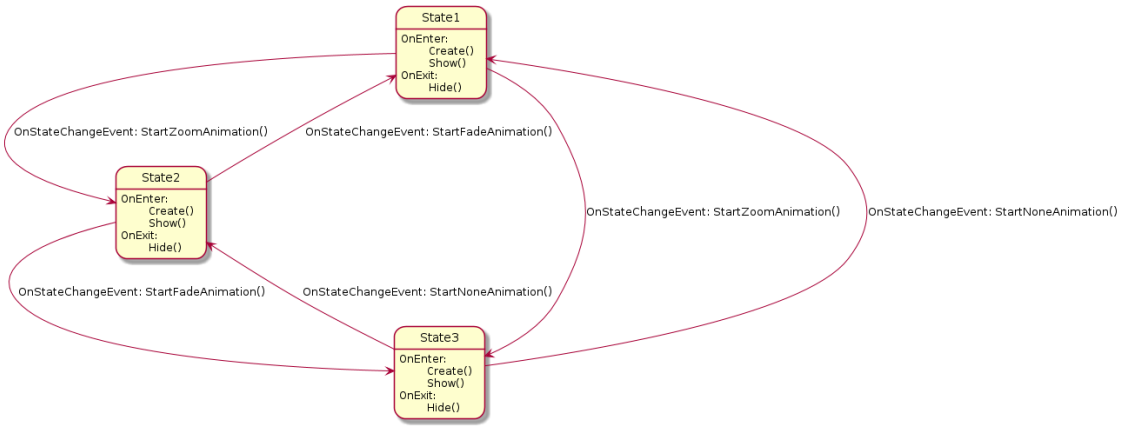


Figure 5.11.: UNDER INVESTIGATION: state model for transition animation

6. Reference Applications

6.1. Overview

HMI-Base provides a couple of reference applications to show basic principles and use cases. These applications can be found in ai_hmi_cgi/products/HMIBASE/Apps. The applications can run on every target as well as LSim. To download the required processes, assets, service files and so on, the package *pkg_hmi_reference_apps* has to be referenced in the package list in *ai_project/config/packages* instead of the original one from the project (if there is any).

Afterwards the applications can be started via systemd (manually or by editing registry).

6.2. Applications

6.2.1. AppHmi_Master

The reference implementation of apphmi_master supports the following features

- Application switching between the other two reference apps
- Setup of audio source control service
- Setup of HMI application control service
- Capability for UI also as master application

6.2.2. AppHmi_Tuner

The reference application for tuner should be used as example for proper HALL/Datamodel cooperation and GUI modeling using composites.

Main features to have a look on

- Audio source switch (band switch request)
- Proper list handling (GUI modeling and filling Datamodel)
- Hardkey forwarding to master application

Pre-Release

6.2.3. AppHmi_Playground

The playground application should basically show proper popup modeling and gives examples for all kinds of scene transition animations. It will be extended.

- Popup configuration using CGIAppPopupTable.cpp
- Popup activation/deactivation
- Popup as independent state to run in parallel (e.g. for popup stacking)
- CloseOnTouch feature
- Hardkey forwarding to master application
- Scene transition animations

Pre-Release

Part IV.

WidgetRepository

Pre-Release

7. HMI-Base default Widgetset

In this chapter you will find the documentation of the HMI-Base Widget Repository, which is provided as a free configurable library and works together with the CGI Studio Scenecomposer via SCHOST.DLL plugin.

7.1. Introduction

A graphical control element or widget is an element of interaction in a graphical user interface (GUI), such as a button, scroll bar or simple a textout. Widgets are software components of the GUI that allows a user to interact with the system (like loundness control) through direct manipulation of system values. Others act as containers that group the widgets added to them, for example panels, controlbars and tabs.

The Bosch HMI Widget Repository library contain a collection of 2D graphical control elements and the logic to render these.

The CGI-Studio graphical user interface in combination with HMI-Contract automatically generates all the data structures and source code for the graphical control element from property configuration provided by the GUI developer.

A widget can be used either in a 2D scene or in a 3D scene. Accordingly, at the top level the CGI Studio Widget Library is structured in two folders, Widget2D and Widget3D.

In following section you will find a detailded description of all Widgets designed in 2D.

7.2. Animation Widget 2D

7.2.1. Introduction

AnimationWidget2D extends functionality of AnimationWidgetBase class, which in turn extends from BaseWidget2D. This widget is used to perform some Animations on a Node. The advantage of using this widget to perform animation is that, the node associated to the widget can be changed at run time.

It has the following Animations which can be performed on a Node: Features: Scaling, fading, moving, rotation, trigger by event or automatically, duration, speed, start and stop, node animation, animation direction, interpolation strategy.

Each of these features makes use of one or multiple property in order to achieve their goal. Other properties have to be supplied to the widget in order to perform the widget operation.

Pre-Release

Examples Use Cases

Scaling the Node. Also use this configuration for the Wheel widget Animation.



Figure 7.1.: Wheel animation

As shown in the above diagram, the round bitmap will have to scale from 0-1 and 1-0, based on some triggers. The following changes have to be done to achieve the above scaling animation.

- The Node which has to be scaled, has to be associated to the AnimationWidget2D.
- Set the Animation Type to ScaleAnimation.
- Set the Name of the Button which should trigger the Animation.
- The Animation Start and End values has to be set.
- The Animation duration has to be set in milli seconds (1000 means 1 sec)
- The Animation speed has to be set to any value more than 0.
- The Toggle Animation Direction property has to be enabled.

The above figure shows a circular bitmap, which is split into 8 sectors. 6 sectors have to function as buttons. To achieve the above feature, we need 6 button widgets. One for each of the below sectors

- Image

- Container
- Clock
- EditField
- ProgressBar
- Slider

When the Star Bitmap is selected, the Animation has to be triggered. To achieve this, another buttonwidget has to be associated to the Star Bitmap. When the ButtonWidget associate to the Star Bitmap is selected, the button widget will fire a ButtonReactionMsg message. This message has to be handled in the ViewController of the current view. In this case, the Animation has to be started on the Button Release event. So the view controller should handle the Message fired by the ButtonWidget and based on the ButtonName StartAnimationMessage or StopAnimationMessage message has to be posted. This message should contain the name of the AnimationWidget2D that is associated to the WheelBackGround GroupNode.

All the Buttons are configurerd as sector based buttons. Refer to the below snap shot for the widgets and Node mapping used in the scene composer.



Figure 7.2.: Nodes Exampe 1

Refer to the below snap shot for configuration the Animationwidget.

Requirements

See Wiki documentation: Wiki AnimationWidget2D.

Properties

See Wiki documentation: Wiki AnimationWidget2D.

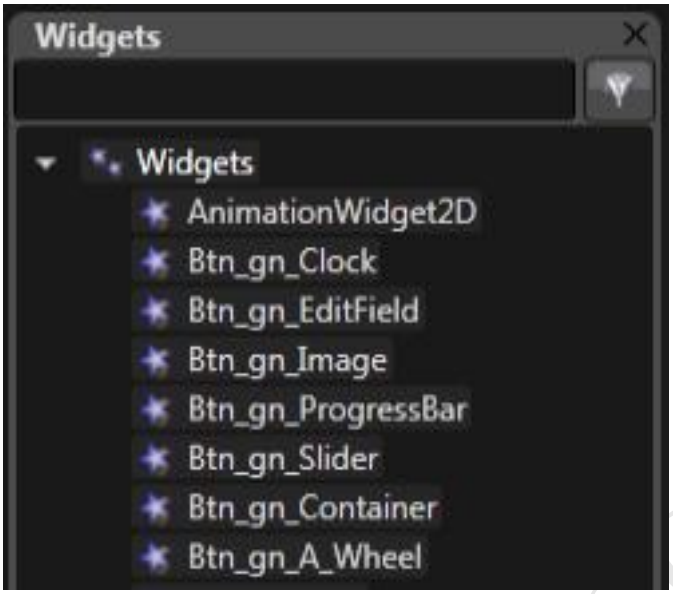


Figure 7.3.: Nodes Exampe 2

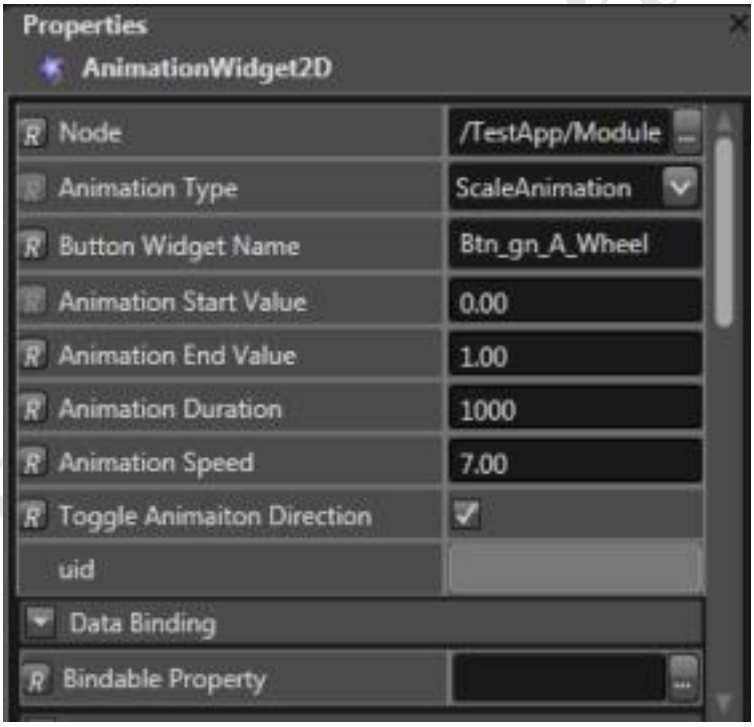


Figure 7.4.: Animation widget configuration

User Guide

See Wiki documentation: Wiki AnimationWidget2D.

7.3. BusyIndicator Widget 2D

7.3.1. Introduction

The BusyIndicatorWidget2D is a generic 2D widget and is used to graphically represent and give a visual feedback to the user, when an application takes some time for loading assets or if the application wishes to show that some other processing is going on. Typically the widget is used as wait animation.

Examples Use Cases



Figure 7.5.: Sample of BusyIndicator that shows the widget in work



Figure 7.6.: Sample of BusyIndicator that shows the widget in work



Figure 7.7.: Sample of BusyIndicator that shows the widget in work

Requirements

- For Incrementally circular type of Busy Indicator, a minimum of 2 Image Nodes has to be present as a child of a GroupNode associated to the widget.
- For Circular Busy Indicator, a GroupNode, which has an ImageNode2D as a Child Node of the GroupNode should be associated to the Widget. The Image to be displayed for the Circular Busy Indicator should be associated to the ImageNode2D in the scene composer.
- For Incremental Circular Busy Indicator, there should be a set of ImageNodes arranged in a Circular pattern. These ImageNodes are added as a Child of a GroupNode. This GroupNode is associated to the Widget. These ImageNodes should have BitmapBrushColorBlend effect associated, and not BitmapBrushBlend.
- In order to use Horizonal Busy Indicator Widget, the image which should be animated as shown in Figure 2 should be associated to an ImageNode in Scenecomposer. This Image Node should be placed under a GroupNode. This GroupNode should be associated to the Widget. An additional camera should be used for Horizontal Busy Indicator. The camera should be added as the Next sibling after the Image Node as shown in the below figure. Some properties of the camera should be set to use the horizontal indicator widget.

Pre-Release

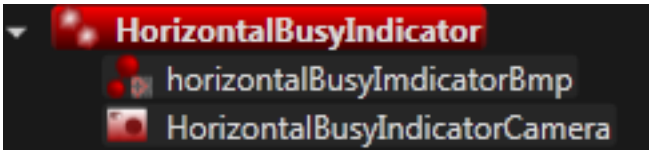


Figure 7.8.: Horizonal Busy Indicator settings



Figure 7.9.: Properties of the camera



Figure 7.10.: Properties of the camera

Pre-Release

Pre-Release

2584 **Properties**

Name	*Type*	*Range*	*Explanation*
Node	ISceneNodeBase2D		The associated root node associated to the widget.
Data Binding Definitions	custom:DataBindingEditorPlugin		Define bindings from data items in the model to bindable properties of this widget.
Enable	Boolean		Widget is enabled or disabled.
Focusable	Boolean		Widget is Focusable or not.
IsPartOfControlBar	Boolean		Widget is part of PushUp Container.
ExitPushUpOnRelease	Boolean		If set to TRUE, OnRelease event on Widget which is part of PushUp container, makes PushUp Invisible, If set to FALSE PushUp will never be made invisible.
Touchable	Boolean		Widget is Touchable or not.
ItemOrder	UInt32		Widget is child of any other widget and what is the order of that widget placed inside a list.
ItemFocusOrder	UInt32		ItemFocusOrder
FocusOrder	UInt32		Focussing of widget is based on rank.
FocusBitmapNode	ISceneNodeBase2D		Texture to be rendered should be associated to this property.
AssociatedContainerFocusOder	UInt32		Added for Multi container, all the widgets which are under container widget should enter its parent container's Focus Order.

2585

Pre-Release

User Guide

Circular Busy Indicator - Configuration *Select CircularBusyIndicator from the drop down menu for the property Busy Wait Indicator type. *The Bitmap which has to be animated in Circular Busy Indicator type has to be imported in scene composer and associated to a ImageNode2D. The ImageNode2D has to be placed inside a GroupNode.This GroupNode is associated to the widget. *Set the different Animation properties like speed factor, animation direction. The Widget will be created an animation depending on the type of BusyIndicatorType selected and the ImageNode will be animated.

Horizontal Busy Indicator - Configuration o Selected HorizontalBusyIndicator option from the drop down menu for the property Busy Wait Indicator type. o The Bitmap which has to be animated should be imported in scene composer and associated to a ImageNode2D. This ImageNode2D has to be placed inside a GroupNode. o The ImageNode should be placed such that the position should be outside the boundary of the Main Camera viewport which captures the view and renders on the target. o The Clear Color Buffer should be set to TRUE and Enable Swapping set to FALSE for the Main Camera which captures the view and renders on target. o Add a camera as Next sibling to the Group Node as shown in the below snap shot and set the Enabled Swapping to TRUE and Clear Color Buffer to FALSE. o The position of the ImageNode and the Camera (Camera_horizontalBusyIndicator) should be same. o Set the Height of the ViewPort of the Camera same as the height of the ImageNode. o Set the Width of the ViewPort of the Camera lesser than the width of the ImageNode. o Set the Left and Top position of the ViewPort of the Camera, which should be same as the point where the ImageNode has to be rendered. o Add the Camera to the DisplayRenderTarget. o Set the Enable Swapping property of both the cameras to TRUE.

Incremental Circular Busy Indicator - Configuration o Selected IncrementalCircularBusyIndicator option from the drop down menu for the property Busy Wait Indicator type. o Arrange the position of the ImageNodes in SceneComposer as required which has to be animated using the widget.

Place them inside a GroupNode. Associate the GroupNode to the widget.

o Set the IncrementalCircularNodeCount property to the number of ImageNodes present in the GroupNode. o The Widget will create an animation which will change the Alpha value of each ImageNode. o If Enable Scaling options is selected, then the Nodes will be scaled from 0 to 1 along the animation duration.

Text - Configuration o To display a Text along with the BusyIndicator Animation, a TextWidget2D should be used. o The TextWidget2D should be passed to the BusyIndicatorWidget2D as a parameter to the property \"TextWidget\" o Once the TextWidget2D is associated, the Text which has to be displayed can be set in the Text property of the Widget. o After the BusyIndicatorAnimation is started and the Animation completes twice, the Text will be appended with \".\" and after the third cycle, the text will be appended with \"..\". Similarly after the fourth cycle the Text will be appended with \"...\".Then the cycle repeats.

Triggering the BusyIndicator Animation o The Application post the StartAnimationMessage or StopAnimationMessage courier message to start or stop the BusyIndicator Animation. o The message should contain the name of the BusyIndicatorWidget as given in the scene composer as the parameter for the courier message.

Pre-Release

7.4. Button Widget 2D

7.4.1. Introduction

The ButtonWidget2D is a widget that provides functionality for touching an area and executing a certain action. It supports the following states:

* Disabled * Normal * Pressed * Selected.

ButtonWidget2D reacts to user action by change perhaps the image and sending a reaction message for press and release (optional: short- and longpress)

Examples

Figure 7.11.: Sample of Button that shows the widget in work

Requirements

Properties

User Guide

Figure 7.12.: The ButtonWidget2D

7.5. Calendar Widget 2D

7.5.1. Introduction

CalendarWiget2D is used to display a monthly calendar with a background image. It is a 7 by 7 grid. The first row displays the weekdays as header. The starting weekday can be configured. The given date (day, month and year) specifies the current month to be displayed. The current day within that month is marked with an additional background image. Some days of the previous and next month are always visible, shown in a different text color. It is guaranteed that at least one day of the previous month is shown meaning the first day of the current month is not displayed in the first cell.

Examples Use Cases

Requirements

- The CalendarWidget2D has to be attached to a group node

Pre-Release



Figure 7.13.: Sample of Calendar that shows the widget in work

- 3 child nodes (RenderNode) with a BitmapBrush have to be added to that group for background image, cursor image and one for the rendered text. Note: instead of having 49 text nodes, the widget uses one single bitmap node where it renders the text manually

Pre-Release

Pre-Release

Properties

Name	*Type*	*Range*	*Explanation*
Starting Weekday	Integer	0..6	Index for the starting weekday (0: Sunday, 1: Monday).
Day	Integer	1..31	The day value.
Month	Integer	1..12	The month value.
Year	Integer	1970..2032	The year value.
Style	Text Style		The text style to be used.
Background Image	Image		The image for the background.
Cur Day Background Image	Image		The background image for the current day.
Weekday List	String List		A list of translatable strings for the weekdays, starting with Sunday.
Weekday Color	Color		Text color for weekdays.
Active Month Color	Color		Text color for the active month.
Active Day Color	Color		Text color for the active day.
Inactive Month Color	Color		Text color for the inactive (previous and next) month.
Offset	Vector2		Position for the upper left cell (the first weekday). The y-coordinate represents the center of the cell.
Weekday gap	Integer		Additional gap between the weekday row and the calendar days (if needed).
Interval	Vector2		X and Y pixel interval for the cell content.

Pre-Release

User Guide

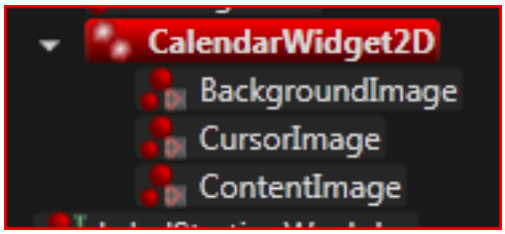


Figure 7.14.: Scene tree: group node with 3 bitmap nodes as child nodes

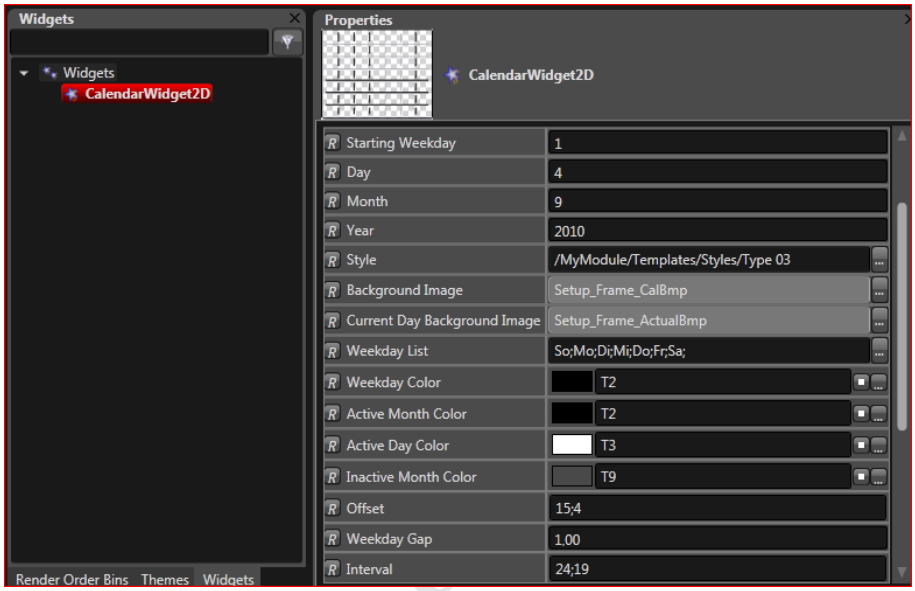


Figure 7.15.: The CalendarWidget2D with properties

7.6. Clock Widget 2D

7.6.1. Introduction

ClockWiget is used to display an analogue clock with a clock face as background image and two clock hand images for hour and minute. The clock is skin-able, meaning the complete set of images (clock face, hour hand and minute hand) can be changed during runtime.

Examples Use Cases

Requirements

Detail requirements are located in Widget Wiki, ClockWidget2D



Figure 7.16.: Sample of Clock that shows the widget in work

Properties

Detail requirements are located in Widget Wiki, ClockWidget2D

User Guide

Detail requirements are located in Widget Wiki, ClockWidget2D

7.7. Container Widget 2D

7.7.1. Introduction

Widget's duty of the Container Widget is to completely keep track of its childs. Here in ContainerWidget2D this has an additional feature of delegating the Focus request message to the framework and ultimately framework will invoke the respective child widget focus related callbacks.

Concept

The ContainerWidget2D uses the concept of Focus Management in order to Focus a widget based on its Focus order. Basically it handles focusing a widget having focus order greater than zero within the Container group.

The ContainerWidget2D maintains a map which contains the focus order number and widget name. This map is populated based on the the widgets present within the Container_Group group node to which the ContainerWidget2D is associated.

The widget provides two properties a) Remember last Focus b) Wrap around, in order acheive different behaviour as per requirement.

Pre-Release

If the remember last focus is enabled then the widget stores the focus order of last focused widget and delegates focus to that particular widget when the scene is rendered again, otherwise the widget having least focus order is focused whenever transition occurs to that scene.

If Wrap around is enabled, whenever the focused widgets focus order is equal to the maximum focus order, on TURN_POSITIVE event the focus order is set to least focus order and vice versa.Thus a cyclic order is followed in this case. Whereas if Wrap around is disabled the focus remains unchanged in this above specifed scenario.

By default it focuses a node having least focus order(greater than zero) in the Container group. Based on rotary, joystick, or touch events and focus order it determines the next widget to be focused and delegates focus to that particular widget.

Input for ContainerWidget2D to change the focused widget is the RotaryMsgForContainerWidget message and TouchMsgForContainerWidget posted by the base widget.

Examples Use Cases

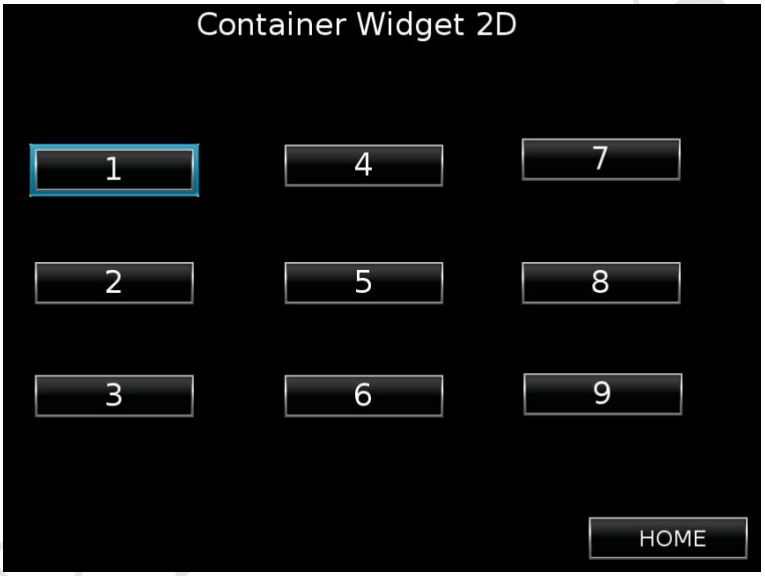


Figure 7.17.: Sample of Container that shows the widget in work

Requirements

For detail requirements see Widget Wiki: ContainerWidget2D

Properties

For detail properties listing see Widget Wiki: ContainerWidget2D

User Guide

For detail implementors guide see Widget Wiki: ContainerWidget2D

7.8. ControlBar Widget 2D

7.8.1. Introduction

The controlBar is a special container which is placed at the bottom of screen. It contains buttons which can have a sub menu in form of a PushUp List.

Examples



Figure 7.18.: The ControlBarWidget2D

Requirements

For detail list of requirements see Widget Wiki: ControlBarWidget2D

Properties

For detail properties listing see Widget Wiki: ControlBarWidget2D

User Guide

For detail properties listing see Widget Wiki: ControlBarWidget2D

Pre-Release

7.9. EditField Widget 2D

7.9.1. Introduction

EditFieldWidget2D is is used to display text triggered by user based on various input events unlike textwidget. Widget supports text display orientation from left to right.

Provide support for string upto a maximum length of 255 characters.The Edit Field widget provides an area which displays entered characters by the user.

Font types, color, styles shall be provided as an input for this widget configuration.

For details refer to widget Wiki: EditFieldWidget2D

Concept

The EditFieldWidget2D basically works in two modes,namely FreeTextMode and MatchTextMode.

FreeTextMode : In this mode the widget uses string(having one char) input from speller/center stack via a binding source update.

In addition to string input to be displayed following functionalities can also be acheived using the same binding source.

o Delete Char - A single char before the cursor position is deleted. Binding source update required for this is ”0x232B”.

o Delete All - Clears the entire editfield string. Binding source update required for this is ”0x2327”.

o Short Hold delete - Deletes all the chars before the cursor position. Currently this feature is not exposed for usage to projects but can exposed on request.

o Shift Cursor Left - Moves the cursor to left by one char position. Binding source update required for this is ”0x2190”.

o Shift Cursor Right - Moves the cursor to right by one char position. Binding source update required for this is ”0x2192”.

Functionalities like Shifting Cursor left and right can also come from rotary encoder and processed by the widget. The widget maintains a EditField String by inserting/deleting the char from the string maintained by the widget before performing insert/ delete operation.

MatchTextMode : In this mode the only inputs for widget would from binding sources updated by application. Widget would use this binding source and display the Entry and match text as per widget configuration.

Binding sources that need to be configured for this are : EntryTextReceived and SuggestionText, both these properties are of type Candera::String.

The widget internally uses following widgets :

o TextEffectWidget2D is used for the dealing with text string.

Pre-Release

o ImageWidget2D is used to display images for cursor and background for editfield.

The widget maintains following two indexes :

o Visible start index - is the index of the first visible char in the string.

o Cursor index - specifies the no. of chars to left of cursor in complete EditField string.

In addition the widget also uses visible char count to maintain the number of visible chars.

Utf8 encoding is being used by widget for the text string.

Examples

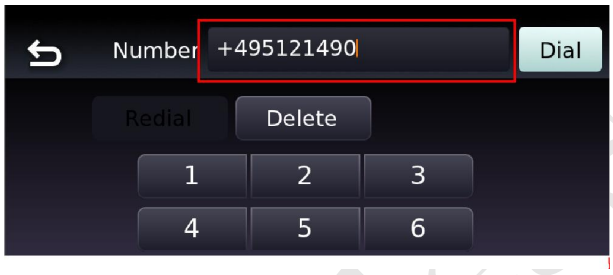


Figure 7.19.: Sample of EditField that shows the widget in work

Requirements

For list of requirements refer to widget Wiki: EditFieldWidget2D

Properties

For list of properties refer to widget Wiki: EditFieldWidget2D

User Guide

For list of properties refer to widget Wiki: EditFieldWidget2D

7.10. FlexList Widget 2D

The FlexListWidget2D is a dynamic list widget mainly for large lists where it is not possible to access all list items directly. Only one data window is requested from the business logic (e.g. 25 items) which allows displaying some of them. In addition the widget is also able to serve static menues, where the configuration is stored in the data model.

The complete FlexList widget collection was created to handle dynamic lists but the various widgets can be used for general purposes too (Switch, Toggle, Set an overlay layouter on a composite node, etc).

Pre-Release

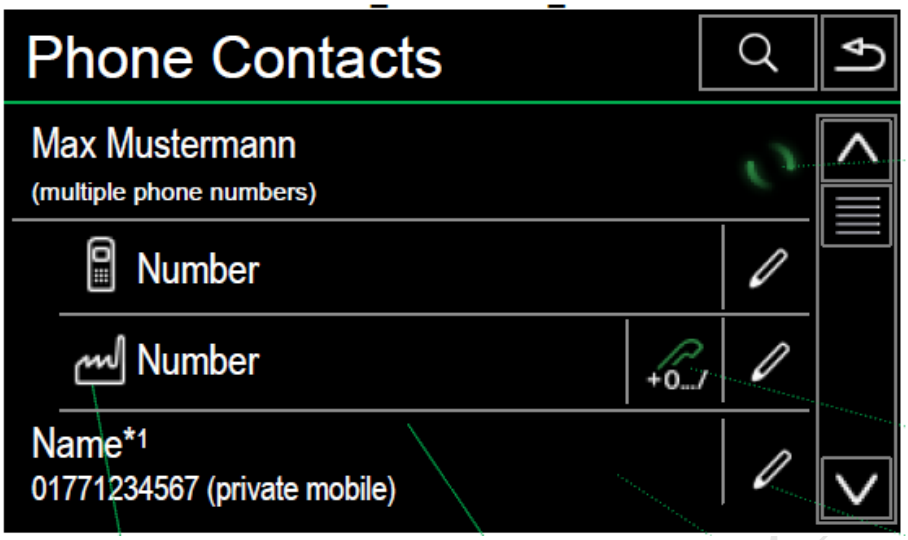


Figure 7.20.: Sample of a view with FlexList based on templates

More information can be found in the Widgets User Guide.

7.11. FlipBook Widget 2D

7.11.1. Introduction

FlipBookWidget2D is used to switch between images like a FlipBook. Bitmaps to use for BitmapBrushes have to be provided within an asset. Switching between images can be done using the corresponding functions or automatically by using an internal animation.

Examples Use Cases

The Turn-By-Turn arrow display of navigation is a flipbook application.



Figure 7.21.: Sample of FlipBook that shows the widget in work

Pre-Release

Requirements

For a detailed list of requirements see Widget Wiki, FlipBookWidget2D

Properties

For a detailed list of properties see Widget Wiki, FlipBookWidget2D

User Guide

For a detailed description of using this widget in your screen see Widget Wiki, FlipBookWidget2D

7.12. GlowEffect Widget 2D

7.12.1. Introduction

GlowEffectWidget2D is used to display to give the content of a touchable widget like e.g. ButtonWidget2D an effect that the pressed item is glowing around its icon.

Example

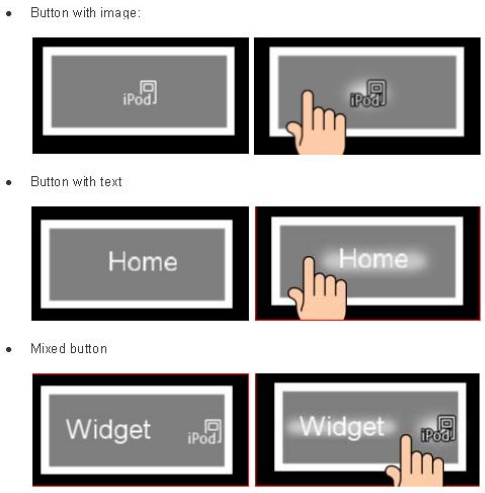


Figure 7.22.: Sample of GlowEffect that shows the widget in work

Requirements

For a detailed description of using this widget in your screen see Widget Wiki, GlowWidget2D

Pre-Release

Properties

For a detailed description of using this widget in your screen see Widget Wiki, GlowWidget2D

User Guide

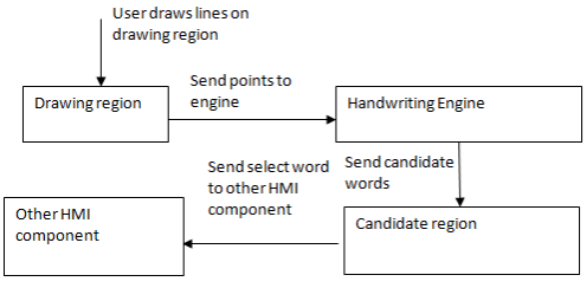
For a detailed description of using this widget in your screen see Widget Wiki, GlowWidget2D

7.13. HandWritingRecognition Widget 2D

7.13.1. Introduction

The HWR Widget is a 2D Widget which can be used to input characters for different languages from the HMI. Input mode is through, free hand drawing on the touch screen.

Concept



Examples

For a detailed examples see Widget Wiki, HandWritingRecognitionWidget2D

Requirements

For a detailed description requirements see Widget Wiki, HandWritingRecognitionWidget2D

Properties

For a detailed description of properties see Widget Wiki, HandWritingRecognitionWidget2D

Pre release

User Guide

For a detailed description of using this widget in your screen see Widget Wiki, HandWritingRecognitionWidget2D

7.14. Image Widget 2D

7.14.1. Introduction

A Imagewidget2D is used to display an image. It can be used to load and display an image statically or dynamically. In addition it also loads image from the asset as well as from any external source outside the asset. The purpose is to display a bitmap to the user in a presentable format(bmp, jpeg, gif, png and other image format).

Concept

The ImageWidget2D basically works in two modes,

1. Displaying Asset Image - Here the image to be loaded and displayed is a part of the Asset from SceneComposer. The widget obtains the image from the asset and displays it using Effect2d and BitmapImage2D.

2. Displaying External Image - Here the image to be loaded and displayed is not a part of the asset and is taken from an external source (say a folder, USB, etc). In this case the widget uses the DevIL (Developers Image Library) an open source library to obtain the image and load it.

Examples



Figure 7.23.: The used talking head image.



Figure 7.24.: The used hang up image at the bottom.

Pre-Release

Requirements

For detailed list of requirements refer to widget Wiki ImageWidget2D

Properties

For detailed list of widget's properties refer to widget Wiki ImageWidget2D

User Guide

For detailed description of implementation guidelines for this widgets refer to widget Wiki ImageWidget2D

7.15. IrisBargraph Widget 2D

7.15.1. Introduction

IrisBargraphWidget2D is used to display a cluster of several bar graphs embedded into a circular shape. The Iris is composed of 18 individual bar graphs. Each of the bar graph can have exactly 5 state machines as: empty, fill as 25%, 50%, 75% or 100%. The Iris also defines one cursor represented by a white image. The cursor purpose is to visually mark the bar graph segment currently being updated/overwrite.

Examples Use Cases

Next to pictures shows an empty and a full configured Iris



Figure 7.25.: Sample of IrisBargraph that shows an empty Iris

Requirements

- The IrisBargraphWidget2D has to be attached to group node called 'Segments'

Pre-Release



Figure 7.26.: Iris with all 18 segments sets to values different by (-1)

- The IrisBargraphWidget2D has to be attached to group node called 'Segments'
- Segments group node, contains 18 child nodes (RenderNode) each one having a BitmapBrushBlend
- A list of threshold values can be configured to define different fill levels for each bar graph Having for example 3 threshold values (30; 60; 80); Values: (-1) bar graph is empty. Values within [0 - 30] the first fill level is used. Values within [31 - 60] the second fill level is reached. Values within [61 - 80] the third fill level is reached. Values within [81 - 100] the forth fill level is reached - segment is 100% full.
- A list of values for each bar graph segment. Because we have 18 segments we need 18 values. If values are not provided to the list, then by default the value is (-1) which means an empty bar graph;
- A cursor position to mark the segment being update
- An inactive count value to designate how many segments are marked as inactive
- Number of segments, which is set to 18
- Cursor Image / Active image and Inactive image - consult the below table for a more detailed explanation

Properties

Name	*Type*	*Range*	*Explanation*
Node	Node*		The Segments group node contains 18 bitmap nodes as children.
Cursor Position	int	0..17	Defines which bar graph segment has the cursor. This is a bindable property.

Pre-Release

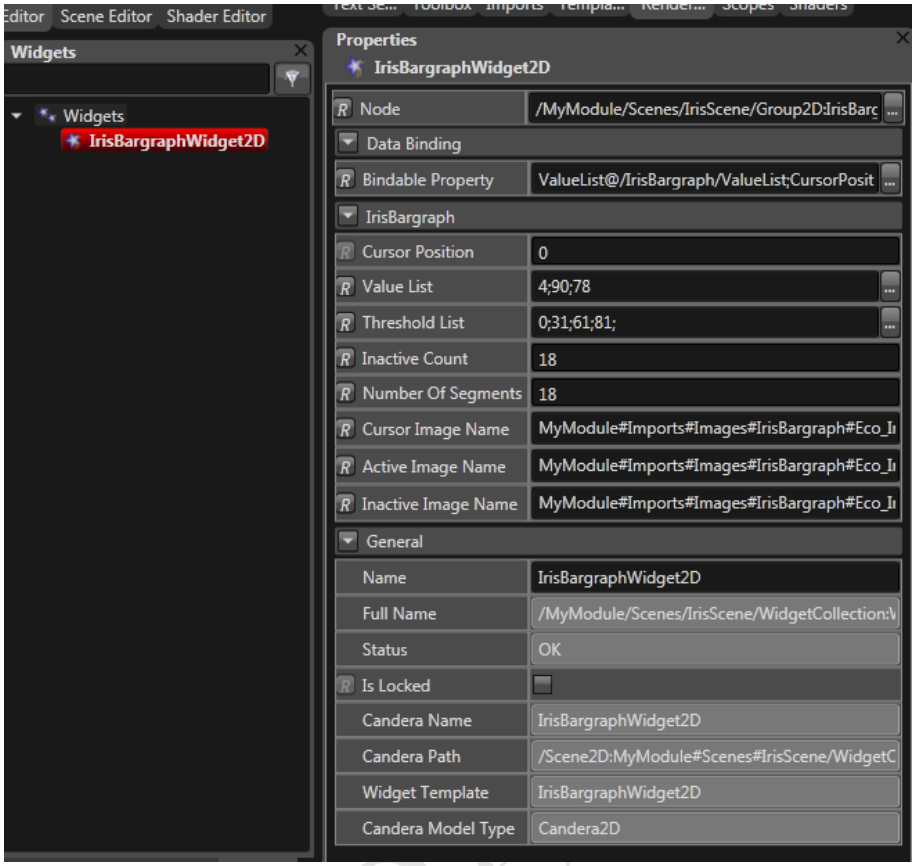


Figure 7.27.: The IrisBargraphWidget2D

User Guide

7.16. LaneGuidance Widget 2D

7.16.1. Introduction

LaneGuidanceWidget2D is used to display ....

Examples



Figure 7.28.: Sample of LaneGuidance that shows the widget in work

Pre-Re

Requirements

Properties

User Guide

7.17. List Widget 2D

7.17.1. Introduction

ListWidget2D is used to display ....

Examples Use Cases



Figure 7.29.: Sample of List that shows the widget in work

Requirements

Properties

User Guide

7.18. Menu Widget 2D

7.18.1. Introduction

MenuWidget2D is used to display ....

Examples Use Cases



Figure 7.30.: Sample of Menu that shows the widget in work

Pre-Release

Requirements

Properties

User Guide

7.19. MulticolorBargraph Widget 2D

7.19.1. Introduction

MulticolorBargraphWidget2D is used to display a single bar graph. The color can be different depending on the current fill level (multi color). For graphical representation three different image sets have to be configured. One image is needed for the low edge, one image for the high edge and on image for the center which is stretched to fill the remaining space.

Examples

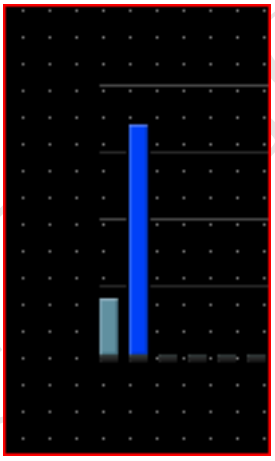


Figure 7.31.: Sample of MulticolorBargraph that shows a vertical widget



Figure 7.32.: Sample of MulticolorBargraph that shows a horizontal widget

Requirements

For detailed list of requirements refer to widget Wiki MulticolorBargrahWidget2D

Pre Release

Properties

For detailed list of widget's properties refer to widget Wiki MulticolorBargrahWidget2D

User Guide

For detailed description of implementation guidelines for this widgets refer to widget Wiki MulticolorBargrahWidget2D

7.20. Notebook Widget 2D

7.20.1. Introduction

NotebookWidget2D is used to display ....

Examples Use Cases



Figure 7.33.: Sample of Notebook that shows the widget in work

Requirements

Properties

User Guide

7.21. ODR Widget 2D

7.21.1. Introduction

ODRWidget2D is used to display ....

Pre-Release



Figure 7.34.: Sample of ODR that shows the widget in work

Examples Use Cases

Requirements

Properties

User Guide

7.22. OPS Widget 2D

7.22.1. Introduction

OPSWidget2D is used to display ....

Examples Use Cases



Figure 7.35.: Sample of OPS that shows the widget in work

Requirements

Properties

User Guide

7.23. ProgressBar Widget 2D

7.23.1. Introduction

ProgressBarWidget2D is used to display ....

Pre-Release



Figure 7.36.: Sample of ProgressBar that shows the widget in work

Examples Use Cases

Requirements

Properties

User Guide

7.24. RichTextBase Widget 2D

7.24.1. Introduction

RichTextBaseWidget2D is used to display ....

Examples Use Cases



Figure 7.37.: Sample of RichTextBase that shows the widget in work

Requirements

Properties

User Guide

7.25. Scrollable Widget 2D

7.25.1. Introduction

ScrollableWidget2D is used to display ....

Pre-Release



Figure 7.38.: Sample of Scrollable that shows the widget in work

Examples Use Cases

Requirements

Properties

User Guide

7.26. ScrollableText Widget 2D

7.26.1. Introduction

ScrollableTextWidget2D allows the display of multiline line, multicolored text. Main features

- Colored text - change text color based on special escape sequences in the text
- Tab support - jump to a given tab position based on special escape sequences in the text
- Scrolling text - allows to scroll text that is otherwise to big to display
- Word wrapping - automatically wraps words or characters that doni¿½t fit into the current line

ScrollableTextWidget2D should only be used if at least one of the following features is used since the multicolor rendering is quite CPU intensive compared to normal text rendering.

Examples Use Cases

This image shows the first page of formated text in a 500x200 pixel text area. As input Text the following string is used:

”This is a very long line that will wrap around if configured properly and will result in 3 lines total A new line \t1Tab1\t2Tab2 \t2Tab2 \c2color2\c3color3\c4color4 ...”

Requirements

- Each ScrollableTextWidget2D has to be attached to a RenderNode with BitmapBrushBlend-Effect or BitmapBrushColorBlend-Effect attached.
- No image must be set on the Effect.
- For correct rendering the ’Color Blend Factor Src’ property of the BitmapBrush-Effect must be set to ’One’.

Pre-Release

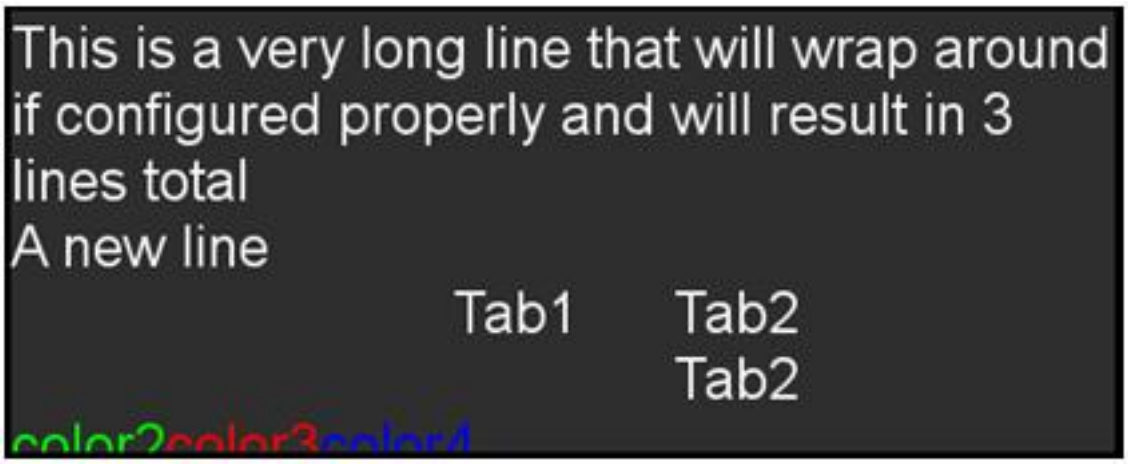


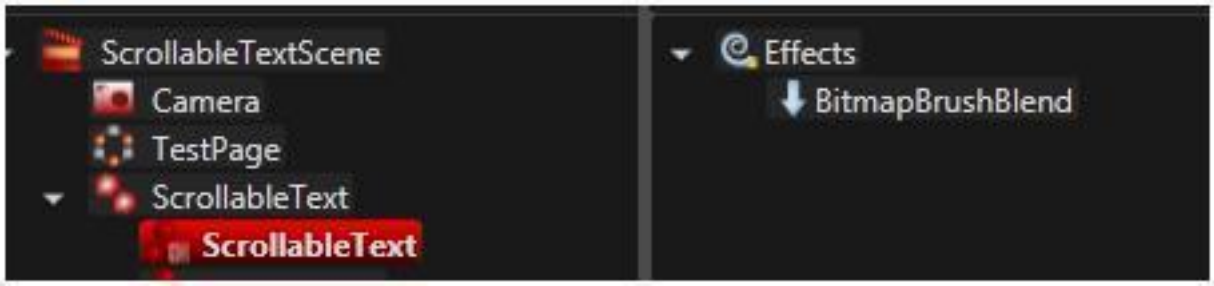
Figure 7.39.: Sample of ScrollableText that shows the widget in work

- If the ”Alpha Value” property of the widget is to be used the BitmapBrushColorBlend-Effect must be set.

Properties

User Guide

1.) Setup The widget needs to be attached to a RenderNode with either a ’BitmapBrushBlend’ or ’BitmapBrushColorBlend’ effect attached to it.



To render the text correct the ’Color Blend Factor Src’ property needs to be changed to ’One’. Also no image must be set.

2.) Writing multiline text To write multi line text in SceneComposer SHIFT+ENTER can be typed in the ’Text’ field of the widget to add a line break. For files it is sufficient to add normal line breaks.

3.) Colored Text The ’Color Table’ will be used by the widget to change the color of the text whenever a ’\cx’ command sequence is found in the input text. As long as no color is set in the text the ’Normal Text Color’ will be used.

Release

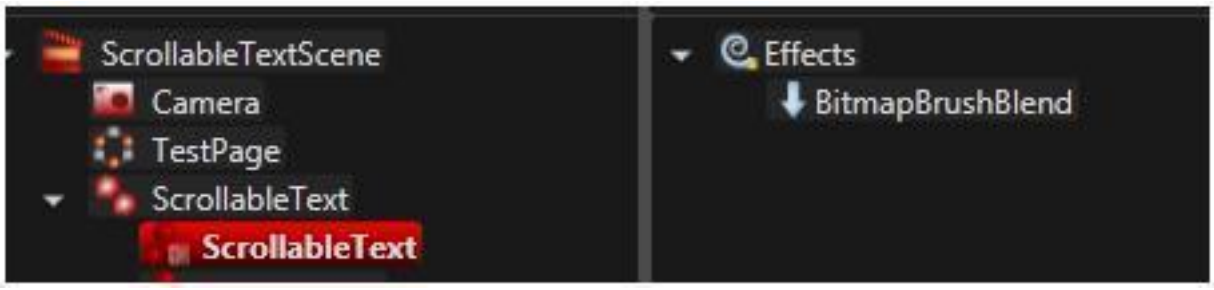
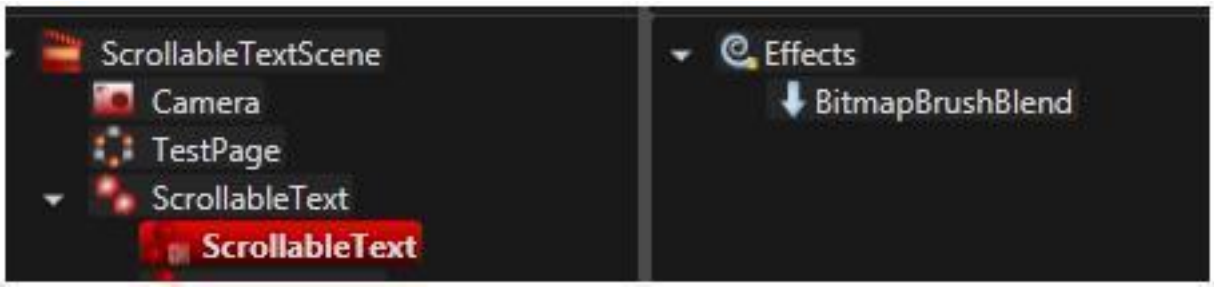
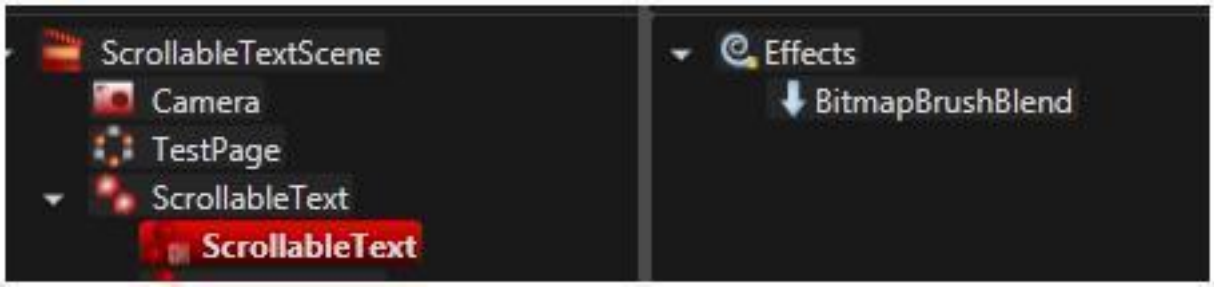


Figure 7.40.: The ScrollableTextWidget2D





7.27. ScrollBar Widget 2D

7.27.1. Introduction

ScrollBarWidget2D is used to display ....

Examples Use Cases



Figure 7.41.: Sample of ScrollBar that shows the widget in work

Requirements

Properties

User Guide

7.28. SequenceImage Widget 2D

7.28.1. Introduction

SequenceImageWidget2D is used to display ....

Examples Use Cases



Figure 7.42.: Sample of SequenceImage that shows the widget in work

Requirements

Properties

User Guide

7.29. Slider Widget 2D

7.29.1. Introduction

SliderWidget2D is used to display ....

Examples Use Cases



Figure 7.43.: Sample of Slider that shows the widget in work

Pre-Release

Requirements

Properties

User Guide

7.30. Speller Widget 2D

7.30.1. Introduction

SpellerWidget2D is used to display ....

Examples



Figure 7.44.: Sample of Speller that shows the widget in work

Requirements

Properties

User Guide

7.31. Splitter Widget 2D

7.31.1. Introduction

SplitterWidget2D is used to display ....

Examples Use Cases



Figure 7.45.: Sample of Splitter that shows the widget in work

Pre-Release

Requirements

Properties

User Guide

7.32. Switcher Widget 2D

7.32.1. Introduction

SwitcherWidget2D is used to display ....

Examples Use Cases



Figure 7.46.: Sample of Switcher that shows the widget in work

Requirements

Properties

User Guide

7.33. Text Widget 2D

7.33.1. Introduction

The TextWidget2D is an extension of FEAT delivered TextEffectWidget with additional interfaces and features. So, TextWidget2D provides all the features of TextEffectWidget, in addition it provides features like Truncation direction, scrolling, states

TextWidget2D acts as a adapter for the TextEffectWidget so that the interfaces of the TextEffectWidget can be reused to achieve the required functionality. TextWidget2D mainly provides the required bindable properties(details see below).

Additional Features like Rolling of Text, Truncation Direction, MultiLine, Word Wrap has been implemented in order to enhance the scope of the widget.

The Widget basically deals with manipulation of text string internally in order to acheive these functionalities.

Pre-Release



Figure 7.47.: Sample of Text that shows the widget in work

3007 **Examples Use Cases**

3008 **Requirements**

3009 **Properties**

3010 **User Guide**

3011 7.34. TextArea Widget 2D

3012 7.34.1. Introduction

3013 TextAreaWidget2D is used to display ....

3014 **Examples Use Cases**



Figure 7.48.: Sample of TextArea that shows the widget in work

3015 **Requirements**

3016 **Properties**

3017 **User Guide**



Figure 7.49.: The TextAreaWidget2D

Pre-Release

7.35. Timer Widget 2D

7.35.1. Introduction

The TimerWidget2D provides interfaces to the TimerUtiliy thereby allowing to start, stop and pause a timer.

TimerWidget2D is derived from the Candera::Widget2D and can be used to manipulate a timer from a scene. On expiry of the timer TimerWidgetExpiryMsg along with the ViewId and the Widget name is posted. For the details about the widget properties and about configuring the same in the SceneComposer, please refer below.

Expiry Behavior

On the timeout of the timer running from a TimerWidget2D a "TimerWidgetExpiryMsg" is posted from the TimerWidget2D. This message will contain the ParentViewId(parent view name) and the WidgetId(name of the widget as in SceneComposer solution) as parameters. This message also contains the Model, View and the Controller components as subscribers. Hence, this message can be used as the timer identifier to check the timer expiry.

DataBindable Properties

This section describes the various properties of the TimerWidget2D which can be binded to the BindingSources from the HMIContract.

o TimerStatus o StartStopTrigger o PauseTrigger

Note: A TimerWidget2D need not be assigned with any node as the widget does not perform any scene manipulation operation.

Examples Use Cases

Requirements

Properties

User Guide

7.36. TransitionMarker Widget 2D

7.36.1. Introduction

TransitionMarkerWidget2D is used to display ....

Pre-Release

Examples



Figure 7.50.: Sample of TransitionMarker that shows the widget in work

Requirements

Properties

User Guide

7.37. TwoDimensionSlider Widget 2D

7.37.1. Introduction

TwoDimensionSliderWidget2D is used to display ....

Examples



Figure 7.51.: Sample of TwoDimensionSlider that shows the widget in work

Requirements

Properties

User Guide

7.38. VideoContainer Widget 2D

7.38.1. Introduction

The purpose of the Video container widget is to makes the video visible on video layer. Also the widget can take the inputs from rotary (encoder), key, touch and joystick and sends reaction events to Application for user interactions with the video.

Pre-Release

Concept

Three different layers includes

- Layer 0 - Video layer
- Layer 1 - HMI layer
- Layer 2 - Popup layer

Out of which, Video is made visible on the innermost video layer. Video which the container is going to make the video visible is drawn on the video layer by the Video Engine.

This widget will make the HMI layer transparent so that the video drawn on the video layer would be visible along with the other widgets and HMI elements on the display.

HMI Application will send the hardware inputs like joystick, key, touch and rotary encoder user inputs as courier messages (similar to touch messages). Widget will take these inputs as input events and sends their corresponding reaction events to the Application.

Example



Figure 7.52.: The VideoContainerWidget2D in work

Configuration

Step 1: Drag and drop a bitmap node to the scene tree of the intended scene .

Step 2: Associate or drag and drop a video container widget to the bitmap node

Step 3: Associate the Video Container group node to the Node.

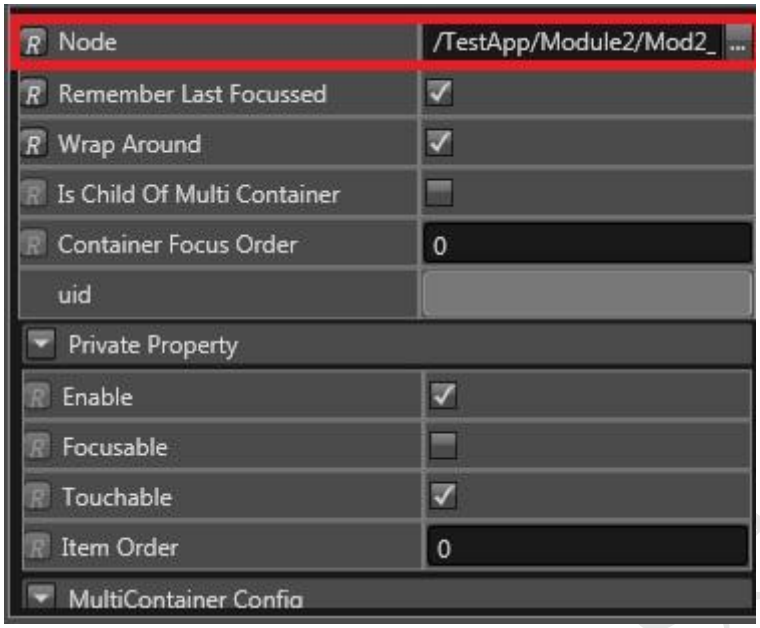


Figure 7.53.: VideoContainer properties window

Properties

User Guide

Figure 7.54.: The VideoContainerWidget2D

Pre-Release

Part V.

Appendix

Pre-Release

Pre-Release

8. List of Terms and Abbreviations

Term or abbreviation	*Description*
ASF	Automotive Service Framework
BL	Business Logic
Candera	CGI Render Engine
CCA	Car Computer Architecture
CGI	Computer Graphics Interface (?)
CMS	Car Multimedia Service
Courier	CGI Message Framework
DBus	D-Bus is a free and open-source inter-process communication (IPC) system, allowing multiple, concurrently-running computer programs (processes) to communicate with one another.
DLT	Genivi Diagnostic Log and Trace
FEAT	Fujitsu Semiconductor Embedded Solutions Austria GmbH
Genivi	Genivi is a non-profit industry alliance committed to driving the broad adoption of an In-Vehicle Infotainment (IVI) open-source development platform.
HALL	HMI Aptraction Logic Layer
HMI-Contract	XML based interface description
ILM	Genivi Ivi Layer Management
IPC	Inter Process Communication
IVI	In-Vehicle Infotainment
LCM	Life Cycle Management
MVC	Model View Controller
PM	Popup Manager
SB	Screen Broker
SPM	System Power Management
UML	Unified Modeling Language
WL	Wayland
WYSIWYG	What You See Is What You Get

Pre-Release

9. List of Internal Links

1. ProjectSettings.h
   - ai_hmi_cgi/components/Framework/AppBase/ScreenBrokerClient/
2. PROJECT_SBPLUGINS_POPUP_PROPERTIES.README
   - ai_hmi_cgi/components/Framework/AppBase/ScreenBrokerClient/
3. PROJECT_SBPLUGINS_PROTOCOL.README
   - ai_hmi_cgi/products/HMIBASE/ScreenBrokerPlugins/
4. XML-FORMAT.README
   - ai_hmi_cgi/products/HMIBASE/ScreenBrokerPlugins/ScreenLayouter/
5. buildScreenBrokerPlugins.sh
   - ai_hmi_cgi/products/HMIBASE/ScreenBrokerPlugins/
6. ProjectSettings.cpp
   - ai_hmi_cgi/components/Framework/AppBase/ScreenBrokerClient/
7. KeyMapping.cpp
   - /ai_hmi_cgi/products/HMIBASE/GlobalConfig/AppBaseSettings/
8. folder for project specific SB plug-ins
   - <PRJ_VOB>/products/<PRJ>/ScreenBrokerPlugins/
9. HMI-Base ScreenLayouts.xml
   - ai_hmi_cgi/products/HMIBASE/ScreenBrokerPlugins/ScreenLayouter/
10. project ScreenLayouts.xml
    - <PRJ_VOB>/products/<PRJ>/ScreenBrokerPlugins/ScreenLayouter/
11. package file
    - ai_hmi_cgi/config/packages/

Pre-Release

12. ProjectBaseTypes.xhcdl

    - <PRJ_VOB>/products/<PRJ>/Apps/Common/ProjectBaseContract/config/

Pre-Release

10. List of External Links

https://inside-wiki.bosch.com/confluence/display/HMITOOL/ControlBarWidget2D
https://inside-wiki.bosch.com/confluence/display/HMITOOL/GlowWidget2D
https://inside-wiki.bosch.com/confluence/display/cmaiasf/Automotive+Service+Framework+%28ASF%29
https://hi-dms.de.bosch.com/docushare/dsweb/Services/Document-661362
https://inside-wiki.bosch.com/confluence/display/HMITOOL/MulticolorBargrahWidget2D
https://hi-dms.de.bosch.com/docushare/dsweb/Get/Document-687825/Widgets%20User%20Guide.pdf
https://inside-wiki.bosch.com/confluence/display/HMITOOL/HandWritingRecognitionWidget2D
https://inside-wiki.bosch.com/confluence/display/HMITOOL/ContainerWidget2D
https://inside-wiki.bosch.com/confluence/download/attachments/224342537/Wiki_ClockWidget2D.docx?version=1&modificationDate=1386152850000
https://inside-wiki.bosch.com/confluence/pages/viewpage.action?pageId=212736863
https://inside-wiki.bosch.com/confluence/display/HMITOOL/FlipBookWidget2D
https://inside-wiki.bosch.com/confluence/display/gen3generic/HowTo+Popups#HowToPopups-CloseOnTouch
https://inside-wiki.bosch.com/confluence/display/HMITOOL/EditFieldWidget2D
https://inside-wiki.bosch.com/confluence/display/HMITOOL/Wiki_AnimationWidget2D
https://inside-wiki.bosch.com/confluence/display/gen3generic/Target+Debugging+with+gdbserver+and+Eclipse
https://inside-wiki.bosch.com/confluence/display/gen3generic/HowTo+Popups
https://hi-dms.de.bosch.com/docushare/dsweb/Get/Document-661390/IAR%20visualSTATE%20Usecases.pdf
https://inside-wiki.bosch.com/confluence/display/gen3generic/Using+DLT+in+G3g
https://inside-wiki.bosch.com/confluence/display/HMITOOL/ImageWidget2D
https://hi-dms.de.bosch.com/docushare/dsweb/Services/Document-490511
//bosch.com/dfsrb/DfsDE/DIV/CM/DI/Projects/HTC_SW-Distribution/Software/External_Suppliers/From_FEAT/CGI_Studio_Releases/License
https://hi-dms.de.bosch.com/docushare/dsweb/Get/Document-661383/XHCDL.chm
https://hi-dms.de.bosch.com/docushare/dsweb/Get/Document-151948/ExtendedTraceGeneratorManual.doc

Pre-Release

Todo list

Internationlisation need more text . . . 40
Add link to "click here" in Toolchain/Build System/Installation . . . 46
Add content to: How To/Build System/Setting up Environment . . . 47
Add content to: How To/Simple Application Sample or is this section not needed? . . . 47
Add info about "application startup mandatory request" in Framework Features/Application Framework/ HMIAppControl Service . . . 50
Screenshot functionality to be implemented . . . 53
Functionality of Sound Settings Service has to be extended to cover the whole AudioManager functionality . . . 54
to be removed, application has to provide a trace class for list tracing by its own . . . 56
Clarify: What is the impact of 'asynchronous' bindingSource? . . . 66
Ask Helmut to fill this section . . . 67
add example for statemachine dataguard definition in contract . . . 68
A skeleton generator to produce an application skeleton is not yet available . . . 68
clarify scope . . . 74
application setup has to be done manually as of now :-(, scripted solution will follow . . . 76
Add content to How To/ Setup/ Guidelines/ Gui application development . . . 81
Add content to How To/ Setup/ Guidelines/ Development with Scene Composer . . . 82
required extention of ProxyHandler class to be described here . . . 83
sat2hi/gs77hi: add howto for SBAnimations when screenbroker version is integrated, that solved Mantis 4496 . . . 87
Provide script/archive to create such wrapper scripts . . . 89

Pre-Release